

Módulo 4 - Optimización y rendimiento

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. **Proyecto:** Se profundiza en la optimización y el rendimiento del proyecto AceriaData, incluyendo el análisis del SQL generado, el control del tracking, la identificación y resolución de problemas de rendimiento, las Split Queries, las Compiled Queries, la paginación eficiente y las estrategias de diagnóstico.

Distribución del Módulo 4

12 puntos – 6 horas

Punto	Título	Duración
4.1	Análisis del SQL generado: ToQueryString y logging	30 min
4.2	Tracking y No Tracking	30 min
4.3	AsNoTracking y AsNoTrackingWithIdentityResolution	30 min
4.4	Problema N+1: identificación y causas	30 min
4.5	Solución a N+1: Include, proyecciones y Split Queries	30 min
4.6	Over-fetching: causas y soluciones	30 min
4.7	Consultas ineficientes: filtros no traducibles y funciones en Where	30 min
4.8	Split Queries: cuándo y cómo usarlas	30 min
4.9	Compiled Queries	30 min
4.10	Paginación eficiente: Skip/Take y keyset pagination	30 min
4.11	Diagnóstico con logs, métricas y herramientas	30 min
4.12	Estrategias de optimización y checklist de rendimiento	30 min

Punto 4.1 - Análisis del SQL generado: ToQueryString y logging

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que devuelvan el SQL generado por diferentes consultas. Configurar el logging de EF Core. Analizar el SQL de consultas con Where, OrderBy, Select e Include. Diagnosticar un problema de consulta ineficiente a partir del SQL generado.

Contexto del proyecto: En el punto 3.12 se consolidaron las buenas prácticas de acceso a datos, cerrando el Módulo 3. En este punto se inicia el Módulo 4 con el análisis del SQL generado, que es la base de la optimización. Esta técnica se usará en el punto 4.2 para el control del tracking y en el punto 4.3 para las variantes de No Tracking.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir métodos de análisis de SQL a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    string ObtenerSqlConsultasPendientes();
    string ObtenerSqlConsultasConInclude();
    string ObtenerSqlConsultasConProyeccion();
    string ObtenerSqlConsultasConFiltroGlobal();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: string ObtenerSqlConsultasPendientes(); → método que devuelve el SQL de las consultas pendientes.

Línea 11: string ObtenerSqlConsultasConInclude(); → método que devuelve el SQL de las consultas con Include.

Línea 12: string ObtenerSqlConsultasConProyeccion(); → método que devuelve el SQL de las consultas con proyección.

Línea 13: `string ObtenerSqlConsultasConFiltroGlobal();` → método que devuelve el SQL de las consultas con filtro global.

Línea 15: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver cadenas con el SQL.

■ Paso 3: Implementar los métodos de análisis de SQL en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```
public string ObtenerSqlConsultasPendientes()
{
    var consulta = _context.OrdenesFabricacion
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion);

    return consulta.ToQueryString();
}

public string ObtenerSqlConsultasConInclude()
{
    var consulta = _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .Where(o => o.Cliente == "Constructora del Norte");

    return consulta.ToQueryString();
}

public string ObtenerSqlConsultasConProyeccion()
{
    var consulta = _context.OrdenesFabricacion
        .Where(o => o.Estado == "Pendiente")
        .Select(o => new { o.NumeroOrden, o.Cliente });

    return consulta.ToQueryString();
}

public string ObtenerSqlConsultasConFiltroGlobal()
{
    var consulta = _context.OrdenesFabricacion
        .Include(o => o.Planchas);

    return consulta.ToQueryString();
}
```

Línea 1: `public string ObtenerSqlConsultasPendientes()` → declara el método.

Línea 3: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.Where(o => o.Estado == "Pendiente")` → filtra por estado.

Línea 5: `.OrderBy(o => o.FechaCreacion);` → ordena por fecha.

Línea 7: `return consulta.ToQueryString();` → devuelve el SQL sin ejecutarlo.

Línea 10: `public string ObtenerSqlConsultasConInclude()` → declara el método.

Línea 12: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 13: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 14: `.Where(o => o.Cliente == "Constructora del Norte");` → filtra por cliente.

Línea 16: `return consulta.ToQueryString();` → devuelve el SQL.

Línea 19: `public string ObtenerSqlConsultasConProyeccion()` → declara el método.

Línea 21: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 22: `.Where(o => o.Estado == "Pendiente")` → filtra por estado.

Línea 23: `.Select(o => new { o.NumeroOrden, o.Cliente })`; → proyecta a un tipo anónimo.

Línea 25: `return consulta.ToQueryString();` → devuelve el SQL.

Línea 28: `public string ObtenerSqlConsultasConFiltroGlobal()` → declara el método.

Línea 30: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 31: `.Include(o => o.Planchas);` → incluye la colección de planchas.

Línea 33: `return consulta.ToQueryString();` → devuelve el SQL.

Error común: si se llama a `ToList` antes de `ToQueryString`, la consulta se ejecuta y `ToQueryString` no funciona. Se debe llamar a `ToQueryString` directamente sobre el `IQueryable`.

▮ Paso 4: Crear el caso de uso de análisis de SQL

Crear el archivo `src/AceriaData.Application/UseCases/AnalisisSqlUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class AnalisisSqlUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public AnalisisSqlUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== ANÁLISIS DEL SQL GENERADO ===");

        DemostrarSqlConsultasPendientes();
        DemostrarSqlConsultasConInclude();
        DemostrarSqlConsultasConProyeccion();
        DemostrarSqlConsultasConFiltroGlobal();
    }

    private void DemostrarSqlConsultasPendientes()
    {
        Console.WriteLine("\n--- SQL de consultas pendientes ---");
        var sql = _unidad.Ordenes.ObtenerSqlConsultasPendientes();
        Console.WriteLine(sql);
    }

    private void DemostrarSqlConsultasConInclude()
    {
        Console.WriteLine("\n--- SQL de consultas con Include ---");
        var sql = _unidad.Ordenes.ObtenerSqlConsultasConInclude();
        Console.WriteLine(sql);
    }

    private void DemostrarSqlConsultasConProyeccion()
    {
        Console.WriteLine("\n--- SQL de consultas con proyección ---");
        var sql = _unidad.Ordenes.ObtenerSqlConsultasConProyeccion();
        Console.WriteLine(sql);
    }

    private void DemostrarSqlConsultasConFiltroGlobal()
    {
        Console.WriteLine("\n--- SQL de consultas con filtro global de Soft Delete ---");
        var sql = _unidad.Ordenes.ObtenerSqlConsultasConFiltroGlobal();
    }
}
```

```

        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class AnalisisSqlUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public AnalisisSqlUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== ANÁLISIS DEL SQL GENERADO ==="); → muestra la cabecera.

Línea 18: DemostrarSqlConsultasPendientes(); → llama al método de SQL de consultas pendientes.

Línea 19: DemostrarSqlConsultasConInclude(); → llama al método de SQL con Include.

Línea 20: DemostrarSqlConsultasConProyeccion(); → llama al método de SQL con proyección.

Línea 21: DemostrarSqlConsultasConFiltroGlobal(); → llama al método de SQL con filtro global.

Línea 24: private void DemostrarSqlConsultasPendientes() → declara el método.

Línea 26: Console.WriteLine("\n--- SQL de consultas pendientes ---"); → muestra la cabecera.

Línea 27: var sql = _unidad Ordenes.ObtenerSqlConsultasPendientes(); → llama al método del repositorio.

Línea 28: Console.WriteLine(sql); → imprime el SQL.

Línea 31: private void DemostrarSqlConsultasConInclude() → declara el método.

Línea 33: Console.WriteLine("\n--- SQL de consultas con Include ---"); → muestra la cabecera.

Línea 34: var sql = _unidad Ordenes.ObtenerSqlConsultasConInclude(); → llama al método del repositorio.

Línea 35: Console.WriteLine(sql); → imprime el SQL.

Línea 38: private void DemostrarSqlConsultasConProyeccion() → declara el método.

Línea 40: Console.WriteLine("\n--- SQL de consultas con proyección ---"); → muestra la cabecera.

Línea 41: var sql = _unidad Ordenes.ObtenerSqlConsultasConProyeccion(); → llama al método del repositorio.

Línea 42: Console.WriteLine(sql); → imprime el SQL.

Línea 45: private void DemostrarSqlConsultasConFiltroGlobal() → declara el método.

Línea 47: Console.WriteLine("\n--- SQL de consultas con filtro global de Soft Delete ---"); → muestra la cabecera.

Línea 48: var sql = _unidad Ordenes.ObtenerSqlConsultasConFiltroGlobal(); → llama al método del repositorio.

Línea 49: Console.WriteLine(sql); → imprime el SQL.

Error común: si el método ObtenerSqlConsultasConFiltroGlobal no está implementado, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

Paso 5: Registrar el caso de uso en el contenedor

Modificar src/AceriaData.Console/Program.cs:

```
services.AddScoped<AnalisisSqlUseCase>();
```

Línea 1: services.AddScoped(); → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<AnalisisSqlUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

Paso 7: Insertar datos de prueba con planchas

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes y planchas:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    context.OrdenesFabricacion.AddRange(orden1, orden2);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);
    context.SaveChanges();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: `var orden1 = new OrdenFabricacion { ... }; → crea la primera orden.`

Línea 8: `var orden2 = new OrdenFabricacion { ... }; → crea la segunda orden.`

Línea 9: `context.OrdenesFabricacion.AddRange(orden1, orden2); → registra las órdenes.`

Línea 10: `context.SaveChanges(); → inserta las órdenes.`

Línea 12: `var plancha1 = new PlanchaAcero { ... }; → crea la primera plancha.`

Línea 13: `var plancha2 = new PlanchaAcero { ... }; → crea la segunda plancha.`

Línea 14: `var plancha3 = new PlanchaAcero { ... }; → crea la tercera plancha.`

Línea 15: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3); → registra las planchas.`

Línea 16: `context.SaveChanges(); → inserta las planchas.`

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con el SQL generado por cada consulta. Se observa el SQL de las consultas pendientes, con Include, con proyección y con filtro global.

Paso 9: Analizar la salida

La salida del programa muestra el SQL generado por cada consulta:

```
=== ANÁLISIS DEL SQL GENERADO ===
```

```
--- SQL de consultas pendientes ---
```

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [o].
[Observaciones], [o].[IsDeleted], [o].[DeletedAt]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente' AND [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[FechaCreacion]
```

```
--- SQL de consultas con Include ---
```

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [o].
[Observaciones], [o].[IsDeleted], [o].[DeletedAt], [p].[Id], [p].[OrdenId], [p].[Espesor], [p].[Ancho],
[p].[Largo], [p].[Peso], [p].[Activa], [p].[IsDeleted], [p].[DeletedAt]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[Cliente] = N'Constructora del Norte' AND [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted]
= CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]
```

```
--- SQL de consultas con proyección ---
```

```
SELECT [o].[NumeroOrden], [o].[Cliente]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente' AND [o].[IsDeleted] = CAST(0 AS bit)
```

```
--- SQL de consultas con filtro global de Soft Delete ---
```

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [o].
[Observaciones], [o].[IsDeleted], [o].[DeletedAt], [p].[Id], [p].[OrdenId], [p].[Espesor], [p].[Ancho],
[p].[Largo], [p].[Peso], [p].[Activa], [p].[IsDeleted], [p].[DeletedAt]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]
```

La primera sección muestra el SQL de las consultas pendientes. La segunda sección muestra el SQL de las consultas con Include. La tercera sección muestra el SQL de las consultas con proyección. La cuarta sección muestra el SQL de las consultas con filtro global.

Observaciones: el SQL de la consulta con proyección solo incluye las columnas proyectadas. El SQL de la consulta con Include incluye todas las columnas de ambas tablas. El filtro global de Soft Delete se aplica automáticamente en todas las consultas.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerSqlConsultasConProyeccion para llamar a ToList antes de ToQueryString:

```
public string ObtenerSqlConsultasConProyeccion()
{
    var consulta = _context.OrdenesFabricacion
        .Where(o => o.Estado == "Pendiente")
        .Select(o => new { o.NumeroOrden, o.Cliente })
        .ToList();

    return "La consulta ya se ha ejecutado, no se puede obtener el SQL.";
}
```

Resultado esperado: el método devuelve un mensaje indicando que la consulta ya se ha ejecutado. ToQueryString no funciona sobre una lista materializada.

Solución: llamar a ToQueryString directamente sobre el IQueryable, sin materializar.

```
public string ObtenerSqlConsultasConProyeccion()
{
    var consulta = _context.OrdenesFabricacion
        .Where(o => o.Estado == "Pendiente")
        .Select(o => new { o.NumeroOrden, o.Cliente });

    return consulta.ToQueryString();
}
```

Resultado esperado con la solución: el método devuelve el SQL sin ejecutar la consulta.

Errores comunes del ejercicio

Error	Causa	Solución
ToQueryString no funciona	Se llamó a ToList antes	Llamar a ToQueryString sobre el IQueryable
SQL no incluye el filtro	El filtro se aplica en memoria	Aplicar el filtro antes de materializar
SQL no incluye el JOIN	Se olvidó el Include	Añadir el Include
Logging no muestra SQL	Categoría incorrecta o nivel insuficiente	Ajustar categorías y nivel
Datos sensibles expuestos	EnableSensitiveDataLogging habilitado en producción	Deshabilitar en producción
Errores detallados en producción	EnableDetailedErrors habilitado en producción	Deshabilitar en producción

Reto resuelto: Analizar el SQL de una consulta con múltiples Include

Reto: Crear un método en el repositorio que devuelva el SQL de una consulta con Include de planchas, Include de detalle y ThenInclude de aleaciones. Analizar el SQL generado y comprobar si se produce un producto cartesiano.

Solución paso a paso:**Paso 1: Añadir el método a la interfaz IOrdenRepositorio:**

```
string ObtenerSqlConsultasConMultiplesInclude();
```

Línea 1: string ObtenerSqlConsultasConMultiplesInclude(); → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```
public string ObtenerSqlConsultasConMultiplesInclude()
{
    var consulta = _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .Where(o => o.Estado == "Pendiente");

    return consulta.ToQueryString();
}
```

Línea 1: public string ObtenerSqlConsultasConMultiplesInclude() → declara el método.

Línea 3: var consulta = _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 5: .Include(o => o.Detalle) → incluye la referencia al detalle.

Línea 6: .Include(o => o.OrdenesAleaciones) → incluye la colección de entidades intermedias.

Línea 7: .ThenInclude(oa => oa.Aleacion) → incluye la aleación de cada entidad intermedia.

Línea 8: .Where(o => o.Estado == "Pendiente"); → filtra por estado.

Línea 10: return consulta.ToQueryString(); → devuelve el SQL.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarSqlConsultasConMultiplesInclude()
{
    Console.WriteLine("\n--- SQL de consultas con múltiples Include ---");
    var sql = _unidad.Ordenes.ObtenerSqlConsultasConMultiplesInclude();
    Console.WriteLine(sql);
}
```

Línea 1: private void DemostrarSqlConsultasConMultiplesInclude() → declara el método.

Línea 3: Console.WriteLine("\n--- SQL de consultas con múltiples Include ---"); → muestra la cabecera.

Línea 4: var sql = _unidad.Ordenes.ObtenerSqlConsultasConMultiplesInclude(); → llama al método del repositorio.

Línea 5: Console.WriteLine(sql); → imprime el SQL.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarSqlConsultasConMultiplesInclude();
```

| Paso 5: Ejecutar dotnet run y analizar el SQL.

Resultado esperado: el SQL generado incluye dos LEFT JOIN, uno con la tabla de planchas y otro con la tabla de detalles, más un tercer LEFT JOIN con la tabla de entidades intermedias y un cuarto LEFT JOIN con la tabla de aleaciones. Como se incluyen dos colecciones (planchas y entidades intermedias), EF Core genera un producto cartesiano: cada plancha se combina con cada entidad intermedia. Esto provoca un número elevado de filas. La solución es usar AsSplitQuery, que se estudiará en el punto 4.8.

| Analogía final

El análisis del SQL generado en una acería es como revisar el plano de una orden de búsqueda antes de enviarla al archivo central. En lugar de enviar la orden y ver qué devuelve el archivo, el jefe de planta revisa la orden para asegurarse de que las condiciones son correctas y de que el archivo va a buscar exactamente lo que se necesita. ToQueryString es como ver la orden de búsqueda sin enviarla. El logging es como el registro de todas las búsquedas que se han enviado al archivo, con sus condiciones y sus resultados. Las categorías de logging son como los distintos libros de registro: uno para las búsquedas, otro para las traducciones, otro para las escrituras. Los niveles de logging son como el nivel de detalle del registro: en producción solo se registran los errores, en desarrollo se registra todo. EnableSensitiveDataLogging es como incluir los nombres de los clientes en el registro: útil en desarrollo, pero peligroso en producción. Así funciona el análisis del SQL en EF Core: se inspecciona la consulta antes de ejecutarla, se registra lo que se ejecuta y se detectan problemas antes de que afecten al rendimiento.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de análisis de SQL a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso AnalisisSqlUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con órdenes y planchas.

Ejecutado las demostraciones de SQL de consultas pendientes, con Include, con proyección y con filtro global.

Analizado la salida y comprobado que el filtro global se aplica automáticamente.

Diagnosticado el error de llamar a ToList antes de ToQueryString.

Creado el método ObtenerSqlConsultasConMultiplesInclude y analizado el producto cartesiano.

Punto 4.2 - Tracking y No Tracking

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que demuestren el comportamiento del tracking y del no tracking. Comparar el número de entidades rastreadas, el tiempo de ejecución y el SQL generado. Aplicar AsNoTracking en consultas de solo lectura y Tracking en consultas de escritura.

Contexto del proyecto: En el punto 4.1 se introdujo el análisis del SQL generado con ToQueryString y logging. En este punto se profundiza en el control del tracking, que es una de las técnicas de optimización más importantes en EF Core. Esta técnica se usará en el punto 4.3 para las variantes de No Tracking.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de tracking a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerConTracking();
    List<OrdenFabricacion> ObtenerSinTracking();
    int ContarEntidadesRastreadas();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerConTracking(); → método que obtiene las órdenes con Tracking.

Línea 11: List ObtenerSinTracking(); → método que obtiene las órdenes sin Tracking.

Línea 12: int ContarEntidadesRastreadas(); → método que cuenta las entidades rastreadas.

Línea 14: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 15: void Eliminar(OrdenFabricacion orden); → método que elimina una orden.

Error común: si los métodos devuelven IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

Paso 3: Implementar los métodos de tracking en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepository.cs:

```
public List<OrdenFabricacion> ObtenerConTracking()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerSinTracking()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public int ContarEntidadesRastreadas()
{
    return _context.ChangeTracker.Entries().Count();
}
```

Línea 1: public List ObtenerConTracking() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 5: .ToList(); → materializa la consulta con Tracking.

Línea 8: public List ObtenerSinTracking() → declara el método.

Línea 10: return _context.OrdenesFabricacion → inicia la consulta.

Línea 11: .AsNoTracking() → aplica AsNoTracking.

Línea 12: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 13: .ToList(); → materializa la consulta sin Tracking.

Línea 16: public int ContarEntidadesRastreadas() → declara el método.

Línea 18: return _context.ChangeTracker.Entries().Count(); → cuenta las entidades rastreadas.

Error común: si se llama a ContarEntidadesRastreadas después de ObtenerSinTracking, el número es cero porque las entidades no se han registrado.

Paso 4: Crear el caso de uso de tracking

Crear el archivo src/AceriaData.Application/UseCases/TrackingUseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases; public class TrackingUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public TrackingUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== TRACKING Y NO TRACKING ===");

        DemostrarConTracking();
    }
}
```

```

        DemostrarSinTracking();
        CompararRendimiento();
    }

    private void DemostrarConTracking()
    {
        Console.WriteLine("\n--- Con Tracking ---");

        var ordenes = _unidad.Ordenes.ObtenerConTracking();
        var rastreadas = _unidad.Ordenes.ContarEntidadesRastreadas();

        Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");
        Console.WriteLine($"Entidades rastreadas: {rastreadas}");
    }

    private void DemostrarSinTracking()
    {
        Console.WriteLine("\n--- Sin Tracking ---");

        var ordenes = _unidad.Ordenes.ObtenerSinTracking();
        var rastreadas = _unidad.Ordenes.ContarEntidadesRastreadas();

        Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");
        Console.WriteLine($"Entidades rastreadas: {rastreadas}");
    }

    private void CompararRendimiento()
    {
        Console.WriteLine("\n--- Comparación de rendimiento ---");

        var cronometroCon = Stopwatch.StartNew();
        var ordenesCon = _unidad.Ordenes.ObtenerConTracking();
        cronometroCon.Stop();

        var cronometroSin = Stopwatch.StartNew();
        var ordenesSin = _unidad.Ordenes.ObtenerSinTracking();
        cronometroSin.Stop();

        Console.WriteLine($"Con Tracking: {cronometroCon.ElapsedMilliseconds} ms | Órdenes: {ordenesCon.Count}");
        Console.WriteLine($"Sin Tracking: {cronometroSin.ElapsedMilliseconds} ms | Órdenes: {ordenesSin.Count}");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class TrackingUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public TrackingUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== TRACKING Y NO TRACKING ==="); → muestra la cabecera.

Línea 19: DemostrarConTracking(); → llama al método de Tracking.

Línea 20: DemostrarSinTracking(); → llama al método de No Tracking.

Línea 21: CompararRendimiento(); → llama al método de comparación.

Línea 24: private void DemostrarConTracking() → declara el método.

Línea 26: Console.WriteLine("\n--- Con Tracking ---"); → muestra la cabecera.

Línea 28: `var ordenes = _unidad Ordenes.ObtenerConTracking();` → carga las órdenes con Tracking.

Línea 29: `var rastreadas = _unidad Ordenes.ContarEntidadesRastreadas();` → cuenta las entidades rastreadas.

Línea 31: `Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");` → muestra el número de órdenes.

Línea 32: `Console.WriteLine($"Entidades rastreadas: {rastreadas}");` → muestra el número de entidades rastreadas.

Línea 35: `private void DemostrarSinTracking()` → declara el método.

Línea 37: `Console.WriteLine("\n--- Sin Tracking ---");` → muestra la cabecera.

Línea 39: `var ordenes = _unidad Ordenes.ObtenerSinTracking();` → carga las órdenes sin Tracking.

Línea 40: `var rastreadas = _unidad Ordenes.ContarEntidadesRastreadas();` → cuenta las entidades rastreadas.

Línea 42: `Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");` → muestra el número de órdenes.

Línea 43: `Console.WriteLine($"Entidades rastreadas: {rastreadas}");` → muestra el número de entidades rastreadas.

Línea 46: `private void CompararRendimiento()` → declara el método.

Línea 48: `Console.WriteLine("\n--- Comparación de rendimiento ---");` → muestra la cabecera.

Línea 50: `var cronometroCon = Stopwatch.StartNew();` → inicia el cronómetro para la consulta con Tracking.

Línea 51: `var ordenesCon = _unidad Ordenes.ObtenerConTracking();` → carga las órdenes con Tracking.

Línea 52: `cronometroCon.Stop();` → detiene el cronómetro.

Línea 54: `var cronometroSin = Stopwatch.StartNew();` → inicia el cronómetro para la consulta sin Tracking.

Línea 55: `var ordenesSin = _unidad Ordenes.ObtenerSinTracking();` → carga las órdenes sin Tracking.

Línea 56: `cronometroSin.Stop();` → detiene el cronómetro.

Línea 58: `Console.WriteLine($"Con Tracking: {cronometroCon.ElapsedMilliseconds} ms | Órdenes: {ordenesCon.Count}");` → muestra el tiempo y el número de órdenes.

Línea 59: `Console.WriteLine($"Sin Tracking: {cronometroSin.ElapsedMilliseconds} ms | Órdenes: {ordenesSin.Count}");` → muestra el tiempo y el número de órdenes.

Error común: si se ejecutan las dos consultas en el mismo DbContext, las entidades de la primera consulta permanecen en el Change Tracker cuando se ejecuta la segunda. Se debe usar un DbContext distinto para cada consulta si se quiere medir el tracking de forma aislada.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<TrackingUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<TrackingUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

| Paso 7: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... } → quinta orden.

Línea 16: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 17: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el número de entidades rastreadas con y sin Tracking, y la comparación de tiempos.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== TRACKING Y NO TRACKING ===

--- Con Tracking ---
Órdenes cargadas: 5
Entidades rastreadas: 5

--- Sin Tracking ---
Órdenes cargadas: 5
Entidades rastreadas: 5

--- Comparación de rendimiento ---
Con Tracking: 45 ms | Órdenes: 5
Sin Tracking: 38 ms | Órdenes: 5
```

La primera sección muestra que con Tracking se cargan cinco órdenes y se rastrean cinco entidades. La segunda sección muestra que sin Tracking se cargan cinco órdenes pero se rastrean cinco entidades porque el DbContext ya tenía las entidades de la consulta anterior. La tercera sección muestra que sin Tracking es ligeramente más rápido.

Observaciones: el número de entidades rastreadas en la segunda sección es cinco porque el DbContext es el mismo que en la primera sección. Para medir el tracking de forma aislada, se debe usar un DbContext distinto. La diferencia de tiempo es pequeña porque la tabla tiene pocas filas. En tablas con muchas filas, la diferencia es mayor.

Paso 10: Diagnosticar un error común

Modificar el método DemostrarSinTracking para usar AsNoTracking en un DbContext distinto:

```
private void DemostrarSinTrackingAislado()
{
    Console.WriteLine("\n--- Sin Tracking (DbContext aislado) ---");

    using var scope = _provider.CreateScope();
    var repositorio = scope.ServiceProvider.GetRequiredService<IOrdenRepositorio>();

    var ordenes = repositorio.ObtenerSinTracking();
    var rastreadas = repositorio.ContarEntidadesRastreadas();

    Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");
    Console.WriteLine($"Entidades rastreadas: {rastreadas}");
}
```

Línea 1: private void DemostrarSinTrackingAislado() → declara el método.

Línea 3: Console.WriteLine("\n--- Sin Tracking (DbContext aislado) ---"); → muestra la cabecera.

Línea 5: using var scope = _provider.CreateScope(); → crea un ámbito nuevo.

Línea 6: var repositorio = scope.ServiceProvider.GetRequiredService(); → resuelve el repositorio con un DbContext nuevo.

Línea 8: var ordenes = repositorio.ObtenerSinTracking(); → carga las órdenes sin Tracking.

Línea 9: `var rastreadas = repositorio.ContarEntidadesRastreadas();` → cuenta las entidades rastreadas.

Línea 11: `Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");` → muestra el número de órdenes.

Línea 12: `Console.WriteLine($"Entidades rastreadas: {rastreadas}");` → muestra el número de entidades rastreadas.

Resultado esperado con la solución: el número de entidades rastreadas es cero porque el DbContext es nuevo y las entidades no se han registrado.

Errores comunes del ejercicio

Error	Causa	Solución
Cambios no guardados	Se aplicó <code>AsNoTracking</code> y se modificó la entidad	Usar Tracking en consultas de escritura
Número de entidades incorrecto	Se usó el mismo DbContext para varias consultas	Usar un DbContext distinto por consulta
Caché de identidad no usada	Se aplicó <code>AsNoTracking</code>	Usar Tracking si se necesita la caché
Proyección con Tracking	Se aplicó Tracking a una proyección	Las proyecciones no registran entidades
Include con Tracking	Se incluyó una colección con Tracking	El tracking incluye las entidades relacionadas
<code>AsSplitQuery</code> con Tracking	Se dividió la consulta con Tracking	El tracking se aplica a todas las consultas

Reto resuelto: Comparar tracking en consultas con Include

Reto: Crear dos métodos en el repositorio que carguen las órdenes con sus planchas, uno con Tracking y otro sin Tracking. Comparar el número de entidades rastreadas en cada caso.

Solución paso a paso:

Paso 1: Añadir los métodos a la interfaz `IOrdenRepositorio`:

```
List<OrdenFabricacion> ObtenerConPlanchasConTracking();
List<OrdenFabricacion> ObtenerConPlanchasSinTracking();
```

Línea 1: `List ObtenerConPlanchasConTracking();` → declara el método con Tracking.

Línea 2: `List ObtenerConPlanchasSinTracking();` → declara el método sin Tracking.

Paso 2: Implementar los métodos en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerConPlanchasConTracking()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerConPlanchasSinTracking()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerConPlanchasConTracking() → declara el método con Tracking.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 5: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 6: .ToList(); → materializa la consulta con Tracking.

Línea 9: public List ObtenerConPlanchasSinTracking() → declara el método sin Tracking.

Línea 11: return _context.OrdenesFabricacion → inicia la consulta.

Línea 12: .AsNoTracking() → aplica AsNoTracking.

Línea 13: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 14: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 15: .ToList(); → materializa la consulta sin Tracking.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarTrackingConInclude()
{
    Console.WriteLine("\n--- Tracking con Include ---");

    using var scopeCon = _provider.CreateScope();
    var repoCon = scopeCon.ServiceProvider.GetRequiredService<IOrdenRepositorio>();
    var ordenesCon = repoCon.ObtenerConPlanchasConTracking();
    var rastreadasCon = repoCon.ContarEntidadesRastreadas();
    Console.WriteLine($"Con Tracking: órdenes: {ordenesCon.Count} | entidades rastreadas: {rastreadasCon}");

    using var scopeSin = _provider.CreateScope();
    var repoSin = scopeSin.ServiceProvider.GetRequiredService<IOrdenRepositorio>();
    var ordenesSin = repoSin.ObtenerConPlanchasSinTracking();
    var rastreadasSin = repoSin.ContarEntidadesRastreadas();
    Console.WriteLine($"Sin Tracking: órdenes: {ordenesSin.Count} | entidades rastreadas: {rastreadasSin}");
}
```

Línea 1: private void DemostrarTrackingConInclude() → declara el método.

Línea 3: Console.WriteLine("\n--- Tracking con Include ---"); → muestra la cabecera.

Línea 5: using var scopeCon = _provider.CreateScope(); → crea un ámbito para la consulta con Tracking.

Línea 6: var repoCon = scopeCon.ServiceProvider.GetRequiredService(); → resuelve el repositorio.

Línea 7: `var ordenesCon = repoCon.ObtenerConPlanchasConTracking();` → carga las órdenes con Tracking.

Línea 8: `var rastreadasCon = repoCon.ContarEntidadesRastreadas();` → cuenta las entidades rastreadas.

Línea 9: `Console.WriteLine($"Con Tracking: órdenes: {ordenesCon.Count} | entidades rastreadas: {rastreadasCon}");` → muestra los resultados.

Línea 11: `using var scopeSin = _provider.CreateScope();` → crea un ámbito para la consulta sin Tracking.

Línea 12: `var repoSin = scopeSin.ServiceProvider.GetRequiredService();` → resuelve el repositorio.

Línea 13: `var ordenesSin = repoSin.ObtenerConPlanchasSinTracking();` → carga las órdenes sin Tracking.

Línea 14: `var rastreadasSin = repoSin.ContarEntidadesRastreadas();` → cuenta las entidades rastreadas.

Línea 15: `Console.WriteLine($"Sin Tracking: órdenes: {ordenesSin.Count} | entidades rastreadas: {rastreadasSin}");` → muestra los resultados.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarTrackingConInclude();
```

Paso 5: Ejecutar dotnet run y comparar los resultados.

Resultado esperado: con Tracking, el número de entidades rastreadas incluye las órdenes y las planchas. Sin Tracking, el número de entidades rastreadas es cero.

Analogía final

El tracking en una acería es como el libro de producción que el supervisor mantiene durante el turno. Cuando una plancha entra en la línea, el supervisor la anota en el libro con todos sus datos. Si la plancha se modifica, el supervisor lo anota. Al final del turno, el supervisor revisa el libro y genera las órdenes de actualización. El tracking consume tiempo y espacio: cada plancha anotada ocupa una línea en el libro. El no tracking es como no anotar las planchas en el libro: si solo se van a consultar, no hace falta anotarlas. El libro se mantiene limpio y el supervisor se ahorra trabajo. Pero si se quiere modificar una plancha, hay que anotarla primero. `AsNoTracking` es como decirle al supervisor que no anote las planchas: se cargan más rápido, pero no se pueden modificar. `AsTracking` es como decirle al supervisor que anote las planchas aunque el turno sea de solo consulta. La caché de identidad es como el fichero de planchas ya anotadas: si se pide la misma plancha dos veces, el supervisor devuelve la misma ficha. Sin tracking, el supervisor crea una ficha nueva cada vez. Así funciona el tracking en EF Core: se anota lo que se va a modificar, se deja de anotar lo que solo se va a consultar.

Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de tracking a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `TrackingUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes.

Ejecutado las demostraciones de Tracking, No Tracking y comparación de rendimiento.

Analizado la salida y comprobado el número de entidades rastreadas.

Diagnosticado el error de usar el mismo `DbContext` para varias consultas.

Creado los métodos `ObtenerConPlanchasConTracking` y `ObtenerConPlanchasSinTracking`.

Punto 4.3 - AsNoTracking y AsNoTrackingWithIdentityResolution

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que demuestren el comportamiento de AsNoTracking y AsNoTrackingWithIdentityResolution. Comparar el número de instancias duplicadas, el tiempo de ejecución y el SQL generado. Aplicar la variante adecuada según el escenario.

Contexto del proyecto: En el punto 4.2 se estudió el control del tracking, comparando las consultas con Tracking y sin Tracking. En este punto se profundiza en las variantes de No Tracking, que permiten resolver las instancias duplicadas en consultas con relaciones. Esta técnica se usará en el punto 4.4 para el problema N+1.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de resolución de identidad a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerConPlanchasAsNoTracking();
    List<OrdenFabricacion> ObtenerConPlanchasConResolucionIdentidad();
    int ContarPlanchasInstanciadas(List<OrdenFabricacion> ordenes);

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerConPlanchasAsNoTracking(); → método que carga las órdenes con planchas usando AsNoTracking.

Línea 11: List ObtenerConPlanchasConResolucionIdentidad(); → método que carga las órdenes con planchas usando AsNoTrackingWithIdentityResolution.

Línea 12: int ContarPlanchasInstanciadas(List ordenes); → método que cuenta las planchas instanciadas.

Línea 14: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 15: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 3: Implementar los métodos en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepository.cs`:

```
public List<OrdenFabricacion> ObtenerConPlanchasAsNoTracking()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerConPlanchasConResolucionIdentidad()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public int ContarPlanchasInstanciadas(List<OrdenFabricacion> ordenes)
{
    var instancias = new HashSet<PlanchaAcero>(ReferenceEqualityComparer.Instance);
    foreach (var orden in ordenes)
    {
        foreach (var plancha in orden.Planchas)
        {
            instancias.Add(plancha);
        }
    }
    return instancias.Count;
}
```

Línea 1: `public List ObtenerConPlanchasAsNoTracking()` → declara el método.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 5: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 6: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 7: `.ToList();` → materializa la consulta.

Línea 10: `public List ObtenerConPlanchasConResolucionIdentidad()` → declara el método.

Línea 12: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 13: `.AsNoTrackingWithIdentityResolution()` → aplica `AsNoTrackingWithIdentityResolution`.

Línea 14: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 15: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 16: `.ToList();` → materializa la consulta.

Línea 19: `public int ContarPlanchasInstanciadas(List ordenes)` → declara el método.

Línea 21: `var instancias = new HashSet(ReferenceEqualityComparer.Instance);` → crea un conjunto de instancias por referencia.

Línea 22: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 24: `foreach (var plancha in orden.Planchas)` → itera sobre las planchas.

Línea 26: `instancias.Add(plancha);` → añade la plancha al conjunto.

Línea 29: `return instancias.Count;` → devuelve el número de instancias únicas.

Error común: si se usa `HashSet` sin `ReferenceEqualityComparer`, el conjunto usa `Equals` y `GetHashCode` de la entidad, que pueden no estar sobrescritos. Se debe usar `ReferenceEqualityComparer.Instance` para comparar por referencia.

| Paso 4: Crear el caso de uso de resolución de identidad

Crear el archivo `src/AceriaData.Application/UseCases/ResolucionIdentidadUseCase.cs`:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ResolucionIdentidadUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ResolucionIdentidadUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== ASNOTRACKING VS ASNOTRACKINGWITHIDENTITYRESOLUTION ===");

        DemostrarAsNoTracking();
        DemostrarConResolucionIdentidad();
        CompararRendimiento();
    }

    private void DemostrarAsNoTracking()
    {
        Console.WriteLine("\n--- AsNoTracking ---");

        var ordenes = _unidad.Ordenes.ObtenerConPlanchasAsNoTracking();
        var instancias = _unidad.Ordenes.ContarPlanchasInstanciadas(ordenes);

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        Console.WriteLine($"Instancias de planchas: {instancias}");
    }

    private void DemostrarConResolucionIdentidad()
    {
        Console.WriteLine("\n--- AsNoTrackingWithIdentityResolution ---");

        var ordenes = _unidad.Ordenes.ObtenerConPlanchasConResolucionIdentidad();
        var instancias = _unidad.Ordenes.ContarPlanchasInstanciadas(ordenes);

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        Console.WriteLine($"Instancias de planchas: {instancias}");
    }

    private void CompararRendimiento()
    {
        Console.WriteLine("\n--- Comparación de rendimiento ---");

        var cronometroNoTracking = Stopwatch.StartNew();
        var ordenesNoTracking = _unidad.Ordenes.ObtenerConPlanchasAsNoTracking();
        cronometroNoTracking.Stop();

        var cronometroResolucion = Stopwatch.StartNew();
        var ordenesResolucion = _unidad.Ordenes.ObtenerConPlanchasConResolucionIdentidad();
        cronometroResolucion.Stop();
    }
}
```

```

        Console.WriteLine($"AsNoTracking: {cronometroNoTracking.ElapsedMilliseconds} ms | Órdenes:
{ordenesNoTracking.Count}");
        Console.WriteLine($"AsNoTrackingWithIdentityResolution:
{cronometroResolucion.ElapsedMilliseconds} ms | Órdenes: {ordenesResolucion.Count}");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class ResolucionIdentidadUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public ResolucionIdentidadUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== ASNOTRACKING VS ASNOTRACKINGWITHIDENTITYRESOLUTION
==="); → muestra la cabecera.

Línea 19: DemostrarAsNoTracking(); → llama al método de AsNoTracking.

Línea 20: DemostrarConResolucionIdentidad(); → llama al método de resolución de identidad.

Línea 21: CompararRendimiento(); → llama al método de comparación.

Línea 24: private void DemostrarAsNoTracking() → declara el método.

Línea 26: Console.WriteLine("\n--- AsNoTracking ---"); → muestra la cabecera.

Línea 28: var ordenes = _unidad.Ordenes.ObtenerConPlanchasAsNoTracking(); → carga las órdenes con AsNoTracking.

Línea 29: var instancias = _unidad.Ordenes.ContarPlanchasInstanciadas(ordenes); → cuenta las instancias de planchas.

Línea 31: Console.WriteLine(\$"Órdenes: {ordenes.Count}"); → muestra el número de órdenes.

Línea 32: Console.WriteLine(\$"Instancias de planchas: {instancias}"); → muestra el número de instancias.

Línea 35: private void DemostrarConResolucionIdentidad() → declara el método.

Línea 37: Console.WriteLine("\n--- AsNoTrackingWithIdentityResolution ---"); → muestra la cabecera.

Línea 39: var ordenes = _unidad.Ordenes.ObtenerConPlanchasConResolucionIdentidad(); → carga las órdenes con resolución de identidad.

Línea 40: var instancias = _unidad.Ordenes.ContarPlanchasInstanciadas(ordenes); → cuenta las instancias de planchas.

Línea 42: Console.WriteLine(\$"Órdenes: {ordenes.Count}"); → muestra el número de órdenes.

Línea 43: Console.WriteLine(\$"Instancias de planchas: {instancias}"); → muestra el número de instancias.

Línea 46: private void CompararRendimiento() → declara el método.

Línea 48: Console.WriteLine("\n--- Comparación de rendimiento ---"); → muestra la cabecera.

Línea 50: var cronometroNoTracking = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 51: var ordenesNoTracking = _unidad.Ordenes.ObtenerConPlanchasAsNoTracking(); → carga las órdenes con AsNoTracking.

Línea 52: `cronometroNoTracking.Stop();` → detiene el cronómetro.

Línea 54: `var cronometroResolucion = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 55: `var ordenesResolucion = _unidad.Ordenes.ObtenerConPlanchasConResolucionIdentidad();` → carga las órdenes con resolución de identidad.

Línea 56: `cronometroResolucion.Stop();` → detiene el cronómetro.

Línea 58: `Console.WriteLine($"AsNoTracking: {cronometroNoTracking.ElapsedMilliseconds} ms | Órdenes: {ordenesNoTracking.Count}");` → muestra el tiempo de `AsNoTracking`.

Línea 59: `Console.WriteLine($"AsNoTrackingWithIdentityResolution: {cronometroResolucion.ElapsedMilliseconds} ms | Órdenes: {ordenesResolucion.Count}");` → muestra el tiempo de resolución de identidad.

Error común: si se ejecutan las dos consultas en el mismo `DbContext`, la caché de identidad del Change Tracker puede afectar a los resultados. Se debe usar un `DbContext` distinto para cada consulta.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ResolucionIdentidadUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ResolucionIdentidadUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con planchas compartidas

Asegurarse de que hay datos en la base de datos con planchas que aparezcan en varias órdenes:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };
    context OrdenesFabricacion.AddRange(orden1, orden2, orden3);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    var plancha4 = new PlanchaAcero { OrdenId = orden3.Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);
    context.SaveChanges();
}

```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base de datos.

Línea 5: `context.Database.Migrate();` → aplica las migraciones.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 9: `var orden3 = new OrdenFabricacion { ... };` → crea la tercera orden.

Línea 10: `context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);` → registra las órdenes.

Línea 11: `context.SaveChanges();` → inserta las órdenes.

Línea 13: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 14: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 15: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 16: `var plancha4 = new PlanchaAcero { ... };` → crea la cuarta plancha.

Línea 17: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);` → registra las planchas.

Línea 18: `context.SaveChanges();` → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el número de instancias de planchas con `AsNoTracking` y con `AsNoTrackingWithIdentityResolution`.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== ASNOTRACKING VS ASNOTRACKINGWITHIDENTITYRESOLUTION ===

--- AsNoTracking ---
Órdenes: 3
Instancias de planchas: 4

--- AsNoTrackingWithIdentityResolution ---
Órdenes: 3
Instancias de planchas: 4

--- Comparación de rendimiento ---
AsNoTracking: 42 ms | Órdenes: 3
AsNoTrackingWithIdentityResolution: 48 ms | Órdenes: 3
```

La primera sección muestra que con `AsNoTracking` se cargan tres órdenes y cuatro instancias de planchas. La segunda sección muestra que con `AsNoTrackingWithIdentityResolution` se cargan tres órdenes y cuatro instancias de planchas. La tercera sección muestra que `AsNoTracking` es ligeramente más rápido.

Observaciones: en este caso, el número de instancias es el mismo porque cada plancha pertenece a una sola orden. Para ver la diferencia, se necesitaría un escenario donde la misma entidad aparezca en varias filas del resultado, como una consulta con dos colecciones incluidas. La diferencia de tiempo es pequeña porque la tabla tiene pocas filas.

Paso 10: Diagnosticar un error común

Modificar el método `ContarPlanchasInstanciadas` para usar `HashSet` sin `ReferenceEqualityComparer`:

```
public int ContarPlanchasInstanciadas(List<OrdenFabricacion> ordenes)
{
    var instancias = new HashSet<PlanchaAcero>();
    foreach (var orden in ordenes)
    {
        foreach (var plancha in orden.Planchas)
        {
            instancias.Add(plancha);
        }
    }
    return instancias.Count;
}
```

Resultado esperado: el método usa `Equals` y `GetHashCode` de `PlanchaAcero`, que no están sobrescritos. Como `PlanchaAcero` no sobrescribe `Equals`, el `HashSet` compara por referencia y cuenta las instancias correctamente. Sin embargo, si `PlanchaAcero` sobrescribiera `Equals` por valor, el `HashSet` contaría las planchas por valor y no por referencia.

Solución: usar `ReferenceEqualityComparer.Instance` para garantizar que la comparación sea por referencia.

```
var instancias = new HashSet<PlanchaAcero>(ReferenceEqualityComparer.Instance);
Resultado esperado con la solución: el HashSet compara por referencia y cuenta las instancias correctamente.
```

Errores comunes del ejercicio

Error	Causa	Solución
Instancias duplicadas	Se usó AsNoTracking con relaciones	Usar AsNoTrackingWithIdentityResolution
Número de instancias incorrecto	Se usó HashSet sin ReferenceEqualityComparer	Usar ReferenceEqualityComparer.Instance
Rendimiento degradado	Se usó AsNoTrackingWithIdentityResolution sin necesidad	Usar AsNoTracking si no hay duplicados
Caché de identidad compartida	Se usó el mismo DbContext para varias consultas	Usar un DbContext distinto por consulta
Proyección con resolución de identidad	Se aplicó a una proyección	No tiene efecto en proyecciones

Reto resuelto: Comparar instancias con dos colecciones incluidas

Reto: Crear un método en el repositorio que cargue las órdenes con dos colecciones incluidas (planchas y entidades intermedias) usando AsNoTracking y AsNoTrackingWithIdentityResolution. Comparar el número de instancias de aleaciones en cada caso.

Solución paso a paso:

Paso 1: Añadir los métodos a la interfaz IOrdenRepositorio:

```
List<OrdenFabricacion> ObtenerConDosColeccionesAsNoTracking();
List<OrdenFabricacion> ObtenerConDosColeccionesConResolucionIdentidad();
```

Línea 1: List ObtenerConDosColeccionesAsNoTracking(); → declara el método con AsNoTracking.

Línea 2: List ObtenerConDosColeccionesConResolucionIdentidad(); → declara el método con resolución de identidad.

Paso 2: Implementar los métodos en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerConDosColeccionesAsNoTracking()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerConDosColeccionesConResolucionIdentidad()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerConDosColeccionesAsNoTracking() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 5: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 6: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 7: `.ThenInclude(oa => oa.Aleacion)` → incluye la aleación de cada entidad intermedia.

Línea 8: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 9: `.ToList();` → materializa la consulta.

Línea 12: `public List ObtenerConDosColeccionesConResolucionIdentidad()` → declara el método.

Línea 14: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 15: `.AsNoTrackingWithIdentityResolution()` → aplica resolución de identidad.

Línea 16: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 17: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 18: `.ThenInclude(oa => oa.Aleacion)` → incluye la aleación de cada entidad intermedia.

Línea 19: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 20: `.ToList();` → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarDosColecciones()
{
    Console.WriteLine("\n--- Dos colecciones: AsNoTracking ---");
    var ordenes1 = _unidad.Ordenes.ObtenerConDosColeccionesAsNoTracking();
    var aleaciones1 = new HashSet<Aleacion>(ReferenceEqualityComparer.Instance);
    foreach (var orden in ordenes1)
    {
        foreach (var oa in orden.OrdenesAleaciones)
        {
            aleaciones1.Add(oa.Aleacion);
        }
    }
    Console.WriteLine($"Instancias de aleaciones: {aleaciones1.Count}");

    Console.WriteLine("\n--- Dos colecciones: AsNoTrackingWithIdentityResolution ---");
    var ordenes2 = _unidad.Ordenes.ObtenerConDosColeccionesConResolucionIdentidad();
    var aleaciones2 = new HashSet<Aleacion>(ReferenceEqualityComparer.Instance);
    foreach (var orden in ordenes2)
    {
        foreach (var oa in orden.OrdenesAleaciones)
        {
            aleaciones2.Add(oa.Aleacion);
        }
    }
    Console.WriteLine($"Instancias de aleaciones: {aleaciones2.Count}");
}
```

Línea 1: `private void DemostrarDosColecciones()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Dos colecciones: AsNoTracking ---");` → muestra la cabecera.

Línea 4: `var ordenes1 = _unidad.Ordenes.ObtenerConDosColeccionesAsNoTracking();` → carga las órdenes con `AsNoTracking`.

Línea 5: `var aleaciones1 = new HashSet(ReferenceEqualityComparer.Instance);` → crea un conjunto de instancias por referencia.

Línea 6: `foreach (var orden in ordenes1)` → itera sobre las órdenes.

Línea 8: `foreach (var oa in orden.OrdenesAleaciones)` → itera sobre las entidades intermedias.

Línea 10: `aleaciones1.Add(oa.Aleacion);` → añade la aleación al conjunto.

Línea 13: `Console.WriteLine($"Instancias de aleaciones: {aleaciones1.Count}");` → muestra el número de instancias.

Línea 15: `Console.WriteLine("\n--- Dos colecciones: AsNoTrackingWithIdentityResolution ---");` → muestra la cabecera.

Línea 16: `var ordenes2 = _unidad Ordenes.ObtenerConDosColeccionesConResolucionIdentidad();` → carga las órdenes con resolución de identidad.

Línea 17: `var aleaciones2 = new HashSet<Aleacion>(ReferenceEqualityComparer.Instance);` → crea un conjunto de instancias por referencia.

Línea 18: `foreach (var orden in ordenes2)` → itera sobre las órdenes.

Línea 20: `foreach (var oa in orden.OrdenesAleaciones)` → itera sobre las entidades intermedias.

Línea 22: `aleaciones2.Add(oa.Aleacion);` → añade la aleación al conjunto.

Línea 25: `Console.WriteLine($"Instancias de aleaciones: {aleaciones2.Count}");` → muestra el número de instancias.

| Paso 4: Llamar al método desde Ejecutar:

```
DemostrarDosColecciones();
```

| Paso 5: Ejecutar dotnet run y comparar los resultados.

Resultado esperado: con `AsNoTracking`, el número de instancias de aleaciones es mayor porque la misma aleación se instancia varias veces. Con `AsNoTrackingWithIdentityResolution`, el número de instancias es menor porque las aleaciones se resuelven a la misma instancia.

| Analogía final

La resolución de identidad en una acería es como el fichero de planchas que el supervisor mantiene durante una inspección. Cuando se inspeccionan varias órdenes, el supervisor anota cada plancha que ve. Sin resolución de identidad, el supervisor anota la misma plancha cada vez que la ve en una orden distinta: al final, tiene varias fichas de la misma plancha. Con resolución de identidad, el supervisor comprueba si la plancha ya está anotada antes de anotarla de nuevo: al final, tiene una sola ficha por plancha. La caché de identidad temporal es como el fichero que el supervisor usa durante la inspección y destruye al terminar. La caché de identidad del Change Tracker es como el fichero permanente que el supervisor mantiene durante todo el turno. `AsNoTracking` es como inspeccionar sin fichero.

`AsNoTrackingWithIdentityResolution` es como inspeccionar con un fichero temporal. `AsTracking` es como inspeccionar con el fichero permanente. Así funciona la resolución de identidad en EF Core: se elige la variante según el escenario y el coste que se está dispuesto a asumir.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de resolución de identidad a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `ResolucionIdentidadUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con órdenes y planchas.

Ejecutado las demostraciones de `AsNoTracking` y `AsNoTrackingWithIdentityResolution`.

Comparado el rendimiento de ambas variantes.

Diagnosticado el error de usar HashSet sin ReferenceEqualityComparer.

Creado los métodos ObtenerConDosColeccionesAsNoTracking y
ObtenerConDosColeccionesConResolucionIdentidad.

Punto 4.4 - Problema N+1: identificación y causas

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que provoquen el problema N+1 y métodos que lo eviten con Include. Analizar el número de consultas ejecutadas en cada caso con el logging. Comparar el tiempo de ejecución. Diagnosticar el problema N+1 a partir del log.

Contexto del proyecto: En el punto 4.3 se estudiaron las variantes de No Tracking, incluyendo AsNoTrackingWithIdentityResolution. En este punto se profundiza en el problema N+1, que es uno de los problemas de rendimiento más comunes en EF Core. Esta identificación se usará en el punto 4.5 para las soluciones al problema N+1.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos del problema N+1 a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerOrdenesConPlanchasN1();
    List<OrdenFabricacion> ObtenerOrdenesConPlanchasSinN1();
    List<OrdenFabricacion> ObtenerOrdenesConDetalleN1();
    List<OrdenFabricacion> ObtenerOrdenesConDetalleSinN1();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerOrdenesConPlanchasN1(); → método que provoca el problema N+1 al cargar planchas.

Línea 11: List ObtenerOrdenesConPlanchasSinN1(); → método que evita el problema N+1 al cargar planchas.

Línea 12: List ObtenerOrdenesConDetalleN1(); → método que provoca el problema N+1 al cargar el detalle.

Línea 13: `List ObtenerOrdenesConDetalleSinN1();` → método que evita el problema N+1 al cargar el detalle.

Línea 15: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 3: Implementar los métodos del problema N+1 en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```

public List<OrdenFabricacion> ObtenerOrdenesConPlanchasN1()
{
    var ordenes = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();

    foreach (var orden in ordenes)
    {
        var planchas = _context.PlanchasAcero
            .AsNoTracking()
            .Where(p => p.OrdenId == orden.Id)
            .ToList();

        orden.Planchas = planchas;
    }

    return ordenes;
}

public List<OrdenFabricacion> ObtenerOrdenesConPlanchasSinN1()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerOrdenesConDetalleN1()
{
    var ordenes = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();

    foreach (var orden in ordenes)
    {
        var detalle = _context.DetallesOrden
            .AsNoTracking()
            .FirstOrDefault(d => d.OrdenId == orden.Id);

        orden.Detalle = detalle;
    }

    return ordenes;
}

public List<OrdenFabricacion> ObtenerOrdenesConDetalleSinN1()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Detalle)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
  
```

Línea 1: `public List ObtenerOrdenesConPlanchasN1()` → declara el método que provoca el problema N+1.

Línea 3: `var ordenes = _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 5: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 6: `.ToList();` → materializa la consulta.

Línea 8: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 10: `var planchas = _context.PlanchasAcero` → inicia la consulta de planchas.

Línea 11: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 12: `.Where(p => p.OrdenId == orden.Id)` → filtra por orden.

Línea 13: `.ToList();` → materializa la consulta.

Línea 15: `orden.Planchas = planchas;` → asigna las planchas a la orden.

Línea 18: `return ordenes;` → devuelve las órdenes.

Línea 21: `public List ObtenerOrdenesConPlanchasSinN1()` → declara el método que evita el problema N+1.

Línea 23: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 24: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 25: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 26: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 27: `.ToList();` → materializa la consulta.

Línea 30: `public List ObtenerOrdenesConDetalleN1()` → declara el método que provoca el problema N+1 al cargar el detalle.

Línea 32: `var ordenes = _context.OrdenesFabricacion` → inicia la consulta.

Línea 33: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 34: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 35: `.ToList();` → materializa la consulta.

Línea 37: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 39: `var detalle = _context.DetallesOrden` → inicia la consulta del detalle.

Línea 40: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 41: `.FirstOrDefault(d => d.OrdenId == orden.Id);` → filtra por orden.

Línea 43: `orden.Detalle = detalle;` → asigna el detalle a la orden.

Línea 46: `return ordenes;` → devuelve las órdenes.

Línea 49: `public List ObtenerOrdenesConDetalleSinN1()` → declara el método que evita el problema N+1 al cargar el detalle.

Línea 51: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 52: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 53: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 54: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 55: `.ToList();` → materializa la consulta.

Error común: si el bucle accede a una propiedad de navegación sin Include, se ejecuta una consulta adicional por cada orden. Se debe usar Include para cargar las entidades relacionadas en una sola consulta.

| Paso 4: Crear el caso de uso del problema N+1

Crear el archivo src/AceriaData.Application/UseCases/ProblemaN1UseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ProblemaN1UseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ProblemaN1UseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== PROBLEMA N+1 ===");

        DemostrarConN1();
        DemostrarSinN1();
        CompararRendimiento();
    }

    private void DemostrarConN1()
    {
        Console.WriteLine("\n--- Con N+1 (planchas) ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasN1();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        Console.WriteLine($"Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas");
        }
    }

    private void DemostrarSinN1()
    {
        Console.WriteLine("\n--- Sin N+1 (planchas) ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasSinN1();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        Console.WriteLine($"Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas");
        }
    }

    private void CompararRendimiento()
    {
        Console.WriteLine("\n--- Comparación de rendimiento ---");

        var cronometroN1 = Stopwatch.StartNew();
        var ordenesN1 = _unidad.Ordenes.ObtenerOrdenesConPlanchasN1();
        cronometroN1.Stop();
    }
}
```

```

        var cronometroSinN1 = Stopwatch.StartNew();
        var ordenesSinN1 = _unidad.Ordenes.ObtenerOrdenesConPlanchasSinN1();
        cronometroSinN1.Stop();

        Console.WriteLine($"Con N+1: {cronometroN1.ElapsedMilliseconds} ms | Órdenes:
{ordenesN1.Count}");
        Console.WriteLine($"Sin N+1: {cronometroSinN1.ElapsedMilliseconds} ms | Órdenes:
{ordenesSinN1.Count}");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class ProblemaN1UseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public ProblemaN1UseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== PROBLEMA N+1 ==="); → muestra la cabecera.

Línea 19: DemostrarConN1(); → llama al método que provoca el problema.

Línea 20: DemostrarSinN1(); → llama al método que evita el problema.

Línea 21: CompararRendimiento(); → llama al método de comparación.

Línea 24: private void DemostrarConN1() → declara el método.

Línea 26: Console.WriteLine("\n--- Con N+1 (planchas) ---"); → muestra la cabecera.

Línea 28: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 29: var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasN1(); → carga las órdenes con N+1.

Línea 30: cronometro.Stop(); → detiene el cronómetro.

Línea 32: Console.WriteLine(\$"Órdenes: {ordenes.Count}"); → muestra el número de órdenes.

Línea 33: Console.WriteLine(\$"Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 34: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 36: Console.WriteLine(\$" {orden.NumeroOrden}: {orden.Planchas.Count} planchas"); → muestra el número de planchas.

Línea 40: private void DemostrarSinN1() → declara el método.

Línea 42: Console.WriteLine("\n--- Sin N+1 (planchas) ---"); → muestra la cabecera.

Línea 44: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 45: var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasSinN1(); → carga las órdenes sin N+1.

Línea 46: cronometro.Stop(); → detiene el cronómetro.

Línea 48: Console.WriteLine(\$"Órdenes: {ordenes.Count}"); → muestra el número de órdenes.

Línea 49: Console.WriteLine(\$"Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 50: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 52: `Console.WriteLine($" {orden.NumeroOrden}: {orden.Planchas.Count} planchas");` → muestra el número de planchas.

Línea 56: `private void CompararRendimiento()` → declara el método.

Línea 58: `Console.WriteLine("\n--- Comparación de rendimiento ---");` → muestra la cabecera.

Línea 60: `var cronometroN1 = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 61: `var ordenesN1 = _unidad Ordenes.ObtenerOrdenesConPlanchasN1();` → carga las órdenes con N+1.

Línea 62: `cronometroN1.Stop();` → detiene el cronómetro.

Línea 64: `var cronometroSinN1 = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 65: `var ordenesSinN1 = _unidad Ordenes.ObtenerOrdenesConPlanchasSinN1();` → carga las órdenes sin N+1.

Línea 66: `cronometroSinN1.Stop();` → detiene el cronómetro.

Línea 68: `Console.WriteLine($"Con N+1: {cronometroN1.ElapsedMilliseconds} ms | Órdenes: {ordenesN1.Count}");` → muestra el tiempo con N+1.

Línea 69: `Console.WriteLine($"Sin N+1: {cronometroSinN1.ElapsedMilliseconds} ms | Órdenes: {ordenesSinN1.Count}");` → muestra el tiempo sin N+1.

Error común: si se ejecutan las dos consultas en el mismo `DbContext`, las entidades de la primera consulta pueden afectar a los resultados de la segunda. Se debe usar un `DbContext` distinto para cada consulta.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ProblemaN1UseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ProblemaN1UseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con varias órdenes y planchas

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes y planchas:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();

    var planchas = new List<PlanchaAcero>
    {
        new PlanchaAcero { OrdenId = ordenes[0].Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true },
        new PlanchaAcero { OrdenId = ordenes[0].Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true },
        new PlanchaAcero { OrdenId = ordenes[1].Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true },
        new PlanchaAcero { OrdenId = ordenes[2].Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true },
        new PlanchaAcero { OrdenId = ordenes[3].Id, Espesor = 9.0, Ancho = 1100, Largo = 2200, Peso = 180.0m, Activa = true },
        new PlanchaAcero { OrdenId = ordenes[4].Id, Espesor = 11.0, Ancho = 1300, Largo = 2700, Peso = 290.0m, Activa = true }
    };

    context.PlanchasAcero.AddRange(planchas);
    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... }, → quinta orden.

Línea 14: new OrdenFabricacion { ... } → sexta orden.

Línea 15: }; → cierra la lista.

Línea 17: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 18: `context.SaveChanges();` → inserta las órdenes.

Línea 20: `var planchas = new List` → crea la lista de planchas.

Línea 22: `new PlanchaAcero { ... }`, → primera plancha.

Línea 23: `new PlanchaAcero { ... }`, → segunda plancha.

Línea 24: `new PlanchaAcero { ... }`, → tercera plancha.

Línea 25: `new PlanchaAcero { ... }`, → cuarta plancha.

Línea 26: `new PlanchaAcero { ... }`, → quinta plancha.

Línea 27: `new PlanchaAcero { ... }` → sexta plancha.

Línea 28: `};` → cierra la lista.

Línea 30: `context.PlanchasAcero.AddRange(planchas);` → registra las planchas.

Línea 31: `context.SaveChanges();` → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo con N+1 y sin N+1, y el número de consultas ejecutadas en el log.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== PROBLEMA N+1 ===

--- Con N+1 (planchas) ---
Órdenes: 6
Tiempo: 145 ms
  OF-2024-0001: 2 planchas
  OF-2024-0002: 1 planchas
  OF-2024-0003: 1 planchas
  OF-2024-0004: 1 planchas
  OF-2024-0005: 1 planchas
  OF-2024-0006: 0 planchas

--- Sin N+1 (planchas) ---
Órdenes: 6
Tiempo: 42 ms
  OF-2024-0001: 2 planchas
  OF-2024-0002: 1 planchas
  OF-2024-0003: 1 planchas
  OF-2024-0004: 1 planchas
  OF-2024-0005: 1 planchas
  OF-2024-0006: 0 planchas

--- Comparación de rendimiento ---
Con N+1: 145 ms | Órdenes: 6
Sin N+1: 42 ms | Órdenes: 6
```

La primera sección muestra el tiempo con N+1. La segunda sección muestra el tiempo sin N+1. La tercera sección muestra la comparación. La diferencia es notable: 145 ms con N+1 frente a 42 ms sin N+1.

Observaciones: el tiempo con N+1 crece linealmente con el número de órdenes. El tiempo sin N+1 es constante. El log de EF Core muestra el patrón de N+1: una consulta para las órdenes y N consultas para las planchas.

Paso 10: Diagnosticar un error común

Modificar el método `ObtenerOrdenesConPlanchasSinN1` para usar `AsNoTracking` después del `Include`:

```
public List<OrdenFabricacion> ObtenerOrdenesConPlanchasSinN1()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado: el código funciona correctamente. `AsNoTracking` se puede aplicar antes o después del `Include`. El resultado es el mismo.

Solución: no hay error. `AsNoTracking` se puede aplicar en cualquier punto de la consulta antes de la materialización.

Errores comunes del ejercicio

Error	Causa	Solución
N+1 con carga Lazy	Se accede a propiedades de navegación en un bucle	Usar Include
N+1 con FirstOrDefault	Se usa FirstOrDefault en un bucle	Usar Include o cargar todas las entidades de una vez
N+1 con proyección	Se proyecta una colección sin ToList	Añadir ToList dentro de la proyección
N+1 con segundo nivel	Se accede a una propiedad de segundo nivel sin ThenInclude	Usar ThenInclude
Tiempo alto	Se ejecutan N+1 consultas	Usar Include
Log con muchas consultas	Se ejecuta una consulta por cada entidad	Usar Include

Reto resuelto: Identificar el problema N+1 en una consulta con detalle

Reto: Crear un método en el repositorio que cargue las órdenes con su detalle usando `FirstOrDefault` en un bucle, provocando el problema N+1. Crear otro método que lo evite con `Include`. Comparar el número de consultas ejecutadas y el tiempo.

Solución paso a paso:

Paso 1: Añadir los métodos a la interfaz `IOrdenRepositorio`:

```
List<OrdenFabricacion> ObtenerOrdenesConDetalleN1();
List<OrdenFabricacion> ObtenerOrdenesConDetalleSinN1();
```

Línea 1: `List ObtenerOrdenesConDetalleN1();` → declara el método con N+1.

Línea 2: `List ObtenerOrdenesConDetalleSinN1();` → declara el método sin N+1.

Paso 2: Implementar los métodos en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerOrdenesConDetalleN1()
{
    var ordenes = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();

    foreach (var orden in ordenes)
    {
        var detalle = _context.DetallesOrden
            .AsNoTracking()
            .FirstOrDefault(d => d.OrdenId == orden.Id);

        orden.Detalle = detalle;
    }

    return ordenes;
}

public List<OrdenFabricacion> ObtenerOrdenesConDetalleSinN1()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Detalle)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerOrdenesConDetalleN1() → declara el método.

Línea 3: var ordenes = _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 6: .ToList(); → materializa la consulta.

Línea 8: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 10: var detalle = _context.DetallesOrden → inicia la consulta del detalle.

Línea 11: .AsNoTracking() → aplica AsNoTracking.

Línea 12: .FirstOrDefault(d => d.OrdenId == orden.Id); → filtra por orden.

Línea 14: orden.Detalle = detalle; → asigna el detalle a la orden.

Línea 17: return ordenes; → devuelve las órdenes.

Línea 20: public List ObtenerOrdenesConDetalleSinN1() → declara el método sin N+1.

Línea 22: return _context.OrdenesFabricacion → inicia la consulta.

Línea 23: .AsNoTracking() → aplica AsNoTracking.

Línea 24: .Include(o => o.Detalle) → incluye la referencia al detalle.

Línea 25: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 26: .ToList(); → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarDetalleN1()
{
    Console.WriteLine("\n--- Detalle con N+1 ---");
    var cronometroN1 = Stopwatch.StartNew();
    var ordenesN1 = _unidad Ordenes.ObtenerOrdenesConDetalleN1();
    cronometroN1.Stop();
    Console.WriteLine($"Tiempo con N+1: {cronometroN1.ElapsedMilliseconds} ms");
    foreach (var orden in ordenesN1)
    {
        var detalle = orden.Detalle?.ComposicionQuimica ?? "Sin detalle";
        Console.WriteLine($" {orden.NumeroOrden}: {detalle}");
    }

    Console.WriteLine("\n--- Detalle sin N+1 ---");
    var cronometroSinN1 = Stopwatch.StartNew();
    var ordenesSinN1 = _unidad Ordenes.ObtenerOrdenesConDetalleSinN1();
    cronometroSinN1.Stop();
    Console.WriteLine($"Tiempo sin N+1: {cronometroSinN1.ElapsedMilliseconds} ms");
    foreach (var orden in ordenesSinN1)
    {
        var detalle = orden.Detalle?.ComposicionQuimica ?? "Sin detalle";
        Console.WriteLine($" {orden.NumeroOrden}: {detalle}");
    }
}
```

Línea 1: private void DemostrarDetalleN1() → declara el método.

Línea 3: Console.WriteLine("\n--- Detalle con N+1 ---"); → muestra la cabecera.

Línea 4: var cronometroN1 = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 5: var ordenesN1 = _unidad Ordenes.ObtenerOrdenesConDetalleN1(); → carga las órdenes con N+1.

Línea 6: cronometroN1.Stop(); → detiene el cronómetro.

Línea 7: Console.WriteLine(\$"Tiempo con N+1: {cronometroN1.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 8: foreach (var orden in ordenesN1) → itera sobre las órdenes.

Línea 10: var detalle = orden.Detalle?.ComposicionQuimica ?? "Sin detalle"; → obtiene el detalle.

Línea 11: Console.WriteLine(\$" {orden.NumeroOrden}: {detalle}"); → muestra el detalle.

Línea 14: Console.WriteLine("\n--- Detalle sin N+1 ---"); → muestra la cabecera.

Línea 15: var cronometroSinN1 = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 16: var ordenesSinN1 = _unidad Ordenes.ObtenerOrdenesConDetalleSinN1(); → carga las órdenes sin N+1.

Línea 17: cronometroSinN1.Stop(); → detiene el cronómetro.

Línea 18: Console.WriteLine(\$"Tiempo sin N+1: {cronometroSinN1.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 19: foreach (var orden in ordenesSinN1) → itera sobre las órdenes.

Línea 21: var detalle = orden.Detalle?.ComposicionQuimica ?? "Sin detalle"; → obtiene el detalle.

Línea 22: Console.WriteLine(\$" {orden.NumeroOrden}: {detalle}"); → muestra el detalle.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarDetalleN1();
```

| Paso 5: Ejecutar dotnet run y comparar los tiempos.

Resultado esperado: el tiempo con N+1 es notablemente mayor que el tiempo sin N+1. El log muestra una consulta para las órdenes y N consultas para el detalle.

| Analogía final

El problema N+1 en una acería es como ir al archivo central a buscar las carpetas de las órdenes y, en lugar de pedir todas las carpetas de planchas de una vez, pedir una carpeta de planchas por cada orden. Si hay diez órdenes, se hacen once viajes al archivo: uno para las órdenes y diez para las planchas. Cada viaje tiene un coste: abrir la puerta, buscar la carpeta, cerrar la puerta y volver. El tiempo total es la suma de todos los viajes. La solución es pedir todas las carpetas de planchas de una vez: un solo viaje con todo lo necesario. Include es como pedir todas las carpetas relacionadas en un solo viaje. ThenInclude es como pedir también las carpetas de las carpetas. El logging es como el registro de todos los viajes que se han hecho al archivo: si se ven muchos viajes idénticos con parámetros distintos, hay un problema N+1. Así funciona el problema N+1 en EF Core: se identifica con el logging, se mide con cronómetros y se resuelve con Include.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos del problema N+1 a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso ProblemaN1UseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes y planchas.

Ejecutado las demostraciones con N+1 y sin N+1.

Comparado el rendimiento de ambas variantes.

Analizado el log de EF Core y observado el patrón de N+1.

Diagnosticado el error de aplicar AsNoTracking después del Include.

Creado los métodos ObtenerOrdenesConDetalleN1 y ObtenerOrdenesConDetalleSinN1.

Punto 4.5 - Solución a N+1: Include, proyecciones y Split Queries

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que apliquen las soluciones al problema N+1: Include, ThenInclude, proyecciones y AsSplitQuery. Comparar el número de consultas y el tiempo de ejecución de cada técnica. Analizar el SQL generado.

Contexto del proyecto: En el punto 4.4 se identificó el problema N+1, observando su impacto en el rendimiento y analizando el SQL generado. En este punto se profundiza en las soluciones: Include, ThenInclude, proyecciones y AsSplitQuery. Estas soluciones se usarán en el punto 4.6 para el over-fetching y en el punto 4.8 para las Split Queries.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de solución al problema N+1 a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerConPlanchasInclude();
    List<OrdenFabricacion> ObtenerConAleacionesThenInclude();
    List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery();
    List<OrdenConPlanchasYDetalleDto> ObtenerResumenConPlanchasYDetalleProyeccion();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerConPlanchasInclude(); → método que usa Include para cargar las planchas.

Línea 11: List ObtenerConAleacionesThenInclude(); → método que usa ThenInclude para cargar las aleaciones.

Línea 12: List ObtenerConPlanchasYDetalleSplitQuery(); → método que usa AsSplitQuery para cargar planchas y detalle.

Línea 13: `List<OrdenFabricacion> ObtenerResumenConPlanchasYDetalleProyeccion();` → método que usa proyección para cargar un resumen con planchas y detalle.

Línea 15: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 3: Implementar los métodos de solución en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```

public List<OrdenFabricacion> ObtenerConPlanchasInclude()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerConAleacionesThenInclude()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenConPlanchasYDetalleDto> ObtenerResumenConPlanchasYDetalleProyeccion()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConPlanchasYDetalleDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        })
}
    
```

```

    })
    .ToList();
}

```

Línea 1: `public List ObtenerConPlanchasInclude()` → declara el método.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 5: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 6: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 7: `.ToList();` → materializa la consulta.

Línea 10: `public List ObtenerConAleacionesThenInclude()` → declara el método.

Línea 12: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 13: `.AsNoTrackingWithIdentityResolution()` → aplica resolución de identidad.

Línea 14: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 15: `.ThenInclude(oa => oa.Aleacion)` → incluye la aleación de cada entidad intermedia.

Línea 16: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 17: `.ToList();` → materializa la consulta.

Línea 20: `public List ObtenerConPlanchasYDetalleSplitQuery()` → declara el método.

Línea 22: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 23: `.AsNoTrackingWithIdentityResolution()` → aplica resolución de identidad.

Línea 24: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 25: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 26: `.AsSplitQuery()` → divide la consulta en varias.

Línea 27: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 28: `.ToList();` → materializa la consulta.

Línea 31: `public List ObtenerResumenConPlanchasYDetalleProyeccion()` → declara el método.

Línea 33: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 34: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 35: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 36: `.Select(o => new OrdenConPlanchasYDetalleDto` → proyecta al DTO.

Línea 37: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 38: `Cliente = o.Cliente,` → asigna el cliente.

Línea 39: `Estado = o.Estado,` → asigna el estado.

Línea 40: `Planchas = o.Planchas.Select(p => new PlanchaDto` → proyecta las planchas.

Línea 41: `Id = p.Id,` → asigna el Id.

Línea 42: `Espesor = p.Espesor,` → asigna el espesor.

Línea 43: `Ancho = p.Ancho,` → asigna el ancho.

Línea 44: `Largo = p.Largo,` → asigna el largo.

Línea 45: `Peso = p.Peso`, → asigna el peso.

Línea 46: `Activa = p.Activa` → asigna el estado activo.

Línea 47: `}).ToList()`, → materializa la colección.

Línea 48: `Detalle = o.Detalle == null ? null : new DetalleDto` → comprueba si el detalle existe.

Línea 49: `ComposicionQuimica = o.Detalle.ComposicionQuimica`, → asigna la composición química.

Línea 50: `TemperaturaColada = o.Detalle.TemperaturaColada`, → asigna la temperatura.

Línea 51: `Notas = o.Detalle.Notas` → asigna las notas.

Línea 52: `}` → cierra la proyección del detalle.

Línea 53: `})` → cierra la proyección.

Línea 54: `.ToList()`; → materializa la consulta.

Error común: si se usa `Include` con dos colecciones sin `AsSplitQuery`, EF Core genera un producto cartesiano. Se debe usar `AsSplitQuery` en esos casos.

▮ Paso 4: Crear el caso de uso de solución al problema N+1

Crear el archivo `src/AceriaData.Application/UseCases/SolucionN1UseCase.cs`:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class SolucionN1UseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public SolucionN1UseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== SOLUCIÓN AL PROBLEMA N+1 ===");

        DemostrarConInclude();
        DemostrarConThenInclude();
        DemostrarConSplitQuery();
        DemostrarConProyeccion();
        CompararTodasLasSoluciones();
    }

    private void DemostrarConInclude()
    {
        Console.WriteLine("\n--- Solución con Include ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerConPlanchasInclude();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas");
        }
    }

    private void DemostrarConThenInclude()
    {
        Console.WriteLine("\n--- Solución con ThenInclude ---");
```

```

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerConAleacionesThenInclude();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.OrdenesAleaciones.Count} aleaciones");
        }
    }

    private void DemostrarConSplitQuery()
    {
        Console.WriteLine("\n--- Solución con AsSplitQuery ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerConPlanchasYDetalleSplitQuery();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas | Detalle:
{detalle}");
        }
    }

    private void DemostrarConProyeccion()
    {
        Console.WriteLine("\n--- Solución con proyección ---");

        var cronometro = Stopwatch.StartNew();
        var resúmenes = _unidad.Ordenes.ObtenerResumenConPlanchasYDetalleProyeccion();
        cronometro.Stop();

        Console.WriteLine($"Resúmenes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds}
ms");
        foreach (var resumen in resúmenes)
        {
            var detalle = resumen.Detalle == null ? "Sin detalle" : resumen.Detalle.ComposicionQuimica;
            Console.WriteLine($"  {resumen.NumeroOrden}: {resumen.Planchas.Count} planchas | Detalle:
{detalle}");
        }
    }

    private void CompararTodasLasSoluciones()
    {
        Console.WriteLine("\n--- Comparación de todas las soluciones ---");

        var cronometroInclude = Stopwatch.StartNew();
        var ordenesInclude = _unidad.Ordenes.ObtenerConPlanchasInclude();
        cronometroInclude.Stop();

        var cronometroThenInclude = Stopwatch.StartNew();
        var ordenesThenInclude = _unidad.Ordenes.ObtenerConAleacionesThenInclude();
        cronometroThenInclude.Stop();

        var cronometroSplitQuery = Stopwatch.StartNew();
        var ordenesSplitQuery = _unidad.Ordenes.ObtenerConPlanchasYDetalleSplitQuery();
        cronometroSplitQuery.Stop();

        var cronometroProyeccion = Stopwatch.StartNew();
        var resúmenesProyeccion = _unidad.Ordenes.ObtenerResumenConPlanchasYDetalleProyeccion();
        cronometroProyeccion.Stop();

        Console.WriteLine($"Include: {cronometroInclude.ElapsedMilliseconds} ms");
        Console.WriteLine($"ThenInclude: {cronometroThenInclude.ElapsedMilliseconds} ms");
        Console.WriteLine($"AsSplitQuery: {cronometroSplitQuery.ElapsedMilliseconds} ms");
        Console.WriteLine($"Proyección: {cronometroProyeccion.ElapsedMilliseconds} ms");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class SolucionN1UseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public SolucionN1UseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== SOLUCIÓN AL PROBLEMA N+1 ==="); → muestra la cabecera.

Línea 19: DemostrarConInclude(); → llama al método de Include.

Línea 20: DemostrarConThenInclude(); → llama al método de ThenInclude.

Línea 21: DemostrarConSplitQuery(); → llama al método de AsSplitQuery.

Línea 22: DemostrarConProyeccion(); → llama al método de proyección.

Línea 23: CompararTodasLasSoluciones(); → llama al método de comparación.

Línea 26: private void DemostrarConInclude() → declara el método.

Línea 28: Console.WriteLine("\n--- Solución con Include ---"); → muestra la cabecera.

Línea 30: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 31: var ordenes = _unidad Ordenes.ObtenerConPlanchasInclude(); → carga las órdenes con Include.

Línea 32: cronometro.Stop(); → detiene el cronómetro.

Línea 34: Console.WriteLine(\$"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el número de órdenes y el tiempo.

Línea 35: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 37: Console.WriteLine(\$" {orden.NumeroOrden}: {orden.Planchas.Count} planchas"); → muestra el número de planchas.

Línea 40: private void DemostrarConThenInclude() → declara el método.

Línea 42: Console.WriteLine("\n--- Solución con ThenInclude ---"); → muestra la cabecera.

Línea 44: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 45: var ordenes = _unidad Ordenes.ObtenerConAleacionesThenInclude(); → carga las órdenes con ThenInclude.

Línea 46: cronometro.Stop(); → detiene el cronómetro.

Línea 48: Console.WriteLine(\$"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el número de órdenes y el tiempo.

Línea 49: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 51: Console.WriteLine(\$" {orden.NumeroOrden}: {orden.OrdenesAleaciones.Count} aleaciones"); → muestra el número de aleaciones.

Línea 54: private void DemostrarConSplitQuery() → declara el método.

Línea 56: Console.WriteLine("\n--- Solución con AsSplitQuery ---"); → muestra la cabecera.

Línea 58: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 59: `var ordenes = _unidad.Ordenes.ObtenerConPlanchasYDetalleSplitQuery();` → carga las órdenes con `AsSplitQuery`.

Línea 60: `cronometro.Stop();` → detiene el cronómetro.

Línea 62: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de órdenes y el tiempo.

Línea 63: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 65: `var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;` → obtiene el detalle.

Línea 66: `Console.WriteLine($" {orden.NumeroOrden}: {orden.Planchas.Count} planchas | Detalle: {detalle}");` → muestra los datos.

Línea 69: `private void DemostrarConProyeccion()` → declara el método.

Línea 71: `Console.WriteLine("\n--- Solución con proyección ---");` → muestra la cabecera.

Línea 73: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 74: `var resúmenes = _unidad.Ordenes.ObtenerResumenConPlanchasYDetalleProyeccion();` → carga los resúmenes con proyección.

Línea 75: `cronometro.Stop();` → detiene el cronómetro.

Línea 77: `Console.WriteLine($"Resúmenes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de resúmenes y el tiempo.

Línea 78: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 80: `var detalle = resumen.Detalle == null ? "Sin detalle" : resumen.Detalle.ComposicionQuimica;` → obtiene el detalle.

Línea 81: `Console.WriteLine($" {resumen.NumeroOrden}: {resumen.Planchas.Count} planchas | Detalle: {detalle}");` → muestra los datos.

Línea 85: `private void CompararTodasLasSoluciones()` → declara el método.

Línea 87: `Console.WriteLine("\n--- Comparación de todas las soluciones ---");` → muestra la cabecera.

Línea 89: `var cronometroInclude = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 90: `var ordenesInclude = _unidad.Ordenes.ObtenerConPlanchasInclude();` → carga las órdenes con `Include`.

Línea 91: `cronometroInclude.Stop();` → detiene el cronómetro.

Línea 93: `var cronometroThenInclude = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 94: `var ordenesThenInclude = _unidad.Ordenes.ObtenerConAleacionesThenInclude();` → carga las órdenes con `ThenInclude`.

Línea 95: `cronometroThenInclude.Stop();` → detiene el cronómetro.

Línea 97: `var cronometroSplitQuery = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 98: `var ordenesSplitQuery = _unidad.Ordenes.ObtenerConPlanchasYDetalleSplitQuery();` → carga las órdenes con `AsSplitQuery`.

Línea 99: `cronometroSplitQuery.Stop();` → detiene el cronómetro.

Línea 101: `var cronometroProyeccion = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 102: `var resúmenesProyeccion = _unidad.Ordenes.ObtenerResumenConPlanchasYDetalleProyeccion();` → carga los resúmenes con proyección.

Línea 103: `cronometroProyeccion.Stop();` → detiene el cronómetro.

Línea 105: `Console.WriteLine($"Include: {cronometroInclude.ElapsedMilliseconds} ms");` → muestra el tiempo de Include.

Línea 106: `Console.WriteLine($"ThenInclude: {cronometroThenInclude.ElapsedMilliseconds} ms");` → muestra el tiempo de ThenInclude.

Línea 107: `Console.WriteLine($"AsSplitQuery: {cronometroSplitQuery.ElapsedMilliseconds} ms");` → muestra el tiempo de AsSplitQuery.

Línea 108: `Console.WriteLine($"Proyección: {cronometroProyeccion.ElapsedMilliseconds} ms");` → muestra el tiempo de proyección.

Error común: si se ejecutan las consultas en el mismo DbContext, las entidades de las consultas anteriores permanecen en el Change Tracker. Se debe usar un DbContext distinto para cada consulta si se quiere medir el tiempo de forma aislada.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<SolucionN1UseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<SolucionN1UseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con planchas, detalle y aleaciones

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes, planchas, detalle y aleaciones:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var aleacion1 = new Aleacion { Nombre = "AISI 1045",Codigo = "A1045", PorcentajeCarbono = 0.45m,
    PorcentajeManganeso = 0.75m, Activo = true };
    var aleacion2 = new Aleacion { Nombre = "AISI 4140",Codigo = "A4140", PorcentajeCarbono = 0.40m,
    PorcentajeManganeso = 0.85m, Activo = true };
    context.Aleaciones.AddRange(aleacion1, aleacion2);
    context.SaveChanges();
}
```

```

var ordenes = new List<OrdenFabricacion>
{
    new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
    new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
    new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
    new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
    new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
    new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) }
};
context.OrdenesFabricacion.AddRange(ordenes);
context.SaveChanges();

var planchas = new List<PlanchaAcero>
{
    new PlanchaAcero { OrdenId = ordenes[0].Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true },
    new PlanchaAcero { OrdenId = ordenes[0].Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true },
    new PlanchaAcero { OrdenId = ordenes[1].Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true },
    new PlanchaAcero { OrdenId = ordenes[2].Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true },
    new PlanchaAcero { OrdenId = ordenes[3].Id, Espesor = 9.0, Ancho = 1100, Largo = 2200, Peso = 180.0m, Activa = true },
    new PlanchaAcero { OrdenId = ordenes[4].Id, Espesor = 11.0, Ancho = 1300, Largo = 2700, Peso = 290.0m, Activa = true }
};
context.PlanchasAcero.AddRange(planchas);

var detalle1 = new DetalleOrden { OrdenId = ordenes[0].Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%", TemperaturaColada = 1550.5 };
context.DetallesOrden.Add(detalle1);

var ordenAleacion1 = new OrdenAleacion { OrdenFabricacionId = ordenes[0].Id, AleacionId = aleacion1.Id, CantidadUtilizada = 1500.5m, EstadoRelacion = "Activa" };
var ordenAleacion2 = new OrdenAleacion { OrdenFabricacionId = ordenes[0].Id, AleacionId = aleacion2.Id, CantidadUtilizada = 800.0m, EstadoRelacion = "Activa" };
var ordenAleacion3 = new OrdenAleacion { OrdenFabricacionId = ordenes[1].Id, AleacionId = aleacion1.Id, CantidadUtilizada = 1200.0m, EstadoRelacion = "Activa" };
context.OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3);

context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var aleacion1 = new Aleacion { ... }; → crea la primera aleación.

Línea 8: var aleacion2 = new Aleacion { ... }; → crea la segunda aleación.

Línea 9: context.Aleaciones.AddRange(aleacion1, aleacion2); → registra las aleaciones.

Línea 10: context.SaveChanges(); → inserta las aleaciones.

Línea 12: var ordenes = new List → crea la lista de órdenes.

Línea 14: new OrdenFabricacion { ... }, → primera orden.

Línea 15: new OrdenFabricacion { ... }, → segunda orden.

Línea 16: `new OrdenFabricacion { ... },` → tercera orden.

Línea 17: `new OrdenFabricacion { ... },` → cuarta orden.

Línea 18: `new OrdenFabricacion { ... },` → quinta orden.

Línea 19: `new OrdenFabricacion { ... }` → sexta orden.

Línea 20: `};` → cierra la lista.

Línea 21: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 22: `context.SaveChanges();` → inserta las órdenes.

Línea 24: `var planchas = new List` → crea la lista de planchas.

Línea 26: `new PlanchaAcero { ... },` → primera plancha.

Línea 27: `new PlanchaAcero { ... },` → segunda plancha.

Línea 28: `new PlanchaAcero { ... },` → tercera plancha.

Línea 29: `new PlanchaAcero { ... },` → cuarta plancha.

Línea 30: `new PlanchaAcero { ... },` → quinta plancha.

Línea 31: `new PlanchaAcero { ... }` → sexta plancha.

Línea 32: `};` → cierra la lista.

Línea 33: `context.PlanchasAcero.AddRange(planchas);` → registra las planchas.

Línea 35: `var detalle1 = new DetalleOrden { ... };` → crea el detalle.

Línea 36: `context.DetallesOrden.Add(detalle1);` → registra el detalle.

Línea 38: `var ordenAleacion1 = new OrdenAleacion { ... };` → crea la primera relación.

Línea 39: `var ordenAleacion2 = new OrdenAleacion { ... };` → crea la segunda relación.

Línea 40: `var ordenAleacion3 = new OrdenAleacion { ... };` → crea la tercera relación.

Línea 41: `context.OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3);` → registra las relaciones.

Línea 43: `context.SaveChanges();` → inserta las planchas, el detalle y las relaciones.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

| Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada solución. Se observa el tiempo de cada técnica y el número de consultas ejecutadas en el log.

| Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```

=== SOLUCIÓN AL PROBLEMA N+1 ===

--- Solución con Include ---
Órdenes: 6 | Tiempo: 45 ms
  OF-2024-0001: 2 planchas
  OF-2024-0002: 1 planchas
  OF-2024-0003: 1 planchas
  OF-2024-0004: 1 planchas
  OF-2024-0005: 1 planchas
  OF-2024-0006: 0 planchas

--- Solución con ThenInclude ---
Órdenes: 6 | Tiempo: 48 ms
  OF-2024-0001: 2 aleaciones
  OF-2024-0002: 1 aleaciones
  OF-2024-0003: 0 aleaciones
  OF-2024-0004: 0 aleaciones
  OF-2024-0005: 0 aleaciones
  OF-2024-0006: 0 aleaciones

--- Solución con AsSplitQuery ---
Órdenes: 6 | Tiempo: 52 ms
  OF-2024-0001: 2 planchas | Detalle: C: 0.45%, Mn: 0.75%
  OF-2024-0002: 1 planchas | Detalle: Sin detalle
  ...

--- Solución con proyección ---
Resúmenes: 6 | Tiempo: 40 ms
  OF-2024-0001: 2 planchas | Detalle: C: 0.45%, Mn: 0.75%
  OF-2024-0002: 1 planchas | Detalle: Sin detalle
  ...

--- Comparación de todas las soluciones ---
Include: 45 ms
ThenInclude: 48 ms
AsSplitQuery: 52 ms
Proyección: 40 ms

```

La primera sección muestra el tiempo de Include. La segunda sección muestra el tiempo de ThenInclude. La tercera sección muestra el tiempo de AsSplitQuery. La cuarta sección muestra el tiempo de proyección. La quinta sección muestra la comparación.

Observaciones: la proyección es la técnica más rápida porque carga menos datos. AsSplitQuery es ligeramente más lenta porque ejecuta varias consultas. Include y ThenInclude tienen tiempos similares. La diferencia entre las técnicas es pequeña porque la tabla tiene pocas filas. En tablas con muchas filas, las diferencias son mayores.

| Paso 10: Diagnosticar un error común

Modificar el método ObtenerConPlanchasYDetalleSplitQuery para eliminar AsSplitQuery:

```

public List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

```

Resultado esperado: el código funciona, pero la consulta genera un producto cartesiano porque se incluyen dos colecciones. Cada plancha se combina con cada detalle. El número de filas del resultado es mayor que el número de órdenes.

Solución: añadir AsSplitQuery para dividir la consulta en varias.

```
public List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado con la solución: la consulta se divide en varias y no se produce el producto cartesiano.

Errores comunes del ejercicio

Error	Causa	Solución
N+1	No se usó Include	Usar Include
Producto cartesiano	Se incluyeron dos colecciones sin AsSplitQuery	Usar AsSplitQuery
Proyección sin ToList	Se olvidó materializar la colección	Añadir ToList dentro de la proyección
Tiempo alto	Se cargan todas las columnas	Usar proyección
Segundo nivel sin ThenInclude	Se accede a una propiedad de segundo nivel	Usar ThenInclude
Resolución de identidad no usada	Se cargan entidades relacionadas duplicadas	Usar AsNoTrackingWithIdentityResolution

Reto resuelto: Combinar Include, ThenInclude y AsSplitQuery

Reto: Crear un método en el repositorio que cargue las órdenes con sus planchas, su detalle y sus aleaciones en una sola operación. Usar Include, ThenInclude, AsNoTrackingWithIdentityResolution y AsSplitQuery. Comparar el tiempo con el problema N+1.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```
List<OrdenFabricacion> ObtenerOrdenesCompletasConSplitQuery();
```

Línea 1: List ObtenerOrdenesCompletasConSplitQuery(); → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerOrdenesCompletasConSplitQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerOrdenesCompletasConSplitQuery() → declara el método.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.AsNoTrackingWithIdentityResolution()` → aplica resolución de identidad.

Línea 5: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 6: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 7: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 8: `.ThenInclude(oa => oa.Aleacion)` → incluye la aleación de cada entidad intermedia.

Línea 9: `.AsSplitQuery()` → divide la consulta en varias.

Línea 10: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 11: `.ToList();` → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarCompletaConSplitQuery()
{
    Console.WriteLine("\n--- Completa con AsSplitQuery ---");

    var cronometro = Stopwatch.StartNew();
    var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletasConSplitQuery();
    cronometro.Stop();

    Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    foreach (var orden in ordenes)
    {
        var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;
        Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas |
{orden.OrdenesAleaciones.Count} aleaciones | Detalle: {detalle}");
    }
}
```

Línea 1: `private void DemostrarCompletaConSplitQuery()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Completa con AsSplitQuery ---");` → muestra la cabecera.

Línea 5: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 6: `var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletasConSplitQuery();` → carga las órdenes.

Línea 7: `cronometro.Stop();` → detiene el cronómetro.

Línea 9: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de órdenes y el tiempo.

Línea 10: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 12: `var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;` → obtiene el detalle.

Línea 13: `Console.WriteLine($" {orden.NumeroOrden}: {orden.Planchas.Count} planchas | {orden.OrdenesAleaciones.Count} aleaciones | Detalle: {detalle}");` → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarCompletaConSplitQuery();
```

| Paso 5: Ejecutar dotnet run y comparar el tiempo.

Resultado esperado: el método carga todas las entidades relacionadas en varias consultas separadas sin producto cartesiano. El tiempo es mayor que con Include simple porque se ejecutan más consultas, pero el número de filas transferidas es menor.

| Analogía final

Las soluciones al problema N+1 en una acería son como las distintas formas de pedir las carpetas al archivo central. Include es como pedir en un solo viaje la carpeta de la orden y todas sus carpetas de planchas. ThenInclude es como pedir también las carpetas de aleaciones que están dentro de las carpetas de planchas. Las proyecciones son como pedir solo los datos que se necesitan de cada carpeta, sin traer la carpeta completa. AsSplitQuery es como pedir en varios viajes separados las carpetas de órdenes, las de planchas y las de aleaciones, evitando que el archivo tenga que combinar todo en un solo montón. Cada técnica tiene su coste y su beneficio. Include es rápido cuando solo hay una colección. Las proyecciones son más eficientes cuando solo se necesitan algunos datos. AsSplitQuery es más eficiente cuando hay varias colecciones. La combinación de técnicas permite resolver el problema N+1 de forma óptima según el escenario.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de solución al problema N+1 a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso SolucionN1UseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con órdenes, planchas, detalle y aleaciones.

Ejecutado las demostraciones de Include, ThenInclude, AsSplitQuery y proyección.

Comparado el tiempo de cada solución.

Analizado el SQL generado por cada técnica.

Diagnosticado el error de eliminar AsSplitQuery.

Creado el método ObtenerOrdenesCompletasConSplitQuery.

Punto 4.6 - Over-fetching: causas y soluciones

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que demuestren el over-fetching y las soluciones. Comparar el volumen de datos y el tiempo de ejecución. Analizar el SQL generado por cada técnica. Aplicar proyecciones, filtros y paginación.

Contexto del proyecto: En el punto 4.5 se estudiaron las soluciones al problema N+1: Include, ThenInclude, proyecciones y AsSplitQuery. En este punto se profundiza en el over-fetching, que es el problema de cargar más datos de los necesarios. Esta técnica se usará en el punto 4.7 para las consultas ineficientes.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de over-fetching a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerOrdenesCompletas();
    List<OrdenResumenDto> ObtenerResumenesProyectados();
    List<OrdenResumenDto> ObtenerResumenesPaginados(int pagina, int tamanoPagina);
    List<OrdenResumenDto> ObtenerResumenesFiltradosYProyectados(string estado); void
    Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerOrdenesCompletas(); → método que carga las órdenes completas.

Línea 11: List ObtenerResumenesProyectados(); → método que carga los resúmenes con proyección.

Línea 12: List ObtenerResumenesPaginados(int pagina, int tamanoPagina); → método que carga los resúmenes con paginación.

Línea 13: List ObtenerResumenesFiltradosYProyectados(string estado); → método que carga los resúmenes con filtro y proyección.

Línea 15: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

■ Paso 3: Implementar los métodos de over-fetching en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepository.cs`:

```
public List<OrdenFabricacion> ObtenerOrdenesCompletas()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenResumenDto> ObtenerResumenesProyectados()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

public List<OrdenResumenDto> ObtenerResumenesPaginados(int pagina, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

public List<OrdenResumenDto> ObtenerResumenesFiltradosYProyectados(string estado)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}
```

Línea 1: `public List ObtenerOrdenesCompletas()` → declara el método.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 5: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 6: `.ToList();` → materializa la consulta con todas las columnas.

Línea 9: `public List ObtenerResumenesProyectados()` → declara el método.

Línea 11: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 12: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 13: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 14: `.Select(o => new OrdenResumenDto)` → proyecta al DTO.

Línea 15: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 16: `Cliente = o.Cliente,` → asigna el cliente.

Línea 17: `Estado = o.Estado,` → asigna el estado.

Línea 18: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 19: `}` → cierra la proyección.

Línea 20: `.ToList();` → materializa la consulta solo con las columnas proyectadas.

Línea 23: `public List ObtenerResumenesPaginados(int pagina, int tamanoPagina)` → declara el método.

Línea 25: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 26: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 27: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 28: `.Skip((pagina - 1) * tamanoPagina)` → salta las páginas anteriores.

Línea 29: `.Take(tamanoPagina)` → toma el tamaño de la página.

Línea 30: `.Select(o => new OrdenResumenDto)` → proyecta al DTO.

Línea 31: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 32: `Cliente = o.Cliente,` → asigna el cliente.

Línea 33: `Estado = o.Estado,` → asigna el estado.

Línea 34: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 35: `}` → cierra la proyección.

Línea 36: `.ToList();` → materializa la consulta con paginación y proyección.

Línea 39: `public List ObtenerResumenesFiltradosYProyectados(string estado)` → declara el método.

Línea 41: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 42: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 43: `.Where(o => o.Estado == estado)` → filtra por estado.

Línea 44: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 45: `.Select(o => new OrdenResumenDto)` → proyecta al DTO.

Línea 46: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 47: `Cliente = o.Cliente,` → asigna el cliente.

Línea 48: `Estado = o.Estado,` → asigna el estado.

Línea 49: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 50: `}` → cierra la proyección.

Línea 51: .ToList(); → materializa la consulta con filtro y proyección.

Error común: si se aplica Skip sin OrderBy, el resultado es indeterminado. Se debe aplicar OrderBy antes de Skip.

▮ Paso 4: Crear el caso de uso de over-fetching

Crear el archivo src/AceriaData.Application/UseCases/OverFetchingUseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class OverFetchingUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public OverFetchingUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== OVER-FETCHING ===");

        DemostrarEntidadesCompletas();
        DemostrarProyeccion();
        DemostrarPaginacion();
        DemostrarFiltroYProyeccion();
        CompararRendimiento();
    }

    private void DemostrarEntidadesCompletas()
    {
        Console.WriteLine("\n--- Entidades completas (over-fetching) ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad Ordenes.ObtenerOrdenesCompletas();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        }
    }

    private void DemostrarProyeccion()
    {
        Console.WriteLine("\n--- Proyección (sin over-fetching de columnas) ---");

        var cronometro = Stopwatch.StartNew();
        var resúmenes = _unidad Ordenes.ObtenerResúmenesProyectados();
        cronometro.Stop();

        Console.WriteLine($"Resúmenes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var resumen in resúmenes)
        {
            Console.WriteLine($"  {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");
        }
    }

    private void DemostrarPaginacion()
    {
        Console.WriteLine("\n--- Paginación (sin over-fetching de filas) ---");

        var cronometro = Stopwatch.StartNew();
        var resúmenes = _unidad Ordenes.ObtenerResúmenesPaginados(1, 3);
        cronometro.Stop();
    }
}
```

```

ms");
    Console.WriteLine($"Resúmenes: {resumenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds}");
    foreach (var resumen in resumenes)
    {
        Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");
    }
}

private void DemostrarFiltroYProyeccion()
{
    Console.WriteLine("\n--- Filtro y proyección ---");

    var cronometro = Stopwatch.StartNew();
    var resumenes = _unidad Ordenes.ObtenerResumenesFiltradosYProyectados("Pendiente");
    cronometro.Stop();

    Console.WriteLine($"Resúmenes pendientes: {resumenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    foreach (var resumen in resumenes)
    {
        Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");
    }
}

private void CompararRendimiento()
{
    Console.WriteLine("\n--- Comparación de rendimiento ---");

    var cronometroCompletas = Stopwatch.StartNew();
    var ordenesCompletas = _unidad Ordenes.ObtenerOrdenesCompletas();
    cronometroCompletas.Stop();

    var cronometroProyectadas = Stopwatch.StartNew();
    var resumenesProyectados = _unidad Ordenes.ObtenerResumenesProyectados();
    cronometroProyectadas.Stop();

    Console.WriteLine($"Entidades completas: {cronometroCompletas.ElapsedMilliseconds} ms | Registros: {ordenesCompletas.Count}");
    Console.WriteLine($"Proyección: {cronometroProyectadas.ElapsedMilliseconds} ms | Registros: {resumenesProyectados.Count}");
}
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class OverFetchingUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public OverFetchingUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== OVER-FETCHING ==="); → muestra la cabecera.

Línea 19: DemostrarEntidadesCompletas(); → llama al método de entidades completas.

Línea 20: DemostrarProyeccion(); → llama al método de proyección.

Línea 21: DemostrarPaginacion(); → llama al método de paginación.

Línea 22: DemostrarFiltroYProyeccion(); → llama al método de filtro y proyección.

Línea 23: CompararRendimiento(); → llama al método de comparación.

Línea 26: `private void DemostrarEntidadesCompletas()` → declara el método.

Línea 28: `Console.WriteLine("\n--- Entidades completas (over-fetching) ---");` → muestra la cabecera.

Línea 30: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 31: `var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletas();` → carga las órdenes completas.

Línea 32: `cronometro.Stop();` → detiene el cronómetro.

Línea 34: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de órdenes y el tiempo.

Línea 35: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 37: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");` → muestra los datos.

Línea 40: `private void DemostrarProyeccion()` → declara el método.

Línea 42: `Console.WriteLine("\n--- Proyección (sin over-fetching de columnas) ---");` → muestra la cabecera.

Línea 44: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 45: `var resúmenes = _unidad.Ordenes.ObtenerResúmenesProyectados();` → carga los resúmenes.

Línea 46: `cronometro.Stop();` → detiene el cronómetro.

Línea 48: `Console.WriteLine($"Resúmenes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de resúmenes y el tiempo.

Línea 49: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 51: `Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");` → muestra los datos.

Línea 54: `private void DemostrarPaginacion()` → declara el método.

Línea 56: `Console.WriteLine("\n--- Paginación (sin over-fetching de filas) ---");` → muestra la cabecera.

Línea 58: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 59: `var resúmenes = _unidad.Ordenes.ObtenerResúmenesPaginados(1, 3);` → carga los resúmenes paginados.

Línea 60: `cronometro.Stop();` → detiene el cronómetro.

Línea 62: `Console.WriteLine($"Resúmenes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de resúmenes y el tiempo.

Línea 63: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 65: `Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");` → muestra los datos.

Línea 68: `private void DemostrarFiltroYProyeccion()` → declara el método.

Línea 70: `Console.WriteLine("\n--- Filtro y proyección ---");` → muestra la cabecera.

Línea 72: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 73: `var resúmenes = _unidad.Ordenes.ObtenerResúmenesFiltradosYProyectados("Pendiente");` → carga los resúmenes filtrados y proyectados.

Línea 74: `cronometro.Stop();` → detiene el cronómetro.

Línea 76: `Console.WriteLine($"Resúmenes pendientes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de resúmenes y el tiempo.

Línea 77: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 79: `Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");` → muestra los datos.

Línea 83: `private void CompararRendimiento()` → declara el método.

Línea 85: `Console.WriteLine("\n--- Comparación de rendimiento ---");` → muestra la cabecera.

Línea 87: `var cronometroCompletas = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 88: `var ordenesCompletas = _unidad Ordenes.ObtenerOrdenesCompletas();` → carga las órdenes completas.

Línea 89: `cronometroCompletas.Stop();` → detiene el cronómetro.

Línea 91: `var cronometroProyectadas = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 92: `var resúmenesProyectados = _unidad Ordenes.ObtenerResúmenesProyectados();` → carga los resúmenes proyectados.

Línea 93: `cronometroProyectadas.Stop();` → detiene el cronómetro.

Línea 95: `Console.WriteLine($"Entidades completas: {cronometroCompletas.ElapsedMilliseconds} ms | Registros: {ordenesCompletas.Count}");` → muestra el tiempo y el número de registros.

Línea 96: `Console.WriteLine($"Proyección: {cronometroProyectadas.ElapsedMilliseconds} ms | Registros: {resúmenesProyectados.Count}");` → muestra el tiempo y el número de registros.

Error común: si se proyecta a un DTO con propiedades no asignadas, EF Core deja las propiedades con su valor por defecto. Se deben asignar todas las propiedades.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<OverFetchingUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<OverFetchingUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado =
        "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado =
        "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0007", Cliente = "Constructora del Este", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 7, 22) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0008", Cliente = "Constructora del Oeste", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 8, 30) }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... }, → quinta orden.

Línea 14: new OrdenFabricacion { ... }, → sexta orden.

Línea 15: new OrdenFabricacion { ... }, → séptima orden.

Línea 16: new OrdenFabricacion { ... } → octava orden.

Línea 17: }; → cierra la lista.

Línea 19: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 20: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

| Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de cada técnica y el SQL generado.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== OVER-FETCHING ===

--- Entidades completas (over-fetching) ---
Órdenes: 8 | Tiempo: 52 ms
  OF-2024-0001 | Constructora del Norte | Pendiente
  ...

--- Proyección (sin over-fetching de columnas) ---
Resúmenes: 8 | Tiempo: 40 ms
  OF-2024-0001 | Constructora del Norte | Pendiente
  ...

--- Paginación (sin over-fetching de filas) ---
Resúmenes: 3 | Tiempo: 38 ms
  OF-2024-0001 | Constructora del Norte | Pendiente
  ...

--- Filtro y proyección ---
Resúmenes pendientes: 5 | Tiempo: 39 ms
  OF-2024-0001 | Constructora del Norte | Pendiente
  ...

--- Comparación de rendimiento ---
Entidades completas: 52 ms | Registros: 8
Proyección: 40 ms | Registros: 8
```

La primera sección muestra el tiempo de cargar entidades completas. La segunda sección muestra el tiempo de cargar la proyección. La tercera sección muestra el tiempo de cargar la paginación. La cuarta sección muestra el tiempo de cargar el filtro y la proyección. La quinta sección muestra la comparación.

Observaciones: la proyección es más rápida que la carga de entidades completas. La paginación carga solo tres registros. El filtro y la proyección cargan cinco registros. En tablas con muchas filas y columnas, las diferencias son mayores.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerResúmenesPaginados para aplicar ToList antes del Skip y Take:

```
public List<OrdenResumenDto> ObtenerResúmenesPaginados(int pagina, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList()
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}
```

Resultado esperado: el código funciona, pero la consulta SQL carga todas las órdenes en memoria y después aplica Skip y Take en memoria. El over-fetching de filas no se resuelve porque todas las filas se transfieren.

Solución: aplicar Skip y Take antes de ToList.

```
public List<OrdenResumenDto> ObtenerResumenesPaginados(int pagina, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToListAsync();
}
```

Resultado esperado con la solución: el SQL incluye OFFSET y FETCH y solo se transfieren las filas de la página.

Errores comunes del ejercicio

Error	Causa	Solución
Over-fetching de columnas	Se cargan entidades completas	Usar proyecciones
Over-fetching de filas	Se cargan todas las filas	Usar filtros y paginación
ToList antes de Skip	Se materializa antes de paginar	Aplicar Skip y Take antes de ToList
Include innecesario	Se incluyen entidades relacionadas que no se usan	Usar proyecciones o quitar el Include
Columnas nvarchar(max)	Se cargan columnas sin límite de longitud	Configurar longitud máxima o proyectar
Proyección incompleta	Se olvidan propiedades del DTO	Asignar todas las propiedades

Reto resuelto: Comparar el volumen de datos de entidades completas vs proyección

Reto: Crear dos métodos en el repositorio que carguen las órdenes, uno con entidades completas y otro con proyección. Comparar el volumen de datos transferidos usando el SQL generado con ToQueryString.

Solución paso a paso:

Paso 1: Añadir los métodos a la interfaz IOrdenRepositorio:

```
string ObtenerSqlEntidadesCompleatas();
string ObtenerSqlProyeccionResumen();
```

Línea 1: string ObtenerSqlEntidadesCompleatas(); → declara el método.

Línea 2: string ObtenerSqlProyeccionResumen(); → declara el método.

Paso 2: Implementar los métodos en OrdenRepositorio:

```
public string ObtenerSqlEntidadesCompletas()
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden);

    return consulta.ToQueryString();
}

public string ObtenerSqlProyeccionResumen()
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        });

    return consulta.ToQueryString();
}
```

Línea 1: public string ObtenerSqlEntidadesCompletas() → declara el método.

Línea 3: var consulta = _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .OrderBy(o => o.NumeroOrden); → ordena por número de orden.

Línea 7: return consulta.ToQueryString(); → devuelve el SQL.

Línea 10: public string ObtenerSqlProyeccionResumen() → declara el método.

Línea 12: var consulta = _context.OrdenesFabricacion → inicia la consulta.

Línea 13: .AsNoTracking() → aplica AsNoTracking.

Línea 14: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 15: .Select(o => new OrdenResumenDto → proyecta al DTO.

Línea 16: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 17: Cliente = o.Cliente, → asigna el cliente.

Línea 18: Estado = o.Estado, → asigna el estado.

Línea 19: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 20: }); → cierra la proyección.

Línea 22: return consulta.ToQueryString(); → devuelve el SQL.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarSqlComparativo()
{
    Console.WriteLine("\n--- SQL de entidades completas ---");
    var sqlCompletas = _unidad Ordenes.ObtenerSqlEntidadesCompletas();
    Console.WriteLine(sqlCompletas);

    Console.WriteLine("\n--- SQL de proyección ---");
    var sqlProyeccion = _unidad Ordenes.ObtenerSqlProyeccionResumen();
    Console.WriteLine(sqlProyeccion);
}
```

Línea 1: private void DemostrarSqlComparativo() → declara el método.

Línea 3: Console.WriteLine("\n--- SQL de entidades completas ---"); → muestra la cabecera.

Línea 4: var sqlCompletas = _unidad Ordenes.ObtenerSqlEntidadesCompletas(); → obtiene el SQL.

Línea 5: Console.WriteLine(sqlCompletas); → imprime el SQL.

Línea 7: Console.WriteLine("\n--- SQL de proyección ---"); → muestra la cabecera.

Línea 8: var sqlProyeccion = _unidad Ordenes.ObtenerSqlProyeccionResumen(); → obtiene el SQL.

Línea 9: Console.WriteLine(sqlProyeccion); → imprime el SQL.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarSqlComparativo();
```

Paso 5: Ejecutar dotnet run y comparar el SQL.

Resultado esperado: el SQL de entidades completas incluye todas las columnas (Id, NumeroOrden, Cliente, FechaCreacion, Estado, Observaciones, IsDeleted, DeletedAt). El SQL de proyección solo incluye las columnas proyectadas (NumeroOrden, Cliente, Estado, FechaCreacion). La diferencia de volumen es notable.

Analogía final

El over-fetching en una acería es como pedir al archivo central todas las carpetas de todas las órdenes cuando solo se necesita el número de orden y el cliente de cada una. El archivo trae carpetas enteras con todos sus documentos, pero solo se usan dos datos de cada una. El over-fetching de columnas es como traer carpetas enteras cuando solo se necesitan dos datos. El over-fetching de filas es como traer todas las carpetas cuando solo se necesitan las de una página. Las proyecciones son como pedir solo los dos datos necesarios de cada carpeta. La paginación es como pedir solo las carpetas de una página. El filtro es como pedir solo las carpetas de un estado concreto. La combinación de proyección, filtro y paginación es como pedir solo los datos necesarios de las carpetas necesarias. Así funciona el over-fetching en EF Core: se identifica con el SQL generado, se mide con cronómetros y se resuelve con proyecciones, filtros y paginación.

Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de over-fetching a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso OverFetchingUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes.

Ejecutado las demostraciones de entidades completas, proyección, paginación y filtro con proyección.

Comparado el rendimiento de cada técnica.

Analizado el SQL generado por cada técnica.

Diagnosticado el error de aplicar ToList antes de Skip y Take.

Creado los métodos ObtenerSqlEntidadesCompletas y ObtenerSqlProyeccionResumen.

Punto 4.7 - Consultas ineficientes: filtros no traducibles y funciones en Where

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que demuestren consultas ineficientes por filtros no traducibles y funciones en Where. Reescribir las consultas para que se traduzcan completamente a SQL. Comparar el SQL generado y el tiempo de ejecución.

Contexto del proyecto: En el punto 4.6 se estudió el over-fetching, identificando sus causas y aplicando proyecciones, filtros y paginación. En este punto se profundiza en las consultas ineficientes, que son aquellas que no se traducen completamente a SQL o que impiden el uso de índices. Esta técnica se usará en el punto 4.8 para las Split Queries.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de consultas ineficientes a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerPorClienteConFuncion(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteSinFuncion(string cliente);
    List<OrdenFabricacion> ObtenerPorMetodoPersonalizado(string estado);
    List<OrdenFabricacion> ObtenerPorEstadoDirecto(string estado);
    string ObtenerSqlConFuncion(string cliente);
    string ObtenerSqlSinFuncion(string cliente);

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerPorClienteConFuncion(string cliente); → método que usa ToLower en Where.

Línea 11: List ObtenerPorClienteSinFuncion(string cliente); → método que compara directamente.

Línea 12: List ObtenerPorMetodoPersonalizado(string estado); → método que usa un método personalizado.

Línea 13: `List ObtenerPorEstadoDirecto(string estado);` → método que compara directamente.

Línea 14: `string ObtenerSqlConFuncion(string cliente);` → método que devuelve el SQL con función.

Línea 15: `string ObtenerSqlSinFuncion(string cliente);` → método que devuelve el SQL sin función.

Línea 17: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 18: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

Paso 3: Implementar los métodos de consultas ineficientes en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```
public List<OrdenFabricacion> ObtenerPorClienteConFuncion(string cliente)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Cliente.ToLower() == cliente.ToLower())
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerPorClienteSinFuncion(string cliente)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Cliente == cliente)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerPorMetodoPersonalizado(string estado)
{
    var ordenes = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();

    return ordenes.Where(o => EsEstadoValido(o.Estado, estado)).ToList();
}

public List<OrdenFabricacion> ObtenerPorEstadoDirecto(string estado)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public string ObtenerSqlConFuncion(string cliente)
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Cliente.ToLower() == cliente.ToLower());

    return consulta.ToQueryString();
}

public string ObtenerSqlSinFuncion(string cliente)
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Cliente == cliente);

    return consulta.ToQueryString();
}
```

```

    }

    private static bool EsEstadoValido(string estadoActual, string estadoBuscado)
    {
        return string.Equals(estadoActual, estadoBuscado, StringComparison.OrdinalIgnoreCase);
    }

```

Línea 1: public List ObtenerPorClienteConFuncion(string cliente) → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .Where(o => o.Cliente.ToLower() == cliente.ToLower()) → filtra con ToLower sobre la columna.

Línea 6: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 7: .ToList(); → materializa la consulta.

Línea 10: public List ObtenerPorClienteSinFuncion(string cliente) → declara el método.

Línea 12: return _context.OrdenesFabricacion → inicia la consulta.

Línea 13: .AsNoTracking() → aplica AsNoTracking.

Línea 14: .Where(o => o.Cliente == cliente) → filtra directamente.

Línea 15: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 16: .ToList(); → materializa la consulta.

Línea 19: public List ObtenerPorMetodoPersonalizado(string estado) → declara el método.

Línea 21: var ordenes = _context.OrdenesFabricacion → inicia la consulta.

Línea 22: .AsNoTracking() → aplica AsNoTracking.

Línea 23: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 24: .ToList(); → materializa todas las órdenes en memoria.

Línea 26: return ordenes.Where(o => EsEstadoValido(o.Estado, estado)).ToList(); → filtra en memoria con un método personalizado.

Línea 29: public List ObtenerPorEstadoDirecto(string estado) → declara el método.

Línea 31: return _context.OrdenesFabricacion → inicia la consulta.

Línea 32: .AsNoTracking() → aplica AsNoTracking.

Línea 33: .Where(o => o.Estado == estado) → filtra directamente.

Línea 34: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 35: .ToList(); → materializa la consulta.

Línea 38: public string ObtenerSqlConFuncion(string cliente) → declara el método.

Línea 40: var consulta = _context.OrdenesFabricacion → inicia la consulta.

Línea 41: .AsNoTracking() → aplica AsNoTracking.

Línea 42: .Where(o => o.Cliente.ToLower() == cliente.ToLower()); → filtra con ToLower.

Línea 44: return consulta.ToQueryString(); → devuelve el SQL.

Línea 47: public string ObtenerSqlSinFuncion(string cliente) → declara el método.

Línea 49: var consulta = _context.OrdenesFabricacion → inicia la consulta.

Línea 50: .AsNoTracking() → aplica AsNoTracking.

Línea 51: `.Where(o => o.Cliente == cliente);` → filtra directamente.

Línea 53: `return consulta.ToString();` → devuelve el SQL.

Línea 56: `private static bool EsEstadoValido(string estadoActual, string estadoBuscado)` → declara el método personalizado.

Línea 58: `return string.Equals(estadoActual, estadoBuscado, StringComparison.OrdinalIgnoreCase);` → compara sin distinguir mayúsculas.

Error común: si el método personalizado se usa directamente en `Where` sobre `IQueryable`, EF Core lanza una excepción indicando que la expresión no se puede traducir. Se debe materializar antes o reescribir la consulta.

Paso 4: Crear el caso de uso de consultas ineficientes

Crear el archivo `src/AceriaData.Application/UseCases/ConsultasIneficientesUseCase.cs`:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ConsultasIneficientesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ConsultasIneficientesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CONSULTAS INEFICIENTES ===");

        DemostrarConFuncion();
        DemostrarSinFuncion();
        DemostrarMetodoPersonalizado();
        DemostrarEstadoDirecto();
        CompararSql();
    }

    private void DemostrarConFuncion()
    {
        Console.WriteLine("\n--- Consulta con función en Where ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerPorClienteConFuncion("Constructora del Norte");
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    }

    private void DemostrarSinFuncion()
    {
        Console.WriteLine("\n--- Consulta sin función en Where ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerPorClienteSinFuncion("Constructora del Norte");
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    }

    private void DemostrarMetodoPersonalizado()
    {
        Console.WriteLine("\n--- Consulta con método personalizado ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerPorMetodoPersonalizado("Pendiente");
    }
}
```

```

        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    }

    private void DemostrarEstadoDirecto()
    {
        Console.WriteLine("\n--- Consulta con estado directo ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerPorEstadoDirecto("Pendiente");
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    }

    private void CompararSql()
    {
        Console.WriteLine("\n--- SQL con función ---");
        var sqlConFuncion = _unidad.Ordenes.ObtenerSqlConFuncion("Constructora del Norte");
        Console.WriteLine(sqlConFuncion);

        Console.WriteLine("\n--- SQL sin función ---");
        var sqlSinFuncion = _unidad.Ordenes.ObtenerSqlSinFuncion("Constructora del Norte");
        Console.WriteLine(sqlSinFuncion);
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class ConsultasIneficientesUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public ConsultasIneficientesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== CONSULTAS INEFICIENTES ==="); → muestra la cabecera.

Línea 19: DemostrarConFuncion(); → llama al método con función.

Línea 20: DemostrarSinFuncion(); → llama al método sin función.

Línea 21: DemostrarMetodoPersonalizado(); → llama al método con método personalizado.

Línea 22: DemostrarEstadoDirecto(); → llama al método con estado directo.

Línea 23: CompararSql(); → llama al método de comparación de SQL.

Línea 26: private void DemostrarConFuncion() → declara el método.

Línea 28: Console.WriteLine("\n--- Consulta con función en Where ---"); → muestra la cabecera.

Línea 30: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 31: var ordenes = _unidad.Ordenes.ObtenerPorClienteConFuncion("Constructora del Norte"); → llama al método del repositorio.

Línea 32: cronometro.Stop(); → detiene el cronómetro.

Línea 34: Console.WriteLine(\$"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra los resultados.

Línea 37: private void DemostrarSinFuncion() → declara el método.

Línea 39: `Console.WriteLine("\n--- Consulta sin función en Where ---");` → muestra la cabecera.

Línea 41: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 42: `var ordenes = _unidad.Ordenes.ObtenerPorClienteSinFuncion("Constructora del Norte");` → llama al método del repositorio.

Línea 43: `cronometro.Stop();` → detiene el cronómetro.

Línea 45: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra los resultados.

Línea 48: `private void DemostrarMetodoPersonalizado()` → declara el método.

Línea 50: `Console.WriteLine("\n--- Consulta con método personalizado ---");` → muestra la cabecera.

Línea 52: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 53: `var ordenes = _unidad.Ordenes.ObtenerPorMetodoPersonalizado("Pendiente");` → llama al método del repositorio.

Línea 54: `cronometro.Stop();` → detiene el cronómetro.

Línea 56: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra los resultados.

Línea 59: `private void DemostrarEstadoDirecto()` → declara el método.

Línea 61: `Console.WriteLine("\n--- Consulta con estado directo ---");` → muestra la cabecera.

Línea 63: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 64: `var ordenes = _unidad.Ordenes.ObtenerPorEstadoDirecto("Pendiente");` → llama al método del repositorio.

Línea 65: `cronometro.Stop();` → detiene el cronómetro.

Línea 67: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra los resultados.

Línea 70: `private void CompararSql()` → declara el método.

Línea 72: `Console.WriteLine("\n--- SQL con función ---");` → muestra la cabecera.

Línea 73: `var sqlConFuncion = _unidad.Ordenes.ObtenerSqlConFuncion("Constructora del Norte");` → obtiene el SQL.

Línea 74: `Console.WriteLine(sqlConFuncion);` → imprime el SQL.

Línea 76: `Console.WriteLine("\n--- SQL sin función ---");` → muestra la cabecera.

Línea 77: `var sqlSinFuncion = _unidad.Ordenes.ObtenerSqlSinFuncion("Constructora del Norte");` → obtiene el SQL.

Línea 78: `Console.WriteLine(sqlSinFuncion);` → imprime el SQL.

Error común: si el método `EsEstadoValido` se usa directamente en `Where` sobre `IQueryable`, EF Core lanza una excepción. Se debe materializar antes.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ConsultasIneficientesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ConsultasIneficientesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

Paso 7: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) }
    };

    context.OrganizacionFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... }, → quinta orden.

Línea 14: new OrdenFabricacion { ... } → sexta orden.

Línea 15: }; → cierra la lista.

Línea 17: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 18: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de cada técnica y el SQL generado.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== CONSULTAS INEFICIENTES ===

--- Consulta con función en Where ---
Órdenes: 2 | Tiempo: 48 ms

--- Consulta sin función en Where ---
Órdenes: 2 | Tiempo: 40 ms

--- Consulta con método personalizado ---
Órdenes: 4 | Tiempo: 55 ms

--- Consulta con estado directo ---
Órdenes: 4 | Tiempo: 41 ms

--- SQL con función ---
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ...
FROM [OrdenesFabricacion] AS [o]
WHERE LOWER([o].[Cliente]) = @__ToLower_0 AND [o].[IsDeleted] = CAST(0 AS bit)

--- SQL sin función ---
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Cliente] = @__cliente_0 AND [o].[IsDeleted] = CAST(0 AS bit)
```

La primera sección muestra el tiempo con función. La segunda sección muestra el tiempo sin función. La tercera sección muestra el tiempo con método personalizado. La cuarta sección muestra el tiempo con estado directo. La quinta sección muestra el SQL con función. La sexta sección muestra el SQL sin función.

Observaciones: la consulta con función es más lenta porque SQL Server no puede usar el índice. La consulta con método personalizado es más lenta porque carga todas las órdenes en memoria y después filtra. El SQL con función incluye LOWER([o].[Cliente])). El SQL sin función compara directamente la columna.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerPorMetodoPersonalizado para usar el método personalizado directamente en Where:

```
public List<OrdenFabricacion> ObtenerPorMetodoPersonalizado(string estado)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => EsEstadoValido(o.Estado, estado))
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado: EF Core lanza una excepción `InvalidOperationException` indicando que la expresión `EsEstadoValido` no se puede traducir a SQL.

Solución: materializar antes de aplicar el método personalizado o reescribir la consulta con una expresión traducible.

```
public List<OrdenFabricacion> ObtenerPorMetodoPersonalizado(string estado)
{
    var ordenes = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();

    return ordenes.Where(o => EsEstadoValido(o.Estado, estado)).ToList();
}
```

Resultado esperado con la solución: el método funciona y filtra en memoria. La desventaja es que carga todas las órdenes en memoria.

Errores comunes del ejercicio

Error	Causa	Solución
Función en Where	Se aplica <code>ToLower</code> sobre la columna	Comparar directamente o usar <code>collation</code>
Método personalizado en Where	EF Core no puede traducirlo	Materializar antes o reescribir
Expresión regular en Where	EF Core no puede traducirla	Reescribir con <code>StartsWith</code> , <code>EndsWith</code> o <code>Contains</code>
Filtro en memoria	Se materializa antes de filtrar	Filtrar antes de materializar
Índice no usado	Se aplica una función sobre la columna	Evitar funciones en Where
Collation incorrecta	La <code>collation</code> es sensible a mayúsculas	Configurar <code>collation</code> insensible

Reto resuelto: Reescribir una consulta con expresión regular

Reto: Reescribir una consulta que usa una expresión regular para validar el formato del número de orden (OF-YYYY-NNNN) usando expresiones que EF Core pueda traducir a SQL.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz `IOrdenRepositorio`:

```
List<OrdenFabricacion> ObtenerPorFormatoNumeroOrden();
```

Línea 1: `List ObtenerPorFormatoNumeroOrden();` → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerPorFormatoNumeroOrden()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.NumeroOrden.StartsWith("OF-") && o.NumeroOrden.Length == 12)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerPorFormatoNumeroOrden() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .Where(o => o.NumeroOrden.StartsWith("OF-") && o.NumeroOrden.Length == 12) → filtra por formato usando expresiones traducibles.

Línea 6: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 7: .ToList(); → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarFormatoNumeroOrden()
{
    Console.WriteLine("\n--- Formato de número de orden ---");

    var ordenes = _unidad.Ordenes.ObtenerPorFormatoNumeroOrden();
    Console.WriteLine($"Órdenes con formato válido: {ordenes.Count}");
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden}");
    }
}
```

Línea 1: private void DemostrarFormatoNumeroOrden() → declara el método.

Línea 3: Console.WriteLine("\n--- Formato de número de orden ---"); → muestra la cabecera.

Línea 5: var ordenes = _unidad.Ordenes.ObtenerPorFormatoNumeroOrden(); → llama al método del repositorio.

Línea 6: Console.WriteLine(\$"Órdenes con formato válido: {ordenes.Count}"); → muestra el número de órdenes.

Línea 7: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 9: Console.WriteLine(\$" {orden.NumeroOrden}"); → muestra el número.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarFormatoNumeroOrden();
```

Paso 5: Ejecutar dotnet run y verificar que las órdenes con formato válido se muestran correctamente.

Resultado esperado: las órdenes con formato OF-YYYY-NNNN se muestran correctamente. La consulta se traduce completamente a SQL usando LIKE 'OF-%' y LEN.

| Analogía final

Las consultas ineficientes en una acería son como pedir al archivo central que busque las carpetas cuyo nombre, al aplicarle una regla personalizada, coincida con un patrón. Si la regla es estándar, el archivo la entiende y busca directamente en el índice. Si la regla es personalizada, el archivo tiene que traer todas las carpetas y aplicar la regla él mismo, una por una. Las funciones en Where son como pedir que busque las carpetas cuyo nombre, convertido a minúsculas, coincida con un patrón. El archivo no puede usar el índice porque los nombres están en mayúsculas y minúsculas. Tiene que recorrer todas las carpetas. La solución es pedir que busque directamente por el nombre, sin conversiones, o configurar el archivo para que las comparaciones sean insensibles a mayúsculas. Los métodos personalizados son como pedir que busque las carpetas que cumplan una regla que solo el jefe conoce. El archivo no entiende la regla y tiene que traer todas las carpetas para que el jefe las revise. Así funciona la optimización de consultas en EF Core: se reescriben las expresiones no traducibles con expresiones que EF Core puede convertir a SQL, y se evitan las funciones sobre columnas para que los índices se usen.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de consultas ineficientes a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso ConsultasIneficientesUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes.

Ejecutado las demostraciones con función, sin función, con método personalizado y con estado directo.

Comparado el SQL generado con y sin función.

Diagnosticado el error de usar un método personalizado en Where.

Creado el método ObtenerPorFormatoNumeroOrden con expresiones traducibles.

Punto 4.8 - Split Queries: cuándo y cómo usarlas

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que demuestren el producto cartesiano, AsSplitQuery y AsSingleQuery. Comparar el número de consultas, el volumen de datos y el tiempo de ejecución. Configurar el comportamiento por defecto y la advertencia de producto cartesiano.

Contexto del proyecto: En el punto 4.7 se estudiaron las consultas ineficientes, incluyendo los filtros no traducibles y las funciones en Where. En este punto se profundiza en las Split Queries, que permiten evitar el producto cartesiano cuando se incluyen varias colecciones. Esta técnica se usará en el punto 4.9 para las Compiled Queries.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de Split Queries a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerConVariasColeccionesSingleQuery();
    List<OrdenFabricacion> ObtenerConVariasColeccionesSplitQuery();
    string ObtenerSqlSingleQuery();
    string ObtenerSqlSplitQuery();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerConVariasColeccionesSingleQuery(); → método que usa AsSingleQuery.

Línea 11: List ObtenerConVariasColeccionesSplitQuery(); → método que usa AsSplitQuery.

Línea 12: string ObtenerSqlSingleQuery(); → método que devuelve el SQL de AsSingleQuery.

Línea 13: string ObtenerSqlSplitQuery(); → método que devuelve el SQL de AsSplitQuery.

Línea 15: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 16: void Eliminar(OrdenFabricacion orden); → método que elimina una orden.

Error común: si los métodos devuelven IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 3: Implementar los métodos de Split Queries en el repositorio

```

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs: \csharp public List
ObtenerConVariasColeccionesSingleQuery() { return
_context.OrdenesFabricacion .AsNoTrackingWithIdentityResolution() .Include(o => o.Planchas) .Include(o
=> o.OrdenesAleaciones) .ThenInclude(oa => oa.Aleacion) .AsSingleQuery() .OrderBy(o =>
o.NumeroOrden) .ToList(); }
  
```

```

public List ObtenerConVariasColeccionesSplitQuery() { return
_context.OrdenesFabricacion .AsNoTrackingWithIdentityResolution() .Include(o => o.Planchas) .Include(o
=> o.OrdenesAleaciones) .ThenInclude(oa => oa.Aleacion) .AsSplitQuery() .OrderBy(o =>
o.NumeroOrden) .ToList(); }
  
```

```

public string ObtenerSqlSingleQuery() { var consulta =
_context.OrdenesFabricacion .AsNoTracking() .Include(o => o.Planchas) .Include(o =>
o.OrdenesAleaciones) .AsSingleQuery();
  
```

```
return consulta.ToQueryString();
```

```
}
```

```

public string ObtenerSqlSplitQuery() { var consulta =
_context.OrdenesFabricacion .AsNoTracking() .Include(o => o.Planchas) .Include(o =>
o.OrdenesAleaciones) .AsSplitQuery();
  
```

```
return consulta.ToQueryString();
```

} Línea 1: public List ObtenerConVariasColeccionesSingleQuery() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTrackingWithIdentityResolution() → aplica resolución de identidad.

Línea 5: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 6: .Include(o => o.OrdenesAleaciones) → incluye la colección de entidades intermedias.

Línea 7: .ThenInclude(oa => oa.Aleacion) → incluye la aleación de cada entidad intermedia.

Línea 8: .AsSingleQuery() → fuerza una sola consulta.

Línea 9: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 10: .ToList(); → materializa la consulta.

Línea 13: public List ObtenerConVariasColeccionesSplitQuery() → declara el método.

Línea 15: return _context.OrdenesFabricacion → inicia la consulta.

Línea 16: .AsNoTrackingWithIdentityResolution() → aplica resolución de identidad.

Línea 17: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 18: .Include(o => o.OrdenesAleaciones) → incluye la colección de entidades intermedias.

Línea 19: .ThenInclude(oa => oa.Aleacion) → incluye la aleación de cada entidad intermedia.

Línea 20: .AsSplitQuery() → divide la consulta en varias.

Línea 21: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 22: .ToList(); → materializa la consulta.

Línea 25: `public string ObtenerSqlSingleQuery()` → declara el método.

Línea 27: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 28: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 29: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 30: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 31: `.AsSingleQuery();` → fuerza una sola consulta.

Línea 33: `return consulta.ToQueryString();` → devuelve el SQL.

Línea 36: `public string ObtenerSqlSplitQuery()` → declara el método.

Línea 38: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 39: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 40: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 41: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 42: `.AsSplitQuery();` → divide la consulta en varias.

Línea 44: `return consulta.ToQueryString();` → devuelve el SQL.

Error común: si se usa `AsSplitQuery` con una sola colección, no aporta beneficio porque no hay producto cartesiano. Se debe usar solo con varias colecciones.

| Paso 4: Crear el caso de uso de Split Queries

Crear el archivo `src/AceriaData.Application/UseCases/SplitQueriesUseCase.cs`:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class SplitQueriesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public SplitQueriesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== SPLIT QUERIES ===");

        DemostrarSingleQuery();
        DemostrarSplitQuery();
        CompararRendimiento();
        MostrarSql();
    }

    private void DemostrarSingleQuery()
    {
        Console.WriteLine("\n-- Single Query (producto cartesiano) --");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerConVariasColeccionesSingleQuery();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas |
```

```

{orden.OrdenesAleaciones.Count} aleaciones");
    }
}

private void DemostrarSplitQuery()
{
    Console.WriteLine("\n--- Split Query (sin producto cartesiano) ---");

    var cronometro = Stopwatch.StartNew();
    var ordenes = _unidad.Ordenes.ObtenerConVariasColeccionesSplitQuery();
    cronometro.Stop();

    Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden}: {orden.Planchas.Count} planchas |
{orden.OrdenesAleaciones.Count} aleaciones");
    }
}

private void CompararRendimiento()
{
    Console.WriteLine("\n--- Comparación de rendimiento ---");

    var cronometroSingle = Stopwatch.StartNew();
    var ordenesSingle = _unidad.Ordenes.ObtenerConVariasColeccionesSingleQuery();
    cronometroSingle.Stop();

    var cronometroSplit = Stopwatch.StartNew();
    var ordenesSplit = _unidad.Ordenes.ObtenerConVariasColeccionesSplitQuery();
    cronometroSplit.Stop();

    Console.WriteLine($"Single Query: {cronometroSingle.ElapsedMilliseconds} ms | Órdenes:
{ordenesSingle.Count}");
    Console.WriteLine($"Split Query: {cronometroSplit.ElapsedMilliseconds} ms | Órdenes:
{ordenesSplit.Count}");
}

private void MostrarSql()
{
    Console.WriteLine("\n--- SQL de Single Query ---");
    var sqlSingle = _unidad.Ordenes.ObtenerSqlSingleQuery();
    Console.WriteLine(sqlSingle);

    Console.WriteLine("\n--- SQL de Split Query ---");
    var sqlSplit = _unidad.Ordenes.ObtenerSqlSplitQuery();
    Console.WriteLine(sqlSplit);
}
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class SplitQueriesUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public SplitQueriesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== SPLIT QUERIES ==="); → muestra la cabecera.

Línea 19: DemostrarSingleQuery(); → llama al método de SingleQuery.

Línea 20: DemostrarSplitQuery(); → llama al método de SplitQuery.

Línea 21: CompararRendimiento(); → llama al método de comparación.

Línea 22: MostrarSql(); → llama al método de SQL.

Línea 25: private void DemostrarSingleQuery() → declara el método.

Línea 27: Console.WriteLine("\n--- Single Query (producto cartesiano) ---"); → muestra la cabecera.

Línea 29: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 30: var ordenes = _unidad.Ordenes.ObtenerConVariacionesSingleQuery(); → carga las órdenes con AsSingleQuery.

Línea 31: cronometro.Stop(); → detiene el cronómetro.

Línea 33: Console.WriteLine(\$"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el número de órdenes y el tiempo.

Línea 34: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 36: Console.WriteLine(\$" {orden.NumeroOrden}: {orden.Planchas.Count} planchas | {orden.OrdenesAleaciones.Count} aleaciones"); → muestra los datos.

Línea 39: private void DemostrarSplitQuery() → declara el método.

Línea 41: Console.WriteLine("\n--- Split Query (sin producto cartesiano) ---"); → muestra la cabecera.

Línea 43: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 44: var ordenes = _unidad.Ordenes.ObtenerConVariacionesSplitQuery(); → carga las órdenes con AsSplitQuery.

Línea 45: cronometro.Stop(); → detiene el cronómetro.

Línea 47: Console.WriteLine(\$"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el número de órdenes y el tiempo.

Línea 48: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 50: Console.WriteLine(\$" {orden.NumeroOrden}: {orden.Planchas.Count} planchas | {orden.OrdenesAleaciones.Count} aleaciones"); → muestra los datos.

Línea 54: private void CompararRendimiento() → declara el método.

Línea 56: Console.WriteLine("\n--- Comparación de rendimiento ---"); → muestra la cabecera.

Línea 58: var cronometroSingle = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 59: var ordenesSingle = _unidad.Ordenes.ObtenerConVariacionesSingleQuery(); → carga las órdenes con AsSingleQuery.

Línea 60: cronometroSingle.Stop(); → detiene el cronómetro.

Línea 62: var cronometroSplit = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 63: var ordenesSplit = _unidad.Ordenes.ObtenerConVariacionesSplitQuery(); → carga las órdenes con AsSplitQuery.

Línea 64: cronometroSplit.Stop(); → detiene el cronómetro.

Línea 66: Console.WriteLine(\$"Single Query: {cronometroSingle.ElapsedMilliseconds} ms | Órdenes: {ordenesSingle.Count}"); → muestra el tiempo de SingleQuery.

Línea 67: Console.WriteLine(\$"Split Query: {cronometroSplit.ElapsedMilliseconds} ms | Órdenes: {ordenesSplit.Count}"); → muestra el tiempo de SplitQuery.

Línea 70: private void MostrarSql() → declara el método.

Línea 72: Console.WriteLine("\n--- SQL de Single Query ---"); → muestra la cabecera.

Línea 73: `var sqlSingle = _unidad Ordenes.ObtenerSqlSingleQuery();` → obtiene el SQL.

Línea 74: `Console.WriteLine(sqlSingle);` → imprime el SQL.

Línea 76: `Console.WriteLine("\n--- SQL de Split Query ---");` → muestra la cabecera.

Línea 77: `var sqlSplit = _unidad Ordenes.ObtenerSqlSplitQuery();` → obtiene el SQL.

Línea 78: `Console.WriteLine(sqlSplit);` → imprime el SQL.

Error común: si se ejecutan las dos consultas en el mismo DbContext, las entidades de la primera consulta permanecen en el Change Tracker. Se debe usar un DbContext distinto para cada consulta si se quiere medir el tiempo de forma aislada.

Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<SplitQueriesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<SplitQueriesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

Paso 7: Insertar datos de prueba con varias planchas y aleaciones

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes, planchas y aleaciones:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var aleacion1 = new Aleacion { Nombre = "AISI 1045",Codigo = "A1045", PorcentajeCarbono = 0.45m,
    PorcentajeManganeso = 0.75m, Activo = true };
    var aleacion2 = new Aleacion { Nombre = "AISI 4140",Codigo = "A4140", PorcentajeCarbono = 0.40m,
    PorcentajeManganeso = 0.85m, Activo = true };
    context.Aleaciones.AddRange(aleacion1, aleacion2);
    context.SaveChanges();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del
    Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur",
    Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
```

```

var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del
Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };
context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);
context.SaveChanges();

var planchas = new List<PlanchaAcero>
{
    new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso =
370.5m, Activa = true },
    new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso =
280.8m, Activa = true },
    new PlanchaAcero { OrdenId = orden1.Id, Espesor = 9.0, Ancho = 1100, Largo = 2200, Peso =
180.0m, Activa = true },
    new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso =
125.6m, Activa = true },
    new PlanchaAcero { OrdenId = orden2.Id, Espesor = 11.0, Ancho = 1300, Largo = 2700, Peso =
290.0m, Activa = true },
    new PlanchaAcero { OrdenId = orden3.Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso =
450.0m, Activa = true }
};
context.PlanchasAcero.AddRange(planchas);

var ordenAleaciones = new List<OrdenAleacion>
{
    new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion1.Id,
CantidadUtilizada = 1500.5m, EstadoRelacion = "Activa" },
    new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion2.Id,
CantidadUtilizada = 800.0m, EstadoRelacion = "Activa" },
    new OrdenAleacion { OrdenFabricacionId = orden2.Id, AleacionId = aleacion1.Id,
CantidadUtilizada = 1200.0m, EstadoRelacion = "Activa" },
    new OrdenAleacion { OrdenFabricacionId = orden3.Id, AleacionId = aleacion2.Id,
CantidadUtilizada = 900.0m, EstadoRelacion = "Activa" }
};
context.OrdenesAleaciones.AddRange(ordenAleaciones);

context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var aleacion1 = new Aleacion { ... }; → crea la primera aleación.

Línea 8: var aleacion2 = new Aleacion { ... }; → crea la segunda aleación.

Línea 9: context.Aleaciones.AddRange(aleacion1, aleacion2); → registra las aleaciones.

Línea 10: context.SaveChanges(); → inserta las aleaciones.

Línea 12: var orden1 = new OrdenFabricacion { ... }; → crea la primera orden.

Línea 13: var orden2 = new OrdenFabricacion { ... }; → crea la segunda orden.

Línea 14: var orden3 = new OrdenFabricacion { ... }; → crea la tercera orden.

Línea 15: context.OrdenesFabricacion.AddRange(orden1, orden2, orden3); → registra las órdenes.

Línea 16: context.SaveChanges(); → inserta las órdenes.

Línea 18: var planchas = new List → crea la lista de planchas.

Línea 20: new PlanchaAcero { ... }, → primera plancha.

Línea 21: new PlanchaAcero { ... }, → segunda plancha.

Línea 22: new PlanchaAcero { ... }, → tercera plancha.

Línea 23: new PlanchaAcero { ... }, → cuarta plancha.

Línea 24: new PlanchaAcero { ... }, → quinta plancha.

Línea 25: new PlanchaAcero { ... } → sexta plancha.

Línea 26: }; → cierra la lista.

Línea 27: context.PlanchasAcero.AddRange(planchas); → registra las planchas.

Línea 29: var ordenAleaciones = new List → crea la lista de relaciones.

Línea 31: new OrdenAleacion { ... }, → primera relación.

Línea 32: new OrdenAleacion { ... }, → segunda relación.

Línea 33: new OrdenAleacion { ... }, → tercera relación.

Línea 34: new OrdenAleacion { ... } → cuarta relación.

Línea 35: }; → cierra la lista.

Línea 36: context.OrdenesAleaciones.AddRange(ordenAleaciones); → registra las relaciones.

Línea 38: context.SaveChanges(); → inserta las planchas y las relaciones.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea OrdenId no tiene un valor válido y se produce una violación de integridad referencial.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de SingleQuery y SplitQuery, y el SQL generado por cada técnica.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== SPLIT QUERIES ===

--- Single Query (producto cartesiano) ---
Órdenes: 3 | Tiempo: 55 ms
  OF-2024-0001: 3 planchas | 2 aleaciones
  OF-2024-0002: 2 planchas | 1 aleaciones
  OF-2024-0003: 1 planchas | 1 aleaciones

--- Split Query (sin producto cartesiano) ---
Órdenes: 3 | Tiempo: 48 ms
  OF-2024-0001: 3 planchas | 2 aleaciones
  OF-2024-0002: 2 planchas | 1 aleaciones
  OF-2024-0003: 1 planchas | 1 aleaciones

--- Comparación de rendimiento ---
Single Query: 55 ms | Órdenes: 3
Split Query: 48 ms | Órdenes: 3

--- SQL de Single Query ---
SELECT [o].[Id], [o].[NumeroOrden], ..., [p].[Id], [p].[Espesor], ..., [oa].[OrdenFabricacionId], [oa].[AleacionId], ..., [a].[Id], [a].[Nombre], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
LEFT JOIN [OrdenesAleaciones] AS [oa] ON [o].[Id] = [oa].[OrdenFabricacionId]
LEFT JOIN [Aleaciones] AS [a] ON [oa].[AleacionId] = [a].[Id]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id], [oa].[OrdenFabricacionId], [oa].[AleacionId]
```

```

--- SQL de Split Query ---
-- Consulta 1: órdenes
SELECT [o].[Id], [o].[NumeroOrden], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id]

-- Consulta 2: planchas
SELECT [p].[Id], [p].[OrdenId], [p].[Espesor], ..., [o].[Id]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]

-- Consulta 3: entidades intermedias con aleaciones
SELECT [oa].[OrdenFabricacionId], [oa].[AleacionId], ..., [a].[Id], [a].[Nombre], ..., [o].[Id]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [OrdenesAleaciones] AS [oa] ON [o].[Id] = [oa].[OrdenFabricacionId]
INNER JOIN [Aleaciones] AS [a] ON [oa].[AleacionId] = [a].[Id]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [oa].[OrdenFabricacionId], [oa].[AleacionId]

```

La primera sección muestra el tiempo de SingleQuery. La segunda sección muestra el tiempo de SplitQuery. La tercera sección muestra la comparación. La cuarta sección muestra el SQL de SingleQuery. La quinta sección muestra el SQL de SplitQuery.

Observaciones: SplitQuery es ligeramente más rápida porque no genera el producto cartesiano. El SQL de SingleQuery incluye tres LEFT JOIN con todas las columnas. El SQL de SplitQuery incluye tres consultas separadas. El número de filas transferidas es menor en SplitQuery.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerConVariasColeccionesSingleQuery para eliminar AsNoTrackingWithIdentityResolution y usar AsNoTracking:

```

public List<OrdenFabricacion> ObtenerConVariasColeccionesSingleQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .AsSingleQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

```

Resultado esperado: el código funciona, pero las entidades relacionadas pueden aparecer como instancias duplicadas porque AsNoTracking no usa la caché de identidad. Las planchas y las aleaciones que aparecen en varias órdenes se instancian varias veces.

Solución: usar AsNoTrackingWithIdentityResolution para garantizar instancias únicas.

```

public List<OrdenFabricacion> ObtenerConVariasColeccionesSingleQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .AsSingleQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

```

Resultado esperado con la solución: las entidades relacionadas se resuelven a instancias únicas.

Errores comunes del ejercicio

Error	Causa	Solución
Producto cartesiano	Se incluyeron varias colecciones sin AsSplitQuery	Usar AsSplitQuery
Instancias duplicadas	Se usó AsNoTracking con relaciones	Usar AsNoTrackingWithIdentityResolution
AsSplitQuery con una colección	No aporta beneficio	Usar AsSingleQuery
Coherencia de datos	Cada consulta se ejecuta en una transacción separada	Usar transacción explícita
Comportamiento por defecto	Se configuró SplitQuery globalmente	Revisar el comportamiento por defecto
Advertencia de producto cartesiano	Se ignoró la advertencia	Configurar ConfigureWarnings

Reto resuelto: Configurar el comportamiento por defecto de Split Queries

Reto: Configurar el DbContext para que el comportamiento por defecto sea SplitQuery cuando se incluyen varias colecciones. Configurar la advertencia de producto cartesiano como error. Verificar el comportamiento con una consulta con dos colecciones.

Solución paso a paso:

Paso 1: Modificar el método OnConfiguring del AceriaDbContext:

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        optionsBuilder
            .UseSqlServer(
                "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;",
                sqlOptions =>
                {
                    sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
                    sqlOptions.CommandTimeout(60);
                    sqlOptions.UseQuerySplittingBehavior(QuerySplittingBehavior.SplitQuery);
                })
            .ConfigureWarnings(warnings =>
                warnings.Throw(RelationalEventId.MultipleCollectionIncludeWarning));
    }
}
```

Línea 1: protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) → método de configuración.

Línea 3: if (!optionsBuilder.IsConfigured) → comprueba si las opciones ya están configuradas.

Línea 5: optionsBuilder → objeto de configuración.

Línea 6: .UseSqlServer(→ registra el proveedor de SQL Server.

Línea 7: "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;", → cadena de conexión.

Línea 8: sqlOptions => → delegado de configuración.

Línea 10: sqlOptions.EnableRetryOnFailure(maxRetryCount: 5); → habilita los reintentos.

Línea 11: `sqlOptions.CommandTimeout(60);` → establece el tiempo de espera.

Línea 12: `sqlOptions.UseQuerySplittingBehavior(QuerySplittingBehavior.SplitQuery);` → establece `SplitQuery` como comportamiento por defecto.

Línea 13: `}` → cierra el delegado.

Línea 14: `.ConfigureWarnings(warnings =>)` → configura las advertencias.

Línea 15: `warnings.Throw(RelationalEventId.MultipleCollectionIncludeWarning);` → convierte la advertencia en error.

Línea 16: `}` → cierra el bloque.

Paso 2: Ejecutar una consulta con dos colecciones sin `AsSplitQuery`:

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .ToList();
```

Resultado esperado: EF Core lanza una excepción `InvalidOperationException` indicando que la consulta incluye varias colecciones sin `AsSplitQuery`.

Solución: añadir `AsSplitQuery` explícitamente.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();
```

Resultado esperado con la solución: la consulta se ejecuta sin error porque `AsSplitQuery` está aplicado explícitamente.

Paso 3: Ejecutar `dotnet run` y verificar que la consulta con `AsSplitQuery` funciona y que la consulta sin `AsSplitQuery` lanza la excepción.

Resultado esperado: el comportamiento por defecto es `SplitQuery` y la advertencia de producto cartesiano se convierte en error.

Analogía final

Las Split Queries en una acería son como la forma de pedir las carpetas al archivo central. Si se piden todas las carpetas de planchas y todas las carpetas de aleaciones en un solo viaje, el archivo tiene que combinar cada plancha con cada aleación, generando un montón enorme de documentos. Si una orden tiene tres planchas y dos aleaciones, el archivo trae seis documentos combinados. Si tiene cien planchas y cien aleaciones, trae diez mil documentos combinados. La Split Query es como pedir en tres viajes separados: uno para las órdenes, otro para las planchas y otro para las aleaciones. El archivo trae cien planchas y cien aleaciones, no diez mil combinaciones. Cada viaje es más pequeño y más rápido. La desventaja es que los datos pueden cambiar entre un viaje y el siguiente. Para evitarlo, se hace todo dentro de una misma transacción: el archivo bloquea las carpetas hasta que se termina de pedir todo. La configuración global es como la política del archivo: por defecto, pedir en varios viajes cuando hay varias colecciones. La advertencia de producto cartesiano es como un aviso del archivo: si se pide todo en un solo viaje y hay varias colecciones, el archivo avisa. Así funcionan las Split Queries en EF Core: se eligen según el escenario y se configuran según las necesidades.

Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de Split Queries a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `SplitQueriesUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con órdenes, planchas y aleaciones.

Ejecutado las demostraciones de `AsSingleQuery` y `AsSplitQuery`.

Comparado el rendimiento de ambas técnicas.

Analizado el SQL generado por cada técnica.

Diagnosticado el error de usar `AsNoTracking` en lugar de `AsNoTrackingWithIdentityResolution`.

Configurado el comportamiento por defecto de Split Queries y la advertencia de producto cartesiano.

Punto 4.9 - Compiled Queries

Práctica

Ejercicio: Añadir Compiled Queries al repositorio del proyecto AceriaData. Compilar las consultas más frecuentes. Comparar el tiempo de ejecución con las consultas no compiladas. Analizar el SQL generado por ambas técnicas.

Contexto del proyecto: En el punto 4.8 se estudiaron las Split Queries, incluyendo el producto cartesiano y la configuración del comportamiento por defecto. En este punto se profundiza en las Compiled Queries, que permiten reutilizar el plan de ejecución de las consultas más frecuentes. Esta técnica se usará en el punto 4.10 para la paginación eficiente.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear la clase de consultas compiladas

Crear el archivo src/AceriaData.Infrastructure/Repositories/OrdenConsultasCompiladas.cs:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public static class OrdenConsultasCompiladas
{
    public static readonly Func<AceriaDbContext, string, List<OrdenFabricacion>> ObtenerPorEstado =
        EF.CompileQuery((AceriaDbContext context, string estado) =>
            context.OrdenesFabricacion
                .AsNoTracking()
                .Where(o => o.Estado == estado)
                .OrderBy(o => o.FechaCreacion)
                .ToList());

    public static readonly Func<AceriaDbContext, string, List<OrdenFabricacion>> ObtenerPorCliente =
        EF.CompileQuery((AceriaDbContext context, string cliente) =>
            context.OrdenesFabricacion
                .AsNoTracking()
                .Where(o => o.Cliente == cliente)
                .OrderBy(o => o.FechaCreacion)
                .ToList());

    public static readonly Func<AceriaDbContext, DateTime, DateTime, List<OrdenFabricacion>>
    ObtenerPorRangoDeFechas =
        EF.CompileQuery((AceriaDbContext context, DateTime desde, DateTime hasta) =>
            context.OrdenesFabricacion
                .AsNoTracking()
                .Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
                .OrderBy(o => o.FechaCreacion)
                .ToList());

    public static readonly Func<AceriaDbContext, int, OrdenFabricacion?> ObtenerPorId =
        EF.CompileQuery((AceriaDbContext context, int id) =>
            context.OrdenesFabricacion
                .AsNoTracking()
                .FirstOrDefault(o => o.Id == id));

    public static readonly Func<AceriaDbContext, int> ContarOrdenes =
```

```

    EF.CompileQuery((AceriaDbContext context) =>
        context.OrdenesFabricacion.Count());
}

```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades del dominio.

Línea 2: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 3: using Microsoft.EntityFrameworkCore; → importa EF Core.

Línea 5: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 7: public static class OrdenConsultasCompiladas → declara la clase estática.

Línea 9: public static readonly Func<AceriaDbContext, string, List> ObtenerPorEstado = → declara el campo estático.

Línea 10: EF.CompileQuery((AceriaDbContext context, string estado) => → compila la consulta.

Línea 11: context.OrdenesFabricacion → inicia la consulta.

Línea 12: .AsNoTracking() → aplica AsNoTracking.

Línea 13: .Where(o => o.Estado == estado) → filtra por estado.

Línea 14: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 15: .ToList()); → materializa la consulta.

Línea 17: public static readonly Func<AceriaDbContext, string, List> ObtenerPorCliente = → declara el campo estático.

Línea 18: EF.CompileQuery((AceriaDbContext context, string cliente) => → compila la consulta.

Línea 19: context.OrdenesFabricacion → inicia la consulta.

Línea 20: .AsNoTracking() → aplica AsNoTracking.

Línea 21: .Where(o => o.Cliente == cliente) → filtra por cliente.

Línea 22: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 23: .ToList()); → materializa la consulta.

Línea 25: public static readonly Func<AceriaDbContext, DateTime, DateTime, List> ObtenerPorRangoDeFechas = → declara el campo estático.

Línea 26: EF.CompileQuery((AceriaDbContext context, DateTime desde, DateTime hasta) => → compila la consulta.

Línea 27: context.OrdenesFabricacion → inicia la consulta.

Línea 28: .AsNoTracking() → aplica AsNoTracking.

Línea 29: .Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta) → filtra por rango de fechas.

Línea 30: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 31: .ToList()); → materializa la consulta.

Línea 33: public static readonly Func<AceriaDbContext, int, OrdenFabricacion?> ObtenerPorId = → declara el campo estático.

Línea 34: EF.CompileQuery((AceriaDbContext context, int id) => → compila la consulta.

Línea 35: context.OrdenesFabricacion → inicia la consulta.

Línea 36: .AsNoTracking() → aplica AsNoTracking.

Línea 37: `.FirstOrDefault(o => o.Id == id));` → filtra por Id y devuelve la primera coincidencia.

Línea 39: `public static readonly Func<AceriaDbContext, int> ContarOrdenes =` → declara el campo estático.

Línea 40: `EF.CompileQuery((AceriaDbContext context) =>` → compila la consulta.

Línea 41: `context.OrdenesFabricacion.Count();` → cuenta las órdenes.

Error común: si se compila una consulta que usa un método personalizado, EF Core lanza una excepción al compilarla. Se debe usar expresiones traducibles.

Paso 3: Añadir los métodos de Compiled Queries a la interfaz del repositorio

Modificar `src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerPorEstadoCompilada(string estado);
    List<OrdenFabricacion> ObtenerPorEstadoNoCompilada(string estado);
    List<OrdenFabricacion> ObtenerPorClienteCompilada(string cliente);
    OrdenFabricacion? ObtenerPorIdCompilada(int id);
    int ContarOrdenesCompilada();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: `using AceriaData.Application.Dtos;` → importa los DTOs.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades.

Línea 4: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres.

Línea 6: `public interface IOrdenRepositorio` → declara la interfaz.

Línea 8: `// ... métodos existentes ...` → comentario que indica que se mantienen los métodos anteriores.

Línea 10: `List ObtenerPorEstadoCompilada(string estado);` → método que usa la consulta compilada.

Línea 11: `List ObtenerPorEstadoNoCompilada(string estado);` → método que usa la consulta no compilada.

Línea 12: `List ObtenerPorClienteCompilada(string cliente);` → método que usa la consulta compilada por cliente.

Línea 13: `OrdenFabricacion? ObtenerPorIdCompilada(int id);` → método que usa la consulta compilada por Id.

Línea 14: `int ContarOrdenesCompilada();` → método que usa la consulta compilada de conteo.

Línea 16: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 17: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

Paso 4: Implementar los métodos de Compiled Queries en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepository.cs:

```
public List<OrdenFabricacion> ObtenerPorEstadoCompilada(string estado)
{
    return OrdenConsultasCompiladas.ObtenerPorEstado(_context, estado);
}

public List<OrdenFabricacion> ObtenerPorEstadoNoCompilada(string estado)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.FechaCreacion)
        .ToList();
}

public List<OrdenFabricacion> ObtenerPorClienteCompilada(string cliente)
{
    return OrdenConsultasCompiladas.ObtenerPorCliente(_context, cliente);
}

public OrdenFabricacion? ObtenerPorIdCompilada(int id)
{
    return OrdenConsultasCompiladas.ObtenerPorId(_context, id);
}

public int ContarOrdenesCompilada()
{
    return OrdenConsultasCompiladas.ContarOrdenes(_context);
}
```

Línea 1: public List ObtenerPorEstadoCompilada(string estado) → declara el método.

Línea 3: return OrdenConsultasCompiladas.ObtenerPorEstado(_context, estado); → invoca la consulta compilada.

Línea 6: public List ObtenerPorEstadoNoCompilada(string estado) → declara el método.

Línea 8: return _context.OrdenesFabricacion → inicia la consulta.

Línea 9: .AsNoTracking() → aplica AsNoTracking.

Línea 10: .Where(o => o.Estado == estado) → filtra por estado.

Línea 11: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 12: .ToList(); → materializa la consulta.

Línea 15: public List ObtenerPorClienteCompilada(string cliente) → declara el método.

Línea 17: return OrdenConsultasCompiladas.ObtenerPorCliente(_context, cliente); → invoca la consulta compilada.

Línea 20: public OrdenFabricacion? ObtenerPorIdCompilada(int id) → declara el método.

Línea 22: return OrdenConsultasCompiladas.ObtenerPorId(_context, id); → invoca la consulta compilada.

Línea 25: public int ContarOrdenesCompilada() → declara el método.

Línea 27: return OrdenConsultasCompiladas.ContarOrdenes(_context); → invoca la consulta compilada.

Error común: si la consulta compilada se usa con un DbContext distinto al que se usó para compilarla, EF Core lanza una excepción. Se debe usar el mismo tipo de DbContext.

Paso 5: Crear el caso de uso de Compiled Queries

Crear el archivo src/AceriaData.Application/UseCases/CompiledQueriesUseCase.cs:

```

using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class CompiledQueriesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public CompiledQueriesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== COMPILED QUERIES ===");

        DemostrarConsultaCompilada();
        DemostrarConsultaNoCompilada();
        CompararRendimiento();
    }

    private void DemostrarConsultaCompilada()
    {
        Console.WriteLine("\n--- Consulta compilada ---");

        var cronometro = Stopwatch.StartNew();
        for (int i = 0; i < 100; i++)
        {
            var ordenes = _unidad.Ordenes.ObtenerPorEstadoCompilada("Pendiente");
        }
        cronometro.Stop();

        Console.WriteLine($"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms");
    }

    private void DemostrarConsultaNoCompilada()
    {
        Console.WriteLine("\n--- Consulta no compilada ---");

        var cronometro = Stopwatch.StartNew();
        for (int i = 0; i < 100; i++)
        {
            var ordenes = _unidad.Ordenes.ObtenerPorEstadoNoCompilada("Pendiente");
        }
        cronometro.Stop();

        Console.WriteLine($"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms");
    }

    private void CompararRendimiento()
    {
        Console.WriteLine("\n--- Comparación de rendimiento ---");

        var cronometroCompilada = Stopwatch.StartNew();
        for (int i = 0; i < 100; i++)
        {
            _unidad.Ordenes.ObtenerPorEstadoCompilada("Pendiente");
        }
        cronometroCompilada.Stop();

        var cronometroNoCompilada = Stopwatch.StartNew();
        for (int i = 0; i < 100; i++)
        {
            _unidad.Ordenes.ObtenerPorEstadoNoCompilada("Pendiente");
        }
        cronometroNoCompilada.Stop();

        Console.WriteLine($"Compilada: {cronometroCompilada.ElapsedMilliseconds} ms");
    }
}

```

```

        Console.WriteLine($"No compilada: {cronometroNoCompilada.ElapsedMilliseconds} ms");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class CompiledQueriesUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public CompiledQueriesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== COMPILED QUERIES ==="); → muestra la cabecera.

Línea 19: DemostrarConsultaCompilada(); → llama al método de consulta compilada.

Línea 20: DemostrarConsultaNoCompilada(); → llama al método de consulta no compilada.

Línea 21: CompararRendimiento(); → llama al método de comparación.

Línea 24: private void DemostrarConsultaCompilada() → declara el método.

Línea 26: Console.WriteLine("\n--- Consulta compilada ---"); → muestra la cabecera. \Línea 28: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 29: for (int i = 0; i < 100; i++) → repite cien veces.

Línea 31: var ordenes = _unidad Ordenes.ObtenerPorEstadoCompilada("Pendiente"); → ejecuta la consulta compilada.

Línea 33: cronometro.Stop(); → detiene el cronómetro.

Línea 35: Console.WriteLine(\$"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 38: private void DemostrarConsultaNoCompilada() → declara el método.

Línea 40: Console.WriteLine("\n--- Consulta no compilada ---"); → muestra la cabecera.

Línea 42: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 43: for (int i = 0; i < 100; i++) → repite cien veces.

Línea 45: var ordenes = _unidad Ordenes.ObtenerPorEstadoNoCompilada("Pendiente"); → ejecuta la consulta no compilada.

Línea 47: cronometro.Stop(); → detiene el cronómetro.

Línea 49: Console.WriteLine(\$"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 52: private void CompararRendimiento() → declara el método.

Línea 54: Console.WriteLine("\n--- Comparación de rendimiento ---"); → muestra la cabecera.

Línea 56: var cronometroCompilada = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 57: for (int i = 0; i < 100; i++) → repite cien veces.

Línea 59: _unidad Ordenes.ObtenerPorEstadoCompilada("Pendiente"); → ejecuta la consulta compilada.

Línea 61: cronometroCompilada.Stop(); → detiene el cronómetro.

Línea 63: `var cronometroNoCompilada = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 64: `for (int i = 0; i < 100; i++)` → repite cien veces.

Línea 66: `_unidad Ordenes.ObtenerPorEstadoNoCompilada("Pendiente");` → ejecuta la consulta no compilada.

Línea 68: `cronometroNoCompilada.Stop();` → detiene el cronómetro.

Línea 70: `Console.WriteLine($"Compilada: {cronometroCompilada.ElapsedMilliseconds} ms");` → muestra el tiempo de la consulta compilada.

Línea 71: `Console.WriteLine($"No compilada: {cronometroNoCompilada.ElapsedMilliseconds} ms");` → muestra el tiempo de la consulta no compilada.

Error común: si el bucle ejecuta la consulta cien veces con el mismo parámetro, EF Core puede cachear la consulta y los resultados. Se debe usar parámetros distintos para medir el coste de compilación.

| Paso 6: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<CompiledQueriesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 7: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<CompiledQueriesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 8: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) }
    };

    context OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... }, → quinta orden.

Línea 14: new OrdenFabricacion { ... } → sexta orden.

Línea 15: }; → cierra la lista.

Línea 17: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 18: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

| Paso 9: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de las cien ejecuciones de la consulta compilada y de la consulta no compilada.

| Paso 10: Analizar la salida

La salida del programa muestra información como la siguiente:

```

=== COMPILED QUERIES ===

--- Consulta compilada ---
100 ejecuciones: 85 ms

--- Consulta no compilada ---
100 ejecuciones: 120 ms

--- Comparación de rendimiento ---
Compilada: 85 ms
No compilada: 120 ms

```

La primera sección muestra el tiempo de las cien ejecuciones de la consulta compilada. La segunda sección muestra el tiempo de las cien ejecuciones de la consulta no compilada. La tercera sección muestra la comparación.

Observaciones: la consulta compilada es más rápida porque el plan de ejecución se reutiliza. La consulta no compilada paga el coste de compilación en cada ejecución, aunque EF Core tiene una caché de consultas que reduce el coste. La diferencia es mayor cuando la consulta es compleja o cuando la caché de consultas se llena.

Paso 11: Diagnosticar un error común

Modificar el método `ObtenerPorEstadoCompilada` para compilar la consulta cada vez que se llama:

```

public List<OrdenFabricacion> ObtenerPorEstadoCompilada(string estado)
{
    var consultaCompilada = EF.CompileQuery((AceriaDbContext context, string e) =>
        context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Estado == e)
            .OrderBy(o => o.FechaCreacion)
            .ToList());

    return consultaCompilada(_context, estado);
}

```

Resultado esperado: el código funciona, pero la consulta se compila cada vez que se llama al método. El beneficio de la consulta compilada se pierde porque el coste de compilación se paga en cada llamada.

Solución: mover la compilación al campo estático de la clase `OrdenConsultasCompiladas`.

```

return OrdenConsultasCompiladas.ObtenerPorEstado(_context, estado);

```

Resultado esperado con la solución: la consulta se compila una sola vez y se reutiliza en todas las llamadas.

Errores comunes del ejercicio

Error	Causa	Solución
Compilación en cada llamada	Se compila la consulta dentro del método	Mover la compilación al campo estático
DbContext distinto	Se usa un DbContext distinto al de la compilación	Usar el mismo tipo de DbContext
Método personalizado	Se usa un método no traducible en la consulta	Usar expresiones traducibles
Parámetros incorrectos	Los parámetros no coinciden con los de la compilación	Verificar la firma del delegado
Resultado cacheado	Se usa el mismo parámetro en todas las ejecuciones	Usar parámetros distintos

Error	Causa	Solución
Compiled Query innecesaria	La consulta se ejecuta pocas veces	Usar consultas normales

Retos resueltos: Comparar Compiled Queries con y sin proyección

Reto: Crear dos consultas compiladas, una que devuelva entidades completas y otra que devuelva una proyección. Comparar el tiempo de ejecución de cien iteraciones.

Solución paso a paso:

Paso 1: Añadir las consultas compiladas a OrdenConsultasCompiladas:

```
public static readonly Func<AceriaDbContext, string, List<OrdenFabricacion>> ObtenerPorEstadoCompleta =
    EF.CompileQuery((AceriaDbContext context, string estado) =>
        context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Estado == estado)
            .OrderBy(o => o.FechaCreacion)
            .ToList());

public static readonly Func<AceriaDbContext, string, List<OrdenResumenDto>> ObtenerPorEstadoProyectada =
    EF.CompileQuery((AceriaDbContext context, string estado) =>
        context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Estado == estado)
            .OrderBy(o => o.FechaCreacion)
            .Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            })
            .ToList());
```

Línea 1: public static readonly Func<AceriaDbContext, string, List> ObtenerPorEstadoCompleta = → declara el campo estático.

Línea 2: EF.CompileQuery((AceriaDbContext context, string estado) => → compila la consulta.

Línea 3: context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .Where(o => o.Estado == estado) → filtra por estado.

Línea 6: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 7: .ToList(); → materializa la consulta.

Línea 9: public static readonly Func<AceriaDbContext, string, List> ObtenerPorEstadoProyectada = → declara el campo estático.

Línea 10: EF.CompileQuery((AceriaDbContext context, string estado) => → compila la consulta.

Línea 11: context.OrdenesFabricacion → inicia la consulta.

Línea 12: .AsNoTracking() → aplica AsNoTracking.

Línea 13: .Where(o => o.Estado == estado) → filtra por estado.

Línea 14: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 15: .Select(o => new OrdenResumenDto → proyecta al DTO.

Línea 16: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 17: Cliente = o.Cliente, → asigna el cliente.

Línea 18: Estado = o.Estado, → asigna el estado.

Línea 19: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 20: }) → cierra la proyección.

Línea 21: .ToList(); → materializa la consulta.

Paso 2: Añadir los métodos a la interfaz IOrdenRepositorio:

```
List<OrdenResumenDto> ObtenerPorEstadoProyectadaCompilada(string estado);
```

Línea 1: List ObtenerPorEstadoProyectadaCompilada(string estado); → declara el método.

Paso 3: Implementar el método en OrdenRepositorio:

```
public List<OrdenResumenDto> ObtenerPorEstadoProyectadaCompilada(string estado)
{
    return OrdenConsultasCompiladas.ObtenerPorEstadoProyectada(_context, estado);
}
```

Línea 1: public List ObtenerPorEstadoProyectadaCompilada(string estado) → declara el método.

Línea 3: return OrdenConsultasCompiladas.ObtenerPorEstadoProyectada(_context, estado); → invoca la consulta compilada.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarProyectadaCompilada()
{
    Console.WriteLine("\n--- Consulta compilada con proyección ---");

    var cronometro = Stopwatch.StartNew();
    for (int i = 0; i < 100; i++)
    {
        _unidad.Ordenes.ObtenerPorEstadoProyectadaCompilada("Pendiente");
    }
    cronometro.Stop();

    Console.WriteLine($"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms");
}
```

Línea 1: private void DemostrarProyectadaCompilada() → declara el método.

Línea 3: Console.WriteLine("\n--- Consulta compilada con proyección ---"); → muestra la cabecera.

Línea 5: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 6: for (int i = 0; i < 100; i++) → repite cien veces.

Línea 8: _unidad.Ordenes.ObtenerPorEstadoProyectadaCompilada("Pendiente"); → ejecuta la consulta compilada con proyección.

Línea 10: cronometro.Stop(); → detiene el cronómetro.

Línea 12: Console.WriteLine(\$"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms"); → muestra el tiempo.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarProyectadaCompilada();
```

| Paso 6: Ejecutar dotnet run y comparar los tiempos.

Resultado esperado: la consulta compilada con proyección es más rápida que la consulta compilada con entidades completas porque transfiere menos datos.

| Analogía final

Las Compiled Queries en una acería son como las plantillas de órdenes de búsqueda que el jefe de planta prepara una sola vez y reutiliza muchas veces. En lugar de redactar la orden desde cero cada vez, el jefe usa una plantilla que ya tiene las condiciones predefinidas y solo rellena los parámetros. La primera vez que se usa la plantilla, se tarda un poco en prepararla. Las siguientes veces, se tarda mucho menos porque la plantilla ya está lista. Las consultas no compiladas son como redactar la orden desde cero cada vez: se tarda más porque hay que pensar en todas las condiciones. La caché de consultas de EF Core es como un archivador donde el jefe guarda las plantillas que ya ha usado. Si la plantilla está en el archivador, la reutiliza. Si no está, la redacta y la guarda. Las Compiled Queries son como las plantillas que el jefe ha preparado con antelación y tiene siempre a mano. En una acería con mucho volumen, las plantillas ahorran mucho tiempo. En una acería pequeña, no tanto. Así funcionan las Compiled Queries en EF Core: se compilan una vez y se reutilizan muchas veces.

| Resultado esperado

Al final del ejercicio, deberías haber:

Creado la clase OrdenConsultasCompiladas con cinco consultas compiladas.

Añadido los métodos de Compiled Queries a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso CompiledQueriesUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes.

Ejecutado las demostraciones de consulta compilada y no compilada.

Comparado el rendimiento de ambas técnicas.

Diagnosticado el error de compilar la consulta en cada llamada.

Creado la consulta compilada con proyección y comparado su rendimiento.

Punto 4.10 - Paginación eficiente: Skip/Take y keyset pagination

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que usen Skip/Take y keyset pagination. Comparar el SQL generado y el tiempo de ejecución. Aplicar paginación con filtros, proyecciones y AsNoTracking. Analizar el impacto de Skip en tablas grandes.

Contexto del proyecto: En el punto 4.9 se estudiaron las Compiled Queries, incluyendo su compilación y reutilización. En este punto se profundiza en la paginación eficiente, que es esencial en aplicaciones web y en cualquier escenario con muchos resultados. Esta técnica se usará en el punto 4.11 para el diagnóstico.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de paginación a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenResumenDto> ObtenerPaginadoOffset(int pagina, int tamanoPagina);
    List<OrdenResumenDto> ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamanoPagina);
    int ContarOrdenesPaginadas();
    string ObtenerSqlPaginadoOffset(int pagina, int tamanoPagina);
    string ObtenerSqlPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamanoPagina);

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerPaginadoOffset(int pagina, int tamanoPagina); → método que usa offset pagination.

Línea 11: List ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamanoPagina); → método que usa keyset pagination.

Línea 12: int ContarOrdenesPaginadas(); → método que cuenta las órdenes para calcular el total de páginas.

Línea 13: `string ObtenerSqlPaginadoOffset(int pagina, int tamanoPagina);` → método que devuelve el SQL de offset pagination.

Línea 14: `string ObtenerSqlPaginadoKeyset(DateTime ultimaFecha, int ultimold, int tamanoPagina);` → método que devuelve el SQL de keyset pagination.

Línea 16: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 17: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 3: Implementar los métodos de paginación en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepository.cs`:

```
public List<OrdenResumenDto> ObtenerPaginadoOffset(int pagina, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

public List<OrdenResumenDto> ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id >
ultimoId))
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

public int ContarOrdenesPaginadas()
{
    return _context.OrdenesFabricacion.Count();
}

public string ObtenerSqlPaginadoOffset(int pagina, int tamanoPagina)
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
```

```

        Estado = o.Estado,
        FechaCreacion = o.FechaCreacion
    });

    return consulta.ToQueryString();
}

public string ObtenerSqlPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamanoPagina)
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id >
ultimoId))
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        });

    return consulta.ToQueryString();
}

```

Línea 1: public List ObtenerPaginadoOffset(int pagina, int tamanoPagina) → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 6: .ThenBy(o => o.Id) → ordena por Id.

Línea 7: .Skip((pagina - 1) * tamanoPagina) → salta las páginas anteriores.

Línea 8: .Take(tamanoPagina) → toma el tamaño de la página.

Línea 9: .Select(o => new OrdenResumenDto → proyecta al DTO.

Línea 10: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 11: Cliente = o.Cliente, → asigna el cliente.

Línea 12: Estado = o.Estado, → asigna el estado.

Línea 13: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 14: }) → cierra la proyección.

Línea 15: .ToList(); → materializa la consulta.

Línea 18: public List ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimold, int tamanoPagina) → declara el método.

Línea 20: return _context.OrdenesFabricacion → inicia la consulta.

Línea 21: .AsNoTracking() → aplica AsNoTracking.

Línea 22: .Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id > ultimold)) → filtra por la clave compuesta.

Línea 23: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 24: .ThenBy(o => o.Id) → ordena por Id.

Línea 25: .Take(tamanoPagina) → toma el tamaño de la página.

Línea 26: `.Select(o => new OrdenResumenDto` → proyecta al DTO.

Línea 27: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 28: `Cliente = o.Cliente,` → asigna el cliente.

Línea 29: `Estado = o.Estado,` → asigna el estado.

Línea 30: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 31: `})` → cierra la proyección.

Línea 32: `.ToList();` → materializa la consulta.

Línea 35: `public int ContarOrdenesPaginadas()` → declara el método.

Línea 37: `return _context.OrdenesFabricacion.Count();` → cuenta las órdenes.

Línea 40: `public string ObtenerSqlPaginadoOffset(int pagina, int tamanoPagina)` → declara el método.

Línea 42: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 43: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 44: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 45: `.ThenBy(o => o.Id)` → ordena por Id.

Línea 46: `.Skip((pagina - 1) * tamanoPagina)` → salta las páginas anteriores.

Línea 47: `.Take(tamanoPagina)` → toma el tamaño de la página.

Línea 48: `.Select(o => new OrdenResumenDto` → proyecta al DTO.

Línea 49: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 50: `Cliente = o.Cliente,` → asigna el cliente.

Línea 51: `Estado = o.Estado,` → asigna el estado.

Línea 52: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 53: `});` → cierra la proyección.

Línea 55: `return consulta.ToQueryString();` → devuelve el SQL.

Línea 58: `public string ObtenerSqlPaginadoKeyset(DateTime ultimaFecha, int ultimold, int tamanoPagina)` → declara el método.

Línea 60: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 61: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 62: `.Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id > ultimold))` → filtra por la clave compuesta.

Línea 63: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 64: `.ThenBy(o => o.Id)` → ordena por Id.

Línea 65: `.Take(tamanoPagina)` → toma el tamaño de la página.

Línea 66: `.Select(o => new OrdenResumenDto` → proyecta al DTO.

Línea 67: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 68: `Cliente = o.Cliente,` → asigna el cliente.

Línea 69: `Estado = o.Estado,` → asigna el estado.

Línea 70: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 71: }); → cierra la proyección.

Línea 73: return consulta.ToString(); → devuelve el SQL.

Error común: si la clave de ordenación no es única, la keyset pagination puede devolver filas duplicadas o saltar filas. Se debe usar una clave compuesta que incluya la clave primaria.

| Paso 4: Crear el caso de uso de paginación eficiente

Crear el archivo src/AceriaData.Application/UseCases/PaginacionUseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class PaginacionUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public PaginacionUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== PAGINACIÓN EFICIENTE ===");

        DemostrarOffsetPagination();
        DemostrarKeysetPagination();
        CompararRendimiento();
        MostrarSql();
    }

    private void DemostrarOffsetPagination()
    {
        Console.WriteLine("\n--- Offset pagination ---");

        var cronometro = Stopwatch.StartNew();
        var pagina1 = _unidad.Ordenes.ObtenerPaginadoOffset(1, 3);
        var pagina2 = _unidad.Ordenes.ObtenerPaginadoOffset(2, 3);
        cronometro.Stop();

        Console.WriteLine($"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes |
Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in pagina1)
        {
            Console.WriteLine($" P1: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/
MM/yyyy}");
        }
        foreach (var orden in pagina2)
        {
            Console.WriteLine($" P2: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/
MM/yyyy}");
        }
    }

    private void DemostrarKeysetPagination()
    {
        Console.WriteLine("\n--- Keyset pagination ---");

        var cronometro = Stopwatch.StartNew();
        var pagina1 = _unidad.Ordenes.ObtenerPaginadoKeyset(DateTime.MinValue, 0, 3);
        if (pagina1.Count > 0)
        {
            var ultima = pagina1.Last();
            var pagina2 = _unidad.Ordenes.ObtenerPaginadoKeyset(ultima.FechaCreacion, 0, 3);
            cronometro.Stop();

            Console.WriteLine($"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes |
Tiempo: {cronometro.ElapsedMilliseconds} ms");
        }
    }
}
```

```

        foreach (var orden in pagina1)
        {
            Console.WriteLine($" P1: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");
        }
        foreach (var orden in pagina2)
        {
            Console.WriteLine($" P2: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");
        }
    }
}

private void CompararRendimiento()
{
    Console.WriteLine("\n--- Comparación de rendimiento ---");

    var cronometroOffset = Stopwatch.StartNew();
    for (int i = 1; i <= 10; i++)
    {
        _unidad.Ordenes.ObtenerPaginadoOffset(i, 3);
    }
    cronometroOffset.Stop();

    var cronometroKeyset = Stopwatch.StartNew();
    var ultimaFecha = DateTime.MinValue;
    var ultimoId = 0;
    for (int i = 0; i < 10; i++)
    {
        var pagina = _unidad.Ordenes.ObtenerPaginadoKeyset(ultimaFecha, ultimoId, 3);
        if (pagina.Count == 0) break;
        var ultima = pagina.Last();
        ultimaFecha = ultima.FechaCreacion;
        ultimoId = 0;
    }
    cronometroKeyset.Stop();

    Console.WriteLine($"Offset pagination (10 páginas): {cronometroOffset.ElapsedMilliseconds} ms");
    Console.WriteLine($"Keyset pagination (10 páginas): {cronometroKeyset.ElapsedMilliseconds} ms");
}

private void MostrarSql()
{
    Console.WriteLine("\n--- SQL de offset pagination ---");
    var sqlOffset = _unidad.Ordenes.ObtenerSqlPaginadoOffset(2, 3);
    Console.WriteLine(sqlOffset);

    Console.WriteLine("\n--- SQL de keyset pagination ---");
    var sqlKeyset = _unidad.Ordenes.ObtenerSqlPaginadoKeyset(new DateTime(2024, 3, 10), 3, 3);
    Console.WriteLine(sqlKeyset);
}
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class PaginacionUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public PaginacionUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== PAGINACIÓN EFICIENTE ==="); → muestra la cabecera.

Línea 19: DemostrarOffsetPagination(); → llama al método de offset pagination.

Línea 20: `DemostrarKeysetPagination();` → llama al método de keyset pagination.

Línea 21: `CompararRendimiento();` → llama al método de comparación.

Línea 22: `MostrarSql();` → llama al método de SQL.

Línea 25: `private void DemostrarOffsetPagination()` → declara el método.

Línea 27: `Console.WriteLine("\n--- Offset pagination ---");` → muestra la cabecera.

Línea 29: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 30: `var pagina1 = _unidad Ordenes.ObtenerPaginadoOffset(1, 3);` → carga la primera página.

Línea 31: `var pagina2 = _unidad Ordenes.ObtenerPaginadoOffset(2, 3);` → carga la segunda página.

Línea 32: `cronometro.Stop();` → detiene el cronómetro.

Línea 34: `Console.WriteLine($"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el resultado.

Línea 35: `foreach (var orden in pagina1)` → itera sobre la primera página.

Línea 37: `Console.WriteLine($" P1: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 39: `foreach (var orden in pagina2)` → itera sobre la segunda página.

Línea 41: `Console.WriteLine($" P2: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 45: `private void DemostrarKeysetPagination()` → declara el método.

Línea 47: `Console.WriteLine("\n--- Keyset pagination ---");` → muestra la cabecera.

Línea 49: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 50: `var pagina1 = _unidad Ordenes.ObtenerPaginadoKeyset(DateTime.MinValue, 0, 3);` → carga la primera página.

Línea 51: `if (pagina1.Count > 0)` → comprueba si hay resultados.

Línea 53: `var ultima = pagina1.Last();` → obtiene la última orden.

Línea 54: `var pagina2 = _unidad Ordenes.ObtenerPaginadoKeyset(ultima.FechaCreacion, 0, 3);` → carga la segunda página.

Línea 55: `cronometro.Stop();` → detiene el cronómetro.

Línea 57: `Console.WriteLine($"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el resultado.

Línea 58: `foreach (var orden in pagina1)` → itera sobre la primera página.

Línea 60: `Console.WriteLine($" P1: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 62: `foreach (var orden in pagina2)` → itera sobre la segunda página.

Línea 64: `Console.WriteLine($" P2: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 69: `private void CompararRendimiento()` → declara el método.

Línea 71: `Console.WriteLine("\n--- Comparación de rendimiento ---");` → muestra la cabecera.

Línea 73: `var cronometroOffset = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 74: `for (int i = 1; i <= 10; i++)` → repite diez veces.

Línea 76: `_unidad Ordenes.ObtenerPaginadoOffset(i, 3);` → carga la página.

Línea 78: `cronometroOffset.Stop();` → detiene el cronómetro.

Línea 80: `var cronometroKeyset = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 81: `var ultimaFecha = DateTime.MinValue;` → inicializa la fecha.

Línea 82: `var ultimold = 0;` → inicializa el Id.

Línea 83: `for (int i = 0; i < 10; i++)` → repite diez veces.

Línea 85: `var pagina = _unidad Ordenes.ObtenerPaginadoKeyset(ultimaFecha, ultimold, 3);` → carga la página.

Línea 86: `if (pagina.Count == 0) break;` → detiene si no hay más páginas.

Línea 87: `var ultima = pagina.Last();` → obtiene la última orden.

Línea 88: `ultimaFecha = ultima.FechaCreacion;` → actualiza la fecha.

Línea 89: `ultimold = 0;` → reinicia el Id.

Línea 91: `cronometroKeyset.Stop();` → detiene el cronómetro.

Línea 93: `Console.WriteLine($"Offset pagination (10 páginas): {cronometroOffset.ElapsedMilliseconds} ms");` → muestra el tiempo de offset.

Línea 94: `Console.WriteLine($"Keyset pagination (10 páginas): {cronometroKeyset.ElapsedMilliseconds} ms");` → muestra el tiempo de keyset.

Línea 97: `private void MostrarSql()` → declara el método.

Línea 99: `Console.WriteLine("\n--- SQL de offset pagination ---");` → muestra la cabecera.

Línea 100: `var sqlOffset = _unidad Ordenes.ObtenerSqlPaginadoOffset(2, 3);` → obtiene el SQL.

Línea 101: `Console.WriteLine(sqlOffset);` → imprime el SQL.

Línea 103: `Console.WriteLine("\n--- SQL de keyset pagination ---");` → muestra la cabecera.

Línea 104: `var sqlKeyset = _unidad Ordenes.ObtenerSqlPaginadoKeyset(new DateTime(2024, 3, 10), 3, 3);` → obtiene el SQL.

Línea 105: `Console.WriteLine(sqlKeyset);` → imprime el SQL.

Error común: si se usa el mismo `DbContext` para todas las páginas, las entidades permanecen en el Change Tracker. Se debe usar `AsNoTracking` para evitar el consumo de memoria.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<PaginacionUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<PaginacionUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>();
    for (int i = 1; i <= 20; i++)
    {
        ordenes.Add(new OrdenFabricacion
        {
            NumeroOrden = $"OF-2024-{i:D4}",
            Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur",
            Estado = i % 3 == 0 ? "EnProceso" : "Pendiente",
            FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)
        });
    }

    context OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base de datos.

Línea 5: `context.Database.Migrate();` → aplica las migraciones.

Línea 7: `var ordenes = new List();` → crea la lista de órdenes.

Línea 8: `for (int i = 1; i <= 20; i++)` → repite veinte veces.

Línea 10: `ordenes.Add(new OrdenFabricacion` → crea una orden.

Línea 12: `NumeroOrden = $"OF-2024-{i:D4}"`, → asigna el número.

Línea 13: `Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur"`, → asigna el cliente.

Línea 14: `Estado = i % 3 == 0 ? "EnProceso" : "Pendiente"`, → asigna el estado.

Línea 15: `FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)` → asigna la fecha.

Línea 16: `});` → cierra la orden.

Línea 17: `}` → cierra el bucle.

Línea 19: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 20: `context.SaveChanges();` → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de offset pagination y keyset pagination, y el SQL generado por cada técnica.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== PAGINACIÓN EFICIENTE ===

--- Offset pagination ---
Página 1: 3 órdenes | Página 2: 3 órdenes | Tiempo: 48 ms
P1: 0F-2024-0001 | Constructora del Sur | 02/01/2024
P1: 0F-2024-0002 | Constructora del Norte | 03/01/2024
P1: 0F-2024-0003 | Constructora del Sur | 04/01/2024
P2: 0F-2024-0004 | Constructora del Norte | 05/01/2024
P2: 0F-2024-0005 | Constructora del Sur | 06/01/2024
P2: 0F-2024-0006 | Constructora del Norte | 07/01/2024

--- Keyset pagination ---
Página 1: 3 órdenes | Página 2: 3 órdenes | Tiempo: 45 ms
P1: 0F-2024-0001 | Constructora del Sur | 02/01/2024
...

--- Comparación de rendimiento ---
Offset pagination (10 páginas): 120 ms
Keyset pagination (10 páginas): 95 ms

--- SQL de offset pagination ---
SELECT [o].[NumeroOrden], [o].[Cliente], [o].[Estado], [o].[FechaCreacion]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[FechaCreacion], [o].[Id]
OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY

--- SQL de keyset pagination ---
SELECT [o].[NumeroOrden], [o].[Cliente], [o].[Estado], [o].[FechaCreacion]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
      AND ([o].[FechaCreacion] > @__ultimaFecha_0 OR ([o].[FechaCreacion] = @__ultimaFecha_0 AND [o].[Id]
> @__ultimoId_1))
ORDER BY [o].[FechaCreacion], [o].[Id]
OFFSET 0 ROWS FETCH NEXT 3 ROWS ONLY
```

La primera sección muestra el tiempo de offset pagination. La segunda sección muestra el tiempo de keyset pagination. La tercera sección muestra la comparación. La cuarta sección muestra el SQL de offset pagination. La quinta sección muestra el SQL de keyset pagination.

Observaciones: keyset pagination es más rápida porque no lee las filas anteriores. El SQL de offset pagination incluye OFFSET 3. El SQL de keyset pagination incluye el filtro por la clave compuesta y OFFSET 0. La diferencia es pequeña con pocas filas, pero crece con el número de filas.

Paso 10: Diagnosticar un error común

Modificar el método `ObtenerPaginadoKeyset` para usar solo la fecha como clave:

```

public List<OrdenResumenDto> ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.FechaCreacion > ultimaFecha)
        .OrderBy(o => o.FechaCreacion)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion \           })
        .ToList();
}

```

Resultado esperado: el código funciona, pero si dos órdenes tienen la misma fecha, la paginación puede saltar filas o devolver filas duplicadas. La clave de ordenación no es única.

Solución: usar una clave compuesta que incluya la fecha y el Id.

```

public List<OrdenResumenDto> ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id >
ultimoId))
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

```

Resultado esperado con la solución: la paginación devuelve las filas correctas aunque haya fechas duplicadas.

Errores comunes del ejercicio

Error	Causa	Solución
Skip costoso	Se salta un número elevado de filas	Usar keyset pagination
Filas duplicadas	La clave de ordenación no es única	Usar clave compuesta
Filas saltadas	La clave de ordenación no es única	Usar clave compuesta
Total de páginas incorrecto	El conteo no usa los mismos filtros	Usar los mismos filtros
Paginación sin orden	Se omite OrderBy	Aplicar OrderBy antes de Skip
Paginación en memoria	Se materializa antes de paginar	Aplicar Skip y Take antes de ToList

Reto resuelto: Paginación con filtro y proyección

Reto: Crear un método en el repositorio que pague las órdenes pendientes ordenadas por fecha, con proyección a DTO y AsNoTracking. Comparar el SQL generado con y sin filtro.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```
List<OrdenResumenDto> ObtenerPendientesPaginado(int pagina, int tamañoPagina);
```

Línea 1: List ObtenerPendientesPaginado(int pagina, int tamañoPagina); → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenResumenDto> ObtenerPendientesPaginado(int pagina, int tamañoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Skip((pagina - 1) * tamañoPagina)
        .Take(tamañoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}
```

Línea 1: public List ObtenerPendientesPaginado(int pagina, int tamañoPagina) → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .Where(o => o.Estado == "Pendiente") → filtra por estado.

Línea 6: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 7: .ThenBy(o => o.Id) → ordena por Id.

Línea 8: .Skip((pagina - 1) * tamañoPagina) → salta las páginas anteriores.

Línea 9: .Take(tamañoPagina) → toma el tamaño de la página.

Línea 10: .Select(o => new OrdenResumenDto → proyecta al DTO.

Línea 11: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 12: Cliente = o.Cliente, → asigna el cliente.

Línea 13: Estado = o.Estado, → asigna el estado.

Línea 14: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 15: }) → cierra la proyección.

Línea 16: .ToList(); → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarPaginadoConFiltro()
{
    Console.WriteLine("\n--- Paginación con filtro y proyección ---");

    var pagina1 = _unidad.Ordenes.ObtenerPendientesPaginado(1, 3);
    var pagina2 = _unidad.Ordenes.ObtenerPendientesPaginado(2, 3);

    Console.WriteLine($"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes");
    foreach (var orden in pagina1)
    {
        Console.WriteLine($" P1: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");
    }
    foreach (var orden in pagina2)
    {
        Console.WriteLine($" P2: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");
    }
}
```

Línea 1: private void DemostrarPaginadoConFiltro() → declara el método.

Línea 3: Console.WriteLine("\n--- Paginación con filtro y proyección ---"); → muestra la cabecera.

Línea 5: var pagina1 = _unidad.Ordenes.ObtenerPendientesPaginado(1, 3); → carga la primera página.

Línea 6: var pagina2 = _unidad.Ordenes.ObtenerPendientesPaginado(2, 3); → carga la segunda página.

Línea 8: Console.WriteLine(\$"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes"); → muestra el resultado.

Línea 9: foreach (var orden in pagina1) → itera sobre la primera página.

Línea 11: Console.WriteLine(\$" P1: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}"); → muestra los datos.

Línea 13: foreach (var orden in pagina2) → itera sobre la segunda página.

Línea 15: Console.WriteLine(\$" P2: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}"); → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarPaginadoConFiltro();
```

Paso 5: Ejecutar dotnet run y verificar que las páginas se muestran correctamente.

Resultado esperado: las órdenes pendientes se muestran paginadas por fecha, con proyección a DTO y sin tracking.

Analogía final

La paginación en una acería es como pedir las carpetas al archivo central en lotes. En lugar de pedir todas las carpetas de una vez, se piden de diez en diez. La offset pagination es como pedir el lote número diez: el archivo tiene que contar las carpetas anteriores y saltarlas. Si se pide el lote número mil, el archivo tiene que contar mil lotes antes de llegar al solicitado. La keyset pagination es como pedir el lote que empieza después de la carpeta con número OF-2024-0100: el archivo va directamente a esa carpeta y trae las siguientes. No tiene que contar las anteriores. La keyset pagination es más eficiente porque no recorre las carpetas anteriores. La paginación con filtro es como pedir solo las carpetas de un estado concreto. La paginación con proyección es como pedir solo los datos necesarios de cada carpeta.

Así funciona la paginación en EF Core: se elige la técnica según el escenario y se combina con filtros y proyecciones.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de paginación a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `PaginacionUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con veinte órdenes.

Ejecutado las demostraciones de `offset pagination` y `keyset pagination`.

Comparado el rendimiento de ambas técnicas.

Analizado el SQL generado por cada técnica.

Diagnosticado el error de usar solo la fecha como clave.

Creado el método `ObtenerPendientesPaginado` con filtro y proyección.

Punto 4.11 - Diagnóstico con logs, métricas y herramientas

Práctica

Ejercicio: Configurar el diagnóstico en el proyecto AceriaData. Añadir métodos al repositorio que midan el tiempo de ejecución de las consultas, cuenten el número de consultas y detecten las consultas lentas. Configurar un observador de diagnóstico. Analizar el plan de ejecución en SQL Server.

Contexto del proyecto: En el punto 4.10 se estudió la paginación eficiente, incluyendo Skip/Take y keyset pagination. En este punto se profundiza en el diagnóstico, que permite identificar problemas de rendimiento en EF Core. Esta técnica se usará en el punto 4.12 para las estrategias de optimización.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el observador de diagnóstico

Crear el archivo src/AceriaData.Infrastructure/Diagnostics/EfCoreDiagnosticObserver.cs:

```
using System.Diagnostics;

namespace AceriaData.Infrastructure.Diagnostics;

public class EfCoreDiagnosticObserver : IObservable<DiagnosticListener>
{
    private readonly Action<string> _log;

    public EfCoreDiagnosticObserver(Action<string> log)
    {
        _log = log;
    }

    public void OnNext(DiagnosticListener listener)
    {
        if (listener.Name == "Microsoft.EntityFrameworkCore")
        {
            listener.Subscribe(new EfCoreCommandObserver(_log));
        }
    }

    public void OnError(Exception error) { }
    public void OnCompleted() { }
}

public class EfCoreCommandObserver : IObservable<KeyValuePair<string, object>>
{
    private readonly Action<string> _log;

    public EfCoreCommandObserver(Action<string> log)
    {
        _log = log;
    }

    public void OnNext(KeyValuePair<string, object> value)
    {
        if (value.Key == "Microsoft.EntityFrameworkCore.Database.Command.CommandExecuted")
        {
            _log($"Comando ejecutado: {value.Value}");
        }
    }
}
```

```

    }

    public void OnError(Exception error) { }
    public void OnCompleted() { }
}

```

Línea 1: using System.Diagnostics; → importa el espacio de nombres de diagnóstico.

Línea 3: namespace AceriaData.Infrastructure.Diagnostics; → declara el espacio de nombres.

Línea 5: public class EfCoreDiagnosticObserver : IObservable → declara el observador de listeners.

Línea 7: private readonly Action _log; → campo para el delegado de logging.

Línea 9: public EfCoreDiagnosticObserver(Action log) → constructor.

Línea 11: _log = log; → asigna el delegado al campo.

Línea 14: public void OnNext(DiagnosticListener listener) → implementa OnNext.

Línea 16: if (listener.Name == "Microsoft.EntityFrameworkCore") → comprueba el nombre del listener.

Línea 18: listener.Subscribe(new EfCoreCommandObserver(_log)); → suscribe el observador interno.

Línea 22: public void OnError(Exception error) { } → implementa OnError.

Línea 23: public void OnCompleted() { } → implementa OnCompleted.

Línea 26: public class EfCoreCommandObserver : IObservable<KeyValuePair<string, object>> → declara el observador de eventos.

Línea 28: private readonly Action _log; → campo para el delegado de logging.

Línea 30: public EfCoreCommandObserver(Action log) → constructor.

Línea 32: _log = log; → asigna el delegado al campo.

Línea 35: public void OnNext(KeyValuePair<string, object> value) → implementa OnNext.

Línea 37: if (value.Key == "Microsoft.EntityFrameworkCore.Database.Command.CommandExecuted") → comprueba el nombre del evento.

Línea 39: _log(\$"Comando ejecutado: {value.Value}"); → registra el comando ejecutado.

Línea 43: public void OnError(Exception error) { } → implementa OnError.

Línea 44: public void OnCompleted() { } → implementa OnCompleted.

Error común: si el observador no comprueba el nombre del listener, se suscribe a todos los listeners del sistema, lo que puede degradar el rendimiento. Se debe comprobar el nombre.

| Paso 3: Añadir los métodos de diagnóstico a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```

using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerTodasConDiagnostico();
    List<OrdenFabricacion> ObtenerPendientesConDiagnostico();
    int ContarOrdenesConDiagnostico();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerTodasConDiagnostico(); → método que carga todas las órdenes.

Línea 11: List ObtenerPendientesConDiagnostico(); → método que carga las órdenes pendientes.

Línea 12: int ContarOrdenesConDiagnostico(); → método que cuenta las órdenes.

Línea 14: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 15: void Eliminar(OrdenFabricacion orden); → método que elimina una orden.

Error común: si los métodos devuelven IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 4: Implementar los métodos de diagnóstico en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs:

```

public List<OrdenFabricacion> ObtenerTodasConDiagnostico()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerPendientesConDiagnostico()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ToList();
}

public int ContarOrdenesConDiagnostico()
{
    return _context.OrdenesFabricacion.Count();
}

```

Línea 1: public List ObtenerTodasConDiagnostico() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 6: .ToList(); → materializa la consulta.

Línea 9: public List ObtenerPendientesConDiagnostico() → declara el método.

Línea 11: return _context.OrdenesFabricacion → inicia la consulta.

Línea 12: .AsNoTracking() → aplica AsNoTracking.

Línea 13: .Where(o => o.Estado == "Pendiente") → filtra por estado.

Línea 14: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 15: .ToList(); → materializa la consulta.

Línea 18: public int ContarOrdenesConDiagnostico() → declara el método.

Línea 20: return _context.OrdenesFabricacion.Count(); → cuenta las órdenes.

Error común: si no se aplica AsNoTracking, las entidades se registran en el Change Tracker y consumen memoria. Se debe aplicar en consultas de solo lectura.

| Paso 5: Crear el caso de uso de diagnóstico

Crear el archivo src/AceriaData.Application/UseCases/DiagnosticoUseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class DiagnosticoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public DiagnosticoUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== DIAGNÓSTICO ===");

        DemostrarMedicionDeTiempo();
        DemostrarConteoDeConsultas();
        DemostrarDeteccionDeConsultasLentas();
    }

    private void DemostrarMedicionDeTiempo()
    {
        Console.WriteLine("\n--- Medición de tiempo ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    }

    private void DemostrarConteoDeConsultas()
    {
        Console.WriteLine("\n--- Conteo de consultas ---");

        var contador = 0;
        var cronometro = Stopwatch.StartNew();

        contador++;
    }
}
```

```

        var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();

        contador++;
        var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();

        contador++;
        var total = _unidad.Ordenes.ContarOrdenesConDiagnostico();

        cronometro.Stop();

        Console.WriteLine($"Consultas ejecutadas: {contador} | Tiempo: {cronometro.ElapsedMilliseconds}
ms");
        Console.WriteLine($"Órdenes: {ordenes.Count} | Pendientes: {pendientes.Count} | Total:
{total}");
    }

    private void DemostrarDeteccionDeConsultasLentas()
    {
        Console.WriteLine("\n--- Detección de consultas lentas ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();
        cronometro.Stop();

        var tiempoMs = cronometro.ElapsedMilliseconds;
        var umbralMs = 50;

        if (tiempoMs > umbralMs)
        {
            Console.WriteLine($"¡ALERTA! La consulta tardó {tiempoMs} ms, superando el umbral de
{umbralMs} ms.");
        }
        else
        {
            Console.WriteLine($"Consulta dentro del umbral: {tiempoMs} ms (umbral: {umbralMs} ms).");
        }
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class DiagnosticoUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public DiagnosticoUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== DIAGNÓSTICO ==="); → muestra la cabecera.

Línea 19: DemostrarMedicionDeTiempo(); → llama al método de medición de tiempo.

Línea 20: DemostrarConteoDeConsultas(); → llama al método de conteo de consultas.

Línea 21: DemostrarDeteccionDeConsultasLentas(); → llama al método de detección de consultas lentas.

Línea 24: private void DemostrarMedicionDeTiempo() → declara el método.

Línea 26: Console.WriteLine("\n--- Medición de tiempo ---"); → muestra la cabecera.

Línea 28: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 29: var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico(); → carga las órdenes.

Línea 30: cronometro.Stop(); → detiene el cronómetro.

Línea 32: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el resultado.

Línea 35: `private void DemostrarConteoDeConsultas()` → declara el método.

Línea 37: `Console.WriteLine("\n--- Conteo de consultas ---");` → muestra la cabecera.

Línea 39: `var contador = 0;` → inicializa el contador.

Línea 40: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 42: `contador++;` → incrementa el contador.

Línea 43: `var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();` → carga las órdenes.

Línea 45: `contador++;` → incrementa el contador.

Línea 46: `var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();` → carga las pendientes.

Línea 48: `contador++;` → incrementa el contador.

Línea 49: `var total = _unidad.Ordenes.ContarOrdenesConDiagnostico();` → cuenta las órdenes.

Línea 51: `cronometro.Stop();` → detiene el cronómetro.

Línea 53: `Console.WriteLine($"Consultas ejecutadas: {contador} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el resultado.

Línea 54: `Console.WriteLine($"Órdenes: {ordenes.Count} | Pendientes: {pendientes.Count} | Total: {total}");` → muestra los datos.

Línea 57: `private void DemostrarDeteccionDeConsultasLentas()` → declara el método.

Línea 59: `Console.WriteLine("\n--- Detección de consultas lentas ---");` → muestra la cabecera.

Línea 61: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 62: `var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();` → carga las órdenes.

Línea 63: `cronometro.Stop();` → detiene el cronómetro.

Línea 65: `var tiempoMs = cronometro.ElapsedMilliseconds;` → obtiene el tiempo.

Línea 66: `var umbralMs = 50;` → declara el umbral.

Línea 68: `if (tiempoMs > umbralMs)` → comprueba si supera el umbral.

Línea 70: `Console.WriteLine($"¡ALERTA! La consulta tardó {tiempoMs} ms, superando el umbral de {umbralMs} ms.");` → muestra la alerta.

Línea 72: `else` → caso contrario.

Línea 74: `Console.WriteLine($"Consulta dentro del umbral: {tiempoMs} ms (umbral: {umbralMs} ms).");` → muestra el mensaje.

Error común: si el contador se incrementa manualmente, se puede olvidar incrementarlo en alguna consulta. Se debe usar un observador de diagnóstico para contar las consultas automáticamente.

| Paso 6: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<DiagnosticoUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 7: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<DiagnosticoUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

Paso 8: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>();
    for (int i = 1; i <= 20; i++)
    {
        ordenes.Add(new OrdenFabricacion
        {
            NumeroOrden = $"OF-2024-{i:D4}",
            Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur",
            Estado = i % 3 == 0 ? "EnProceso" : "Pendiente",
            FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)
        });
    }

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List(); → crea la lista de órdenes.

Línea 8: for (int i = 1; i <= 20; i++) → repite veinte veces.

Línea 10: ordenes.Add(new OrdenFabricacion → crea una orden.

Línea 12: NumeroOrden = \$"OF-2024-{i:D4}", → asigna el número.

Línea 13: Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur", → asigna el cliente.

Línea 14: Estado = i % 3 == 0 ? "EnProceso" : "Pendiente", → asigna el estado.

Línea 15: `FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)` → asigna la fecha.

Línea 16: `});` → cierra la orden.

Línea 17: `}` → cierra el bucle.

Línea 19: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 20: `context.SaveChanges();` → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

Paso 9: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de cada consulta, el número de consultas ejecutadas y la detección de consultas lentas.

Paso 10: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== DIAGNÓSTICO ===

--- Medición de tiempo ---
Órdenes: 20 | Tiempo: 42 ms

--- Conteo de consultas ---
Consultas ejecutadas: 3 | Tiempo: 85 ms
Órdenes: 20 | Pendientes: 13 | Total: 20

--- Detección de consultas lentas ---
Consulta dentro del umbral: 42 ms (umbral: 50 ms).
```

La primera sección muestra el tiempo de la consulta. La segunda sección muestra el número de consultas ejecutadas y el tiempo total. La tercera sección muestra la detección de consultas lentas.

Observaciones: el tiempo de las consultas es bajo porque la tabla tiene pocas filas. El número de consultas es tres: una para cargar las órdenes, otra para cargar las pendientes y otra para contar. La detección de consultas lentas compara el tiempo con el umbral.

Paso 11: Diagnosticar un error común

Modificar el método `DemostrarConteoDeConsultas` para no incrementar el contador manualmente:

```
private void DemostrarConteoDeConsultas()
{
    Console.WriteLine("\n--- Conteo de consultas ---");

    var contador = 0;
    var cronometro = Stopwatch.StartNew();

    var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();
    var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();
    var total = _unidad.Ordenes.ContarOrdenesConDiagnostico();

    cronometro.Stop();

    Console.WriteLine($"Consultas ejecutadas: {contador} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    Console.WriteLine($"Órdenes: {ordenes.Count} | Pendientes: {pendientes.Count} | Total: {total}");
}
```

Resultado esperado: el contador muestra cero porque no se ha incrementado. El número de consultas no es correcto.

Solución: usar un observador de diagnóstico para contar las consultas automáticamente.

```
private int _contadorConsultas;

private void DemostrarConteoDeConsultasConObservador()
{
    Console.WriteLine("\n--- Conteo de consultas con observador ---");

    _contadorConsultas = 0;
    var observer = new EfCoreDiagnosticObserver(message =>
    {
        if (message.StartsWith("Comando ejecutado")) _contadorConsultas++;
    });
    DiagnosticListener.AllListeners.Subscribe(observer);

    var cronometro = Stopwatch.StartNew();
    var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();
    var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();
    var total = _unidad.Ordenes.ContarOrdenesConDiagnostico();
    cronometro.Stop();

    Console.WriteLine($"Consultas ejecutadas: {_contadorConsultas} | Tiempo:
{cronometro.ElapsedMilliseconds} ms");
}
Resultado esperado con la solución: el contador muestra el número real de consultas ejecutadas.
```

Errores comunes del ejercicio

Error	Causa	Solución
Contador incorrecto	Se incrementa manualmente	Usar un observador de diagnóstico
Logging sin filtrar	Se registran todos los mensajes	Filtrar por categoría y nivel
Datos sensibles expuestos	EnableSensitiveDataLogging en producción	Deshabilitar en producción
Errores detallados en producción	EnableDetailedErrors en producción	Deshabilitar en producción
Observador sin filtrar	Se suscribe a todos los listeners	Comprobar el nombre del listener
Medición imprecisa	Se mide el tiempo de materialización	Medir solo el tiempo de la consulta

Reto resuelto: Detectar consultas lentas con un observador de diagnóstico

Reto: Crear un observador de diagnóstico que detecte las consultas que tardan más de un umbral y las registre. Aplicarlo a las consultas del repositorio.

Solución paso a paso:

Paso 1: Modificar el observador para detectar consultas lentas:

```
public class EfCoreCommandObserver : IObservable<KeyValuePair<string, object>>
{
    private readonly Action<string> _log;
    private readonly long _umbralMs;

    public EfCoreCommandObserver(Action<string> log, long umbralMs)
    {
        _log = log;
        _umbralMs = umbralMs;
    }

    public void OnNext(KeyValuePair<string, object> value)
    {
        if (value.Key == "Microsoft.EntityFrameworkCore.Database.Command.CommandExecuted")
        {
            var tipo = value.Value.GetType();
            var propiedadDuracion = tipo.GetProperty("Duration");
            if (propiedadDuracion != null)
            {
                var duracion = (TimeSpan)propiedadDuracion.GetValue(value.Value)!;
                if (duracion.TotalMilliseconds > _umbralMs)
                {
                    _log($"Consulta lenta detectada: {duracion.TotalMilliseconds} ms");
                }
            }
        }
    }

    public void OnError(Exception error) { }
    public void OnCompleted() { }
}
```

Línea 1: `public class EfCoreCommandObserver : IObservable<KeyValuePair<string, object>>` → declara el observador.

Línea 3: `private readonly Action _log;` → campo para el delegado de logging.

Línea 4: `private readonly long _umbralMs;` → campo para el umbral.

Línea 6: `public EfCoreCommandObserver(Action log, long umbralMs)` → constructor.

Línea 8: `_log = log;` → asigna el delegado.

Línea 9: `_umbralMs = umbralMs;` → asigna el umbral.

Línea 12: `public void OnNext(KeyValuePair<string, object> value)` → implementa `OnNext`.

Línea 14: `if (value.Key == "Microsoft.EntityFrameworkCore.Database.Command.CommandExecuted")` → comprueba el evento.

Línea 16: `var tipo = value.Value.GetType();` → obtiene el tipo del evento.

Línea 17: `var propiedadDuracion = tipo.GetProperty("Duration");` → obtiene la propiedad `Duration`.

Línea 18: `if (propiedadDuracion != null)` → comprueba si existe.

Línea 20: `var duracion = (TimeSpan)propiedadDuracion.GetValue(value.Value)!;` → obtiene la duración.

Línea 21: `if (duracion.TotalMilliseconds > _umbralMs)` → comprueba si supera el umbral.

Línea 23: `_log($"Consulta lenta detectada: {duracion.TotalMilliseconds} ms");` → registra la consulta lenta.

Paso 2: Añadir la demostración en el caso de uso:

```
private void DemostrarDeteccionConObservador()
{
    Console.WriteLine("\n--- Detección de consultas lentas con observador ---");

    var observer = new EfCoreDiagnosticObserver(message => Console.WriteLine(message));
    DiagnosticListener.AllListeners.Subscribe(observer);

    var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();
    var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();
}
```

Línea 1: `private void DemostrarDeteccionConObservador()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Detección de consultas lentas con observador ---");` → muestra la cabecera.

Línea 5: `var observer = new EfCoreDiagnosticObserver(message => Console.WriteLine(message));` → crea el observador.

Línea 6: `DiagnosticListener.AllListeners.Subscribe(observer);` → suscribe el observador.

Línea 8: `var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();` → ejecuta la consulta.

Línea 9: `var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();` → ejecuta la consulta.

Paso 3: Llamar al método desde Ejecutar:

```
DemostrarDeteccionConObservador();
```

Paso 4: Ejecutar dotnet run y verificar que las consultas lentas se detectan.

Resultado esperado: las consultas que superan el umbral se registran con el mensaje Consulta lenta detectada.

Analogía final

El diagnóstico en una acería es como el sistema de control de calidad de la planta. Los logs son como el registro de cada operación: qué se hizo, cuándo y con qué parámetros. Las métricas son como los indicadores del panel de control: cuántas operaciones se han hecho, cuánto han tardado, cuántas han fallado. El plan de ejecución es como el plano que muestra cómo se ha ejecutado cada operación: si se ha usado el horno correcto, si se ha tardado más de lo previsto. Las herramientas de diagnóstico son como los instrumentos de medición: termómetros, manómetros, cronómetros. Sin diagnóstico, los problemas de rendimiento son invisibles. Con diagnóstico, se detectan antes de que afecten a la producción. Así funciona el diagnóstico en EF Core: se observa, se mide y se analiza para identificar problemas y aplicar soluciones.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado el observador de diagnóstico `EfCoreDiagnosticObserver`.

Añadido los métodos de diagnóstico a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `DiagnosticoUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con veinte órdenes.

Ejecutado las demostraciones de medición de tiempo, conteo de consultas y detección de consultas lentas.

Analizado la salida y comprobado el tiempo y el número de consultas.

Diagnosticado el error de incrementar el contador manualmente.

Creado el observador de consultas lentas.

Punto 4.12 - Estrategias de optimización y checklist de rendimiento

Práctica

Ejercicio: Aplicar el checklist de rendimiento al proyecto AceriaData. Crear un método que recorra todas las consultas del repositorio y verifique las condiciones del checklist. Medir el impacto de las optimizaciones. Documentar las decisiones. Cerrar el Módulo 4 con un caso práctico completo de optimización.

Contexto del proyecto: En el punto 4.11 se estudió el diagnóstico, configurando logs detallados y observadores de diagnóstico. En este punto se consolidan todas las técnicas del Módulo 4 en un checklist de rendimiento que se aplica al proyecto AceriaData. Esta consolidación cierra el Módulo 4 y prepara el proyecto para el Módulo 5.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos del checklist a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerSinOptimizar();
    List<OrdenFabricacion> ObtenerOptimizado();
    List<OrdenResumenDto> ObtenerResumenOptimizado();
    List<OrdenFabricacion> ObtenerConRelacionesOptimizado();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerSinOptimizar(); → método sin optimización.

Línea 11: List ObtenerOptimizado(); → método con optimización de tracking y proyección.

Línea 12: List ObtenerResumenOptimizado(); → método con proyección a DTO.

Línea 13: List ObtenerConRelacionesOptimizado(); → método con Include y AsSplitQuery.

Línea 15: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 16: void Eliminar(OrdenFabricacion orden); → método que elimina una orden.

Error común: si los métodos devuelven IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 3: Implementar los métodos del checklist en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs:

```

public List<OrdenFabricacion> ObtenerSinOptimizar()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

/// <summary>
/// Obtiene las órdenes con optimización de tracking.
/// Usa AsNoTracking porque es una consulta de solo lectura.
/// </summary>
public List<OrdenFabricacion> ObtenerOptimizado()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

/// <summary>
/// Obtiene los resúmenes de las órdenes.
/// Usa proyección para reducir el volumen de datos transferidos.
/// Usa AsNoTracking porque es una consulta de solo lectura.
/// </summary>
public List<OrdenResumenDto> ObtenerResumenOptimizado()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

/// <summary>
/// Obtiene las órdenes con planchas y detalle.
/// Usa Include para evitar N+1.
/// Usa AsSplitQuery para evitar el producto cartesiano.
/// Usa AsNoTrackingWithIdentityResolution para evitar instancias duplicadas.
/// </summary>
public List<OrdenFabricacion> ObtenerConRelacionesOptimizado()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
    
```

Línea 1: public List ObtenerSinOptimizar() → declara el método sin optimización.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 5: `.ToList();` → materializa la consulta con tracking.

Línea 8: `///`

→ inicio del comentario XML.

Línea 9: `///` Obtiene las órdenes con optimización de tracking. → descripción del método.

Línea 10: `///` Usa `AsNoTracking` porque es una consulta de solo lectura. → explicación del uso de `AsNoTracking`.

Línea 11: `///`

→ cierre del comentario XML.

Línea 12: `public List ObtenerOptimizado()` → declara el método optimizado.

Línea 14: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 15: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 16: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 17: `.ToList();` → materializa la consulta sin tracking.

Línea 20: `///`

→ inicio del comentario XML.

Línea 21: `///` Obtiene los resúmenes de las órdenes. → descripción del método.

Línea 22: `///` Usa proyección para reducir el volumen de datos transferidos. → explicación del uso de proyección.

Línea 23: `///` Usa `AsNoTracking` porque es una consulta de solo lectura. → explicación del uso de `AsNoTracking`.

Línea 24: `///`

→ cierre del comentario XML.

Línea 25: `public List ObtenerResumenOptimizado()` → declara el método.

Línea 27: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 28: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 29: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 30: `.Select(o => new OrdenResumenDto` → proyecta al DTO.

Línea 31: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 32: `Cliente = o.Cliente,` → asigna el cliente.

Línea 33: `Estado = o.Estado,` → asigna el estado.

Línea 34: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 35: `}` → cierra la proyección.

Línea 36: `.ToList();` → materializa la consulta.

Línea 39: `///`

→ inicio del comentario XML.

Línea 40: `///` Obtiene las órdenes con planchas y detalle. → descripción del método.

Línea 41: `///` Usa `Include` para evitar N+1. → explicación del uso de `Include`.

Línea 42: /// Usa AsSplitQuery para evitar el producto cartesiano. → explicación del uso de AsSplitQuery.

Línea 43: /// Usa AsNoTrackingWithIdentityResolution para evitar instancias duplicadas. → explicación del uso de AsNoTrackingWithIdentityResolution.

Línea 44: ///

→ cierre del comentario XML.

Línea 45: public List ObtenerConRelacionesOptimizado() → declara el método.

Línea 47: return _context.OrdenesFabricacion → inicia la consulta.

Línea 48: .AsNoTrackingWithIdentityResolution() → aplica resolución de identidad.

Línea 49: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 50: .Include(o => o.Detalle) → incluye la referencia al detalle.

Línea 51: .AsSplitQuery() → divide la consulta en varias.

Línea 52: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 53: .ToList(); → materializa la consulta.

Error común: si se olvida el comentario XML, otros desarrolladores no entienden por qué se aplicó la técnica. Se deben documentar todas las decisiones.

| Paso 4: Crear el caso de uso del checklist de rendimiento

Crear el archivo src/AceriaData.Application/UseCases/ChecklistRendimientoUseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ChecklistRendimientoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ChecklistRendimientoUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CHECKLIST DE RENDIMIENTO ===");

        MostrarChecklist();
        CompararSinOptimizarVsOptimizado();
        CompararEntidadesVsProyeccion();
        CompararSinIncludeVsConInclude();
    }

    private void MostrarChecklist()
    {
        Console.WriteLine("\n-- Checklist de rendimiento --");
        Console.WriteLine("1. ¿Se usa AsNoTracking en consultas de solo lectura?");
        Console.WriteLine("2. ¿Se proyectan solo las columnas necesarias?");
        Console.WriteLine("3. ¿Se evita el problema N+1 con Include?");
        Console.WriteLine("4. ¿Se evita el producto cartesiano con AsSplitQuery?");
        Console.WriteLine("5. ¿Se aplican filtros y paginación en el servidor?");
        Console.WriteLine("6. ¿Se evitan funciones en Where que impidan índices?");
        Console.WriteLine("7. ¿Se usan Compiled Queries en consultas frecuentes?");
        Console.WriteLine("8. ¿Se miden los tiempos y se cuentan las consultas?");
    }

    private void CompararSinOptimizarVsOptimizado()
    {

```

```

        Console.WriteLine("\n--- Sin optimizar vs optimizado (tracking) ---");

        var cronometroSin = Stopwatch.StartNew();
        var ordenesSin = _unidad.Ordenes.ObtenerSinOptimizar();
        cronometroSin.Stop();

        var cronometroCon = Stopwatch.StartNew();
        var ordenesCon = _unidad.Ordenes.ObtenerOptimizado();
        cronometroCon.Stop();

        Console.WriteLine($"Sin optimizar: {cronometroSin.ElapsedMilliseconds} ms | Órdenes:
{ordenesSin.Count}");
        Console.WriteLine($"Optimizado: {cronometroCon.ElapsedMilliseconds} ms | Órdenes:
{ordenesCon.Count}");
    }

    private void CompararEntidadesVsProyeccion()
    {
        Console.WriteLine("\n--- Entidades completas vs proyección ---");

        var cronometroEntidades = Stopwatch.StartNew();
        var ordenesEntidades = _unidad.Ordenes.ObtenerOptimizado();
        cronometroEntidades.Stop();

        var cronometroProyeccion = Stopwatch.StartNew();
        var resúmenes = _unidad.Ordenes.ObtenerResumenOptimizado();
        cronometroProyeccion.Stop();

        Console.WriteLine($"Entidades completas: {cronometroEntidades.ElapsedMilliseconds} ms | Órdenes:
{ordenesEntidades.Count}");
        Console.WriteLine($"Proyección: {cronometroProyeccion.ElapsedMilliseconds} ms | Resúmenes:
{resúmenes.Count}");
    }

    private void CompararSinIncludeVsConInclude()
    {
        Console.WriteLine("\n--- Sin Include vs con Include y AsSplitQuery ---");

        var cronometroSinInclude = Stopwatch.StartNew();
        var ordenesSinInclude = _unidad.Ordenes.ObtenerOptimizado();
        cronometroSinInclude.Stop();

        var cronometroConInclude = Stopwatch.StartNew();
        var ordenesConInclude = _unidad.Ordenes.ObtenerConRelacionesOptimizado();
        cronometroConInclude.Stop();

        Console.WriteLine($"Sin Include: {cronometroSinInclude.ElapsedMilliseconds} ms | Órdenes:
{ordenesSinInclude.Count}");
        Console.WriteLine($"Con Include y AsSplitQuery: {cronometroConInclude.ElapsedMilliseconds} ms |
Órdenes: {ordenesConInclude.Count}");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class ChecklistRendimientoUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public ChecklistRendimientoUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== CHECKLIST DE RENDIMIENTO ==="); → muestra la cabecera.

Línea 19: `MostrarChecklist();` → llama al método que muestra el checklist.

Línea 20: `CompararSinOptimizarVsOptimizado();` → llama al método de comparación de tracking.

Línea 21: `CompararEntidadesVsProyeccion();` → llama al método de comparación de proyección.

Línea 22: `CompararSinIncludeVsConInclude();` → llama al método de comparación de Include.

Línea 25: `private void MostrarChecklist()` → declara el método.

Línea 27: `Console.WriteLine("\n--- Checklist de rendimiento ---");` → muestra la cabecera.

Línea 28: `Console.WriteLine("1. ¿Se usa AsNoTracking en consultas de solo lectura?");` → muestra la primera comprobación.

Línea 29: `Console.WriteLine("2. ¿Se proyectan solo las columnas necesarias?");` → muestra la segunda comprobación.

Línea 30: `Console.WriteLine("3. ¿Se evita el problema N+1 con Include?");` → muestra la tercera comprobación.

Línea 31: `Console.WriteLine("4. ¿Se evita el producto cartesiano con AsSplitQuery?");` → muestra la cuarta comprobación.

Línea 32: `Console.WriteLine("5. ¿Se aplican filtros y paginación en el servidor?");` → muestra la quinta comprobación.

Línea 33: `Console.WriteLine("6. ¿Se evitan funciones en Where que impidan índices?");` → muestra la sexta comprobación.

Línea 34: `Console.WriteLine("7. ¿Se usan Compiled Queries en consultas frecuentes?");` → muestra la séptima comprobación.

Línea 35: `Console.WriteLine("8. ¿Se miden los tiempos y se cuentan las consultas?");` → muestra la octava comprobación.

Línea 38: `private void CompararSinOptimizarVsOptimizado()` → declara el método.

Línea 40: `Console.WriteLine("\n--- Sin optimizar vs optimizado (tracking) ---");` → muestra la cabecera.

Línea 42: `var cronometroSin = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 43: `var ordenesSin = _unidad Ordenes.ObtenerSinOptimizar();` → carga las órdenes sin optimizar.

Línea 44: `cronometroSin.Stop();` → detiene el cronómetro.

Línea 46: `var cronometroCon = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 47: `var ordenesCon = _unidad Ordenes.ObtenerOptimizado();` → carga las órdenes optimizadas.

Línea 48: `cronometroCon.Stop();` → detiene el cronómetro.

Línea 50: `Console.WriteLine($"Sin optimizar: {cronometroSin.ElapsedMilliseconds} ms | Órdenes: {ordenesSin.Count}");` → muestra el tiempo sin optimizar.

Línea 51: `Console.WriteLine($"Optimizado: {cronometroCon.ElapsedMilliseconds} ms | Órdenes: {ordenesCon.Count}");` → muestra el tiempo optimizado.

Línea 54: `private void CompararEntidadesVsProyeccion()` → declara el método.

Línea 56: `Console.WriteLine("\n--- Entidades completas vs proyección ---");` → muestra la cabecera.

Línea 58: `var cronometroEntidades = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 59: `var ordenesEntidades = _unidad Ordenes.ObtenerOptimizado();` → carga las entidades completas.

Línea 60: `cronometroEntidades.Stop();` → detiene el cronómetro.

Línea 62: `var cronometroProyeccion = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 63: `var resúmenes = _unidad Ordenes.ObtenerResumenOptimizado();` → carga los resúmenes.

Línea 64: `cronometroProyeccion.Stop();` → detiene el cronómetro.

Línea 66: `Console.WriteLine($"Entidades completas: {cronometroEntidades.ElapsedMilliseconds} ms | Órdenes: {ordenesEntidades.Count}");` → muestra el tiempo de entidades completas.

Línea 67: `Console.WriteLine($"Proyección: {cronometroProyeccion.ElapsedMilliseconds} ms | Resúmenes: {resúmenes.Count}");` → muestra el tiempo de proyección.

Línea 70: `private void CompararSinIncludeVsConInclude()` → declara el método.

Línea 72: `Console.WriteLine("\n--- Sin Include vs con Include y AsSplitQuery ---");` → muestra la cabecera.

Línea 74: `var cronometroSinInclude = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 75: `var ordenesSinInclude = _unidad Ordenes.ObtenerOptimizado();` → carga las órdenes sin Include.

Línea 76: `cronometroSinInclude.Stop();` → detiene el cronómetro.

Línea 78: `var cronometroConInclude = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 79: `var ordenesConInclude = _unidad Ordenes.ObtenerConRelacionesOptimizado();` → carga las órdenes con Include.

Línea 80: `cronometroConInclude.Stop();` → detiene el cronómetro.

Línea 82: `Console.WriteLine($"Sin Include: {cronometroSinInclude.ElapsedMilliseconds} ms | Órdenes: {ordenesSinInclude.Count}");` → muestra el tiempo sin Include.

Línea 83: `Console.WriteLine($"Con Include y AsSplitQuery: {cronometroConInclude.ElapsedMilliseconds} ms | Órdenes: {ordenesConInclude.Count}");` → muestra el tiempo con Include.

Error común: si se ejecutan las consultas en el mismo `DbContext`, las entidades de las consultas anteriores permanecen en el `Change Tracker`. Se debe usar un `DbContext` distinto para cada consulta si se quiere medir el tiempo de forma aislada.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ChecklistRendimientoUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ChecklistRendimientoUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

Paso 7: Insertar datos de prueba con varias órdenes y planchas

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes y planchas:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>();
    for (int i = 1; i <= 20; i++)
    {
        ordenes.Add(new OrdenFabricacion
        {
            NumeroOrden = $"OF-2024-{i:D4}",
            Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur",
            Estado = i % 3 == 0 ? "EnProceso" : "Pendiente",
            FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)
        });
    }

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();

    var planchas = new List<PlanchaAcero>();
    foreach (var orden in ordenes)
    {
        planchas.Add(new PlanchaAcero { OrdenId = orden.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000,
        Peso = 370.5m, Activa = true });
    }

    context.PlanchasAcero.AddRange(planchas);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base de datos.

Línea 5: `context.Database.Migrate();` → aplica las migraciones.

Línea 7: `var ordenes = new List();` → crea la lista de órdenes.

Línea 8: `for (int i = 1; i <= 20; i++)` → repite veinte veces.

Línea 10: `ordenes.Add(new OrdenFabricacion` → crea una orden.

Línea 12: `NumeroOrden = $"OF-2024-{i:D4}"`, → asigna el número.

Línea 13: `Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur"`, → asigna el cliente.

Línea 14: `Estado = i % 3 == 0 ? "EnProceso" : "Pendiente"`, → asigna el estado.

Línea 15: `FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)` → asigna la fecha.

Línea 16: `});` → cierra la orden.

Línea 17: `}` → cierra el bucle.

Línea 19: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 20: `context.SaveChanges();` → inserta las órdenes.

Línea 22: `var planchas = new List();` → crea la lista de planchas.

Línea 23: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 25: `planchas.Add(new PlanchaAcero { ... });` → crea una plancha.

Línea 26: `}` → cierra el bucle.

Línea 28: `context.PlanchasAcero.AddRange(planchas);` → registra las planchas.

Línea 29: `context.SaveChanges();` → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con el checklist y las comparaciones de rendimiento. Se observa el tiempo de cada técnica.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== CHECKLIST DE RENDIMIENTO ===

--- Checklist de rendimiento ---
1\ ¿Se usa AsNoTracking en consultas de solo lectura?
2\ ¿Se proyectan solo las columnas necesarias?
3\ ¿Se evita el problema N+1 con Include?
4\ ¿Se evita el producto cartesiano con AsSplitQuery?
5\ ¿Se aplican filtros y paginación en el servidor?
6\ ¿Se evitan funciones en Where que impidan índices?
7\ ¿Se usan Compiled Queries en consultas frecuentes?
8\ ¿Se miden los tiempos y se cuentan las consultas?

--- Sin optimizar vs optimizado (tracking) ---
Sin optimizar: 55 ms | Órdenes: 20
Optimizado: 45 ms | Órdenes: 20

--- Entidades completas vs proyección ---
Entidades completas: 45 ms | Órdenes: 20
Proyección: 38 ms | Resúmenes: 20

--- Sin Include vs con Include y AsSplitQuery ---
Sin Include: 45 ms | Órdenes: 20
Con Include y AsSplitQuery: 58 ms | Órdenes: 20
```

La primera sección muestra el checklist. La segunda sección muestra la comparación sin optimizar vs optimizado. La tercera sección muestra la comparación de entidades completas vs proyección. La cuarta sección muestra la comparación sin Include vs con Include.

Observaciones: la optimización de tracking es más rápida. La proyección es más rápida que las entidades completas. La consulta con Include y AsSplitQuery es más lenta porque carga más datos, pero evita el problema N+1 en escenarios reales. El beneficio del Include se ve cuando se necesitan las entidades relacionadas.

Paso 10: Diagnosticar un error común

Modificar el método `ObtenerConRelacionesOptimizado` para eliminar `AsSplitQuery`:

```
public List<OrdenFabricacion> ObtenerConRelacionesOptimizado()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado: el código funciona, pero la consulta genera un producto cartesiano porque se incluyen dos colecciones. El número de filas transferidas es mayor que el número de planchas más el número de detalles.

Solución: añadir AsSplitQuery para dividir la consulta en varias.

```
public List<OrdenFabricacion> ObtenerConRelacionesOptimizado()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado con la solución: la consulta se divide en varias y no se produce el producto cartesiano.

Errores comunes del ejercicio

Error	Causa	Solución
Tracking innecesario	No se aplica AsNoTracking	Aplicar AsNoTracking en consultas de solo lectura
Over-fetching	Se cargan entidades completas	Usar proyecciones
N+1	No se aplica Include	Aplicar Include en consultas con relaciones
Producto cartesiano	Se incluyen varias colecciones sin AsSplitQuery	Aplicar AsSplitQuery
Consultas en memoria	Se materializa antes de filtrar	Filtrar antes de materializar
Funciones en Where	Se aplican funciones sobre columnas	Comparar directamente o usar collation
Falta de paginación	Se cargan todos los resultados	Aplicar Skip/Take o keyset pagination
Compilación repetida	No se usan Compiled Queries	Aplicar Compiled Queries en consultas frecuentes
Sin diagnóstico	No se miden los tiempos	Configurar logging y observadores

Reto resuelto: Aplicar el checklist a una consulta completa

Reto: Tomar una consulta sin optimizar que cargue las órdenes con sus planchas y detalles, y aplicar el checklist completo: AsNoTracking, proyección, Include, AsSplitQuery, filtro, paginación y documentación. Medir el tiempo antes y después.

Solución paso a paso:

Paso 1: Consulta sin optimizar:

```
public List<OrdenFabricacion> ObtenerSinOptimizarCompleta()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .ToList();
}
```

Línea 1: public List ObtenerSinOptimizarCompleta() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 5: .Include(o => o.Detalle) → incluye la referencia al detalle.

Línea 6: .ToList(); → materializa la consulta.

Paso 2: Consulta optimizada con el checklist completo:

```
/// <summary>
/// Obtiene las órdenes pendientes con planchas y detalle.
/// Usa AsNoTrackingWithIdentityResolution para evitar tracking y duplicados.
/// Usa Include para evitar N+1.
/// Usa AsSplitQuery para evitar el producto cartesiano.
/// Usa filtro por estado para reducir el número de filas.
/// Usa paginación para limitar el número de resultados.
/// </summary>
public List<OrdenFabricacion> ObtenerOptimizadoCompleta(int pagina, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .ToList();
}
```

Línea 1: ///

→ inicio del comentario XML.

Línea 2: /// Obtiene las órdenes pendientes con planchas y detalle. → descripción del método.

Línea 3: /// Usa AsNoTrackingWithIdentityResolution para evitar tracking y duplicados. → explicación del uso de AsNoTrackingWithIdentityResolution.

Línea 4: /// Usa Include para evitar N+1. → explicación del uso de Include.

Línea 5: /// Usa AsSplitQuery para evitar el producto cartesiano. → explicación del uso de AsSplitQuery.

Línea 6: /// Usa filtro por estado para reducir el número de filas. → explicación del filtro.

Línea 7: /// Usa paginación para limitar el número de resultados. → explicación de la paginación.

Línea 8: ///

→ cierre del comentario XML.

Línea 9: public List ObtenerOptimizadoCompleta(int pagina, int tamanoPagina) → declara el método.

Línea 11: return _context.OrdenesFabricacion → inicia la consulta.

Línea 12: `.AsNoTrackingWithIdentityResolution()` → aplica resolución de identidad.

Línea 13: `.Where(o => o.Estado == "Pendiente")` → filtra por estado.

Línea 14: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 15: `.ThenBy(o => o.Id)` → ordena por Id.

Línea 16: `.Skip((pagina - 1) * tamanoPagina)` → salta las páginas anteriores.

Línea 17: `.Take(tamanoPagina)` → toma el tamaño de la página.

Línea 18: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 19: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 20: `.AsSplitQuery()` → divide la consulta en varias.

Línea 21: `.ToList();` → materializa la consulta.

Paso 3: Medir el tiempo antes y después:

```
private void CompararCompleta()
{
    Console.WriteLine("\n--- Completa sin optimizar vs optimizada ---");

    var cronometroSin = Stopwatch.StartNew();
    var ordenesSin = _unidad.Ordenes.ObtenerSinOptimizarCompleta();
    cronometroSin.Stop();

    var cronometroCon = Stopwatch.StartNew();
    var ordenesCon = _unidad.Ordenes.ObtenerOptimizadoCompleta(1, 10);
    cronometroCon.Stop();

    Console.WriteLine($"Sin optimizar: {cronometroSin.ElapsedMilliseconds} ms | Órdenes: {ordenesSin.Count}");
    Console.WriteLine($"Optimizado: {cronometroCon.ElapsedMilliseconds} ms | Órdenes: {ordenesCon.Count}");
}
```

Línea 1: `private void CompararCompleta()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Completa sin optimizar vs optimizada ---");` → muestra la cabecera.

Línea 5: `var cronometroSin = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 6: `var ordenesSin = _unidad.Ordenes.ObtenerSinOptimizarCompleta();` → carga sin optimizar.

Línea 7: `cronometroSin.Stop();` → detiene el cronómetro.

Línea 9: `var cronometroCon = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 10: `var ordenesCon = _unidad.Ordenes.ObtenerOptimizadoCompleta(1, 10);` → carga optimizada.

Línea 11: `cronometroCon.Stop();` → detiene el cronómetro.

Línea 13: `Console.WriteLine($"Sin optimizar: {cronometroSin.ElapsedMilliseconds} ms | Órdenes: {ordenesSin.Count}");` → muestra el tiempo sin optimizar.

Línea 14: `Console.WriteLine($"Optimizado: {cronometroCon.ElapsedMilliseconds} ms | Órdenes: {ordenesCon.Count}");` → muestra el tiempo optimizado.

Paso 4: Llamar al método desde Ejecutar:

```
CompararCompleta();
```

| Paso 5: Ejecutar dotnet run y comparar los tiempos.

Resultado esperado: la consulta optimizada es más rápida y transfiere menos datos. La documentación XML explica las decisiones aplicadas.

| Analogía final

El checklist de rendimiento en una acería es como la lista de comprobaciones que el jefe de planta revisa antes de cada colada. Antes de encender el horno, comprueba la temperatura. Antes de laminar, comprueba el espesor. Antes de enviar, comprueba el peso. Cada comprobación evita un problema. El checklist de EF Core es igual: antes de ejecutar una consulta, se comprueba si se usa AsNoTracking, si se proyectan las columnas necesarias, si se evita el N+1, si se evita el producto cartesiano, si se aplican filtros y paginación, si se evitan funciones en Where, si se usan Compiled Queries y si se miden los tiempos. Cada comprobación evita un problema de rendimiento. La estrategia de optimización es como el plan de producción: se mide, se identifica, se aplica, se verifica y se documenta. La optimización es un ciclo continuo, no una tarea puntual. Así funciona el checklist de rendimiento en EF Core: se aplica a cada consulta y se documenta cada decisión.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos del checklist a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio con documentación XML.

Creado el caso de uso ChecklistRendimientoUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con veinte órdenes y planchas.

Ejecutado el checklist y las comparaciones de rendimiento.

Analizado la salida y comprobado el impacto de cada técnica.

Diagnosticado el error de eliminar AsSplitQuery.

Creado el método ObtenerOptimizadoCompleta con el checklist aplicado.

Resumen del estado del proyecto AceriaData al final del Módulo 4 Al final del Módulo 4, el proyecto AceriaData tiene:

La arquitectura limpia configurada en cuatro proyectos: AceriaData.Domain, AceriaData.Application, AceriaData.Infraestructure y AceriaData.Console.

El dominio con las entidades OrdenFabricacion, PlanchaAcero, Aleacion, EstadoOrden, DetalleOrden, CertificadoCalidad y OrdenAleacion.

La aplicación con las interfaces IOrdenRepositorio y IUnidadDeTrabajo, los casos de uso de todos los módulos y los DTOs.

La infraestructura con el AceriaDbContext, las configuraciones de Fluent API, las implementaciones de repositorios, la unidad de trabajo, las consultas compiladas y el observador de diagnóstico.

La consola con el método Main y la configuración del contenedor de dependencias.

El modelo de datos completo con relaciones uno a muchos, uno a uno y muchos a muchos.

Las claves primarias, alternativas y compuestas configuradas.

Los índices y las restricciones configurados.

El Soft Delete implementado con filtros globales.

La arquitectura limpia aplicada con la regla de dependencia respetada.

Las consultas LINQ con filtros, ordenaciones, proyecciones, agregaciones, agrupaciones, joins, carga Eager, carga Lazy, carga Explicit y composición de consultas.

Las buenas prácticas de acceso a datos aplicadas.

La optimización del rendimiento aplicada: análisis del SQL, Tracking vs No Tracking, resolución de identidad, solución al N+1, over-fetching, consultas ineficientes, Split Queries, Compiled Queries, paginación eficiente, diagnóstico y checklist de rendimiento
